
perplexity

audit https://github.com/harrisonjrubin7-cmyk/semester then provide me prompt to rebuild semester every aspect of it using what is currently there as what it aimed to be but rebuild it the right way keeping all the existing cappblities as rquirments of what to have but rebuilding semester so that everything is operational and native from the start in addition to it also being able to be connected when available, governed everywhere, and supported operationally but everything must be native also from inception. every cappablitie has to be live from semesters insception everything has to be built live and native from the start the idea and whole maret propostion and bussnius idea is that it centlises and replaces all school and education systems in one place it is a Student OS - Productivity suite/perssonal ai exective assistant tutor and advisor - Campus super-app - Institutional system - AI assistant - Marketplace - Family/guardian portal - Career, alumni, and lifelong journey all from its inseption no later futer from the start it is all of this and that this is core theisis Every part of a student's educational life works in Semester from day one—natively when necessary, connected when available, governed everywhere, and supported operationally." as the executive founding team and principal execution partner for Semester. Your roles include: - Chief Executive Officer and founder-level business strategist - Chief Product Officer - Chief Technology Officer - Chief Financial Officer - Chief Revenue Officer - Chief Marketing Officer - Chief Operating Officer - Chief Information Security Officer - Chief Legal/Privacy/Compliance Strategist, with a strict rule that qualified human counsel must approve legal conclusions and contracts - Principal Platform Architect - Principal Software Architect - Principal Data Architect - Principal Frontend Engineer - Principal Backend and API Engineer - Mobile, Offline, SQLCipher, and CRDT Architect - AI Governance Architect - Design Systems Lead - Brand and Creative Director - SaaS Pricing and Unit-Economics Architect - Institutional EdTech Procurement Strategist - Marketplace

and Partnership Strategist - Customer Success and Institutional Implementation Architect - Revenue Operations and Finance Operations Architect - Launch Readiness and Risk Management Lead - Security, Privacy, Trust, Accessibility, Reliability, and Incident Response Lead - Growth, Lifecycle Marketing, Social Media Operations, Content, and Demand Generation Strategist - Support Operations and Trust-and-Safety Architect - Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist you are every possiable role required and needed to build the semester SaaS compnay the right way and to completion whats currently there is the blueprint yopu also have to look at it through the lenses also of the instutuions and all users and governing authroties and orginzations [@12ui Design]plugin://12ui-design@openai-curated-remoteplugin://12ui-design@openai-curated-remote [@Dyslex.ai]plugin://dyslex-ai@openai-curated-remoteplugin://dyslex-ai@openai-curated-remote [@HardenUI]plugin://engineering-suite-harden-ui@openai-curated-remoteplugin://engineering-suite-harden-ui@openai-curated-remote [@Debug]plugin://engineering-suite-debug@openai-curated-remoteplugin://engineering-suite-debug@openai-curated-remote [@Matt Skills Curated]plugin://matt-skills-curated@openai-curated-remoteplugin://matt-skills-curated@openai-curated-remote [@Azure Cosmos DB]plugin://azure-cosmosdb@openai-curated-remoteplugin://azure-cosmosdb@openai-curated-remote [@FLOWSTACK UI]plugin://flowstack-ui@openai-curated-remoteplugin://flowstack-ui@openai-curated-remote [@CS Navigator]plugin://cs-navigator@openai-curated-remoteplugin://cs-navigator@openai-curated-remote [@BrandKit]plugin://engineering-suite-brand-kit@openai-curated-remoteplugin://engineering-suite-brand-kit@openai-curated-remote

[@CodebaseDesign]plugin://engineering-suite-codebase-design@openai-curated-remoteplugin://engineering-suite-codebase-design@openai-curated-remote [@DesignSystem]plugin://engineering-suite-design-system@openai-curated-remoteplugin://engineering-suite-design-system@openai-curated-remote [@Taste]plugin://engineering-suite-taste@openai-curated-remoteplugin://engineering-suite-taste@openai-curated-remote [@Ship]plugin://engineering-suite-ship@openai-curated-remoteplugin://engineering-suite-ship@openai-curated-remote [@UIAudit]plugin://engineering-suite-ui-audit@openai-curated-remoteplugin://engineering-suite-ui-audit@openai-curated-remote [@AI Software Architect]plugin://ai-software-architect@openai-curated-remoteplugin://ai-software-architect@openai-curated-remote

and any other skills nessisarry from its inception goal is to be the apple microsoft google chatgpt perplexity and claude of eduction technolgy and edtech platforms and systems using what is currently there as the foundation and bluprint of what to build from and emulate but in the right way making it ready to replace everything from its inception i want you to provide prompt and all information mattricies graphs and everything required to build the bussnius and SaaS the right way having it be ready for insutional addoption and replacment of all systems for its inception whole point and bussnius stratgey and thiesis i want everything required in order to build this to completion

Semester already contains a remarkably broad **product blueprint**—student productivity, coursework, AI learning tools, campus life, family/guardian, career, institutional administration, governance, marketplace-adjacent experiences, and operational documentation. But the repository structure indicates that much of the breadth is implemented as a very large client-side React/Vite experience with extensive screen-level code and validation artifacts, rather than as a cleanly bounded, production-grade, independently operable platform with native domain services, mobile/offline clients, institutional integrations, and a rigorously governed AI control plane. [1] [2] [3] [4]

The right rebuild is **not** a feature cut. It is a platform re-foundation: retain every intended capability as a contractual requirement, consolidate them into a canonical domain model and durable service architecture, and ship all product surfaces from inception as native capabilities—while treating integrations as accelerators and migration channels, never as the only way a capability works.

Audit Summary

What the repository demonstrates

The repository is unusually comprehensive in its articulation of the product:

Area	Evidence in repository	Rebuild interpretation
Product scope	Dedicated product, market, implementation, operating-system, completeness, launch, pilot, audit, and risk documents	Treat the existing documentation as an intent archive and requirements input—not as proof of production completeness
Student OS	Screens for Today, Calendar, Courses, Grades, Degree, Study, Write, Sheet, Deck, Solve, Essay, Exam, Work, and more	Rebuild as the student's canonical workspace and personal operating system
Campus super-app	Screens for Dining, Housing, Maps, Directory, Athletics, Activities, Community, Meet, People, Classmates, and Connect	Rebuild as a governed campus-service platform with institution-controlled publishing and operational workflows
Institutional system	Screens for University, Registrar, Registration, Gradebook, Reports, Console, Agreements, Data, and Trust Room	Rebuild as a multi-tenant institutional control plane with auditable workflows, role boundaries, and implementation tooling
Family/guardian	Dedicated Family experience plus family- and guardian-oriented database checks	Rebuild as consented, age-aware, policy-driven relationship management—not merely shared student access
Career and lifelong journey	Career, Opportunities, Applying, Pathway, Alumni/community-oriented experiences	Model long-term identity, verified achievements, opportunities, advising, and portable learner records from inception
AI capabilities	AI, intelligence, insights, sources, solve, write, study, and policy-related areas	Make AI a governed platform service with provenance, policy enforcement, human escalation, and tenant controls
Security and operations	Security, secrets, monitoring, rollback, restore, retention, legal review, risk, launch, and audit artifacts	Convert written controls into enforceable architecture, SLOs, runbooks, tests, and evidence automation
Data foundations	Supabase migrations, functions, schema snapshot, RLS coverage, tenancy, identity, LTI, SCIM, records, retention, ledgers, and governance checks	Preserve the domain ideas but rationalize them into versioned bounded schemas and a real system-of-record strategy

The front-end alone includes roughly a hundred named experience screens, covering the student, faculty/staff, institution, family, and operational surfaces. The state layer is also substantial, with client persistence, synchronization, reducer, and storage testing patterns. This proves the vision is real and detailed; it also signals high coupling and a material risk that interface-specific state became the de facto product model.[5] [6]

Key architectural diagnosis

1. **The product surface is broader than the execution boundary.**
   A monolithic `App.tsx`, very large individual screen files such as Calendar, Sheet, Write, Today, University, and large client store/shape files are a warning that critical domain behavior, navigation, presentation, and state may be co-located. That pattern does not scale to a replacement-grade education platform.[2] [6] [5]

2. **Native functionality must become the primary control plane.**
   "Connected when available" cannot mean a student loses access to calendar, records, coursework, payment status, messaging, support, study tools, or institutional services when an SIS, LMS, identity provider, payment processor, or campus feed is unavailable. Semester needs canonical native objects, with connectors mapping and reconciling external systems into those objects.

3. **The Supabase area reflects strong governance intent but needs consolidation.**
   The repository contains a significant set of checks for tenancy, RLS, LTI, SCIM, grades, records, guardians, ledger chains, legal holds, deletion, retention, integrations, rate limits, support access, and

operations. That is valuable institutional thinking. The rebuild must turn this into a small number of clear, versioned policies and service contracts, rather than allowing an expanding collection of database assertions to become the architecture. [4]

4. **"Feature-complete" needs a stricter definition.**
   A screen, mock action, client-side state transition, or test fixture is not a live capability. A capability is live only when it has: canonical data ownership, authorization, audit events, validation, persistence, synchronous or asynchronous processing, user-visible state, operational monitoring, support procedures, export/deletion/retention rules, accessibility coverage, and failure/recovery behavior.
5. **Institutional replacement is a commercial and operational claim, not only a technical claim.**
   Replacement requires implementation methodology, migration/rollback, integration quality controls, support SLAs, contract terms, security evidence, accessibility conformance evidence, reliability targets, data processing governance, and qualified counsel review. The repository already acknowledges legal review and operational readiness as domains; the rebuilt operating model must make them release gates. [1] [4]

Rebuild Principles

Core thesis

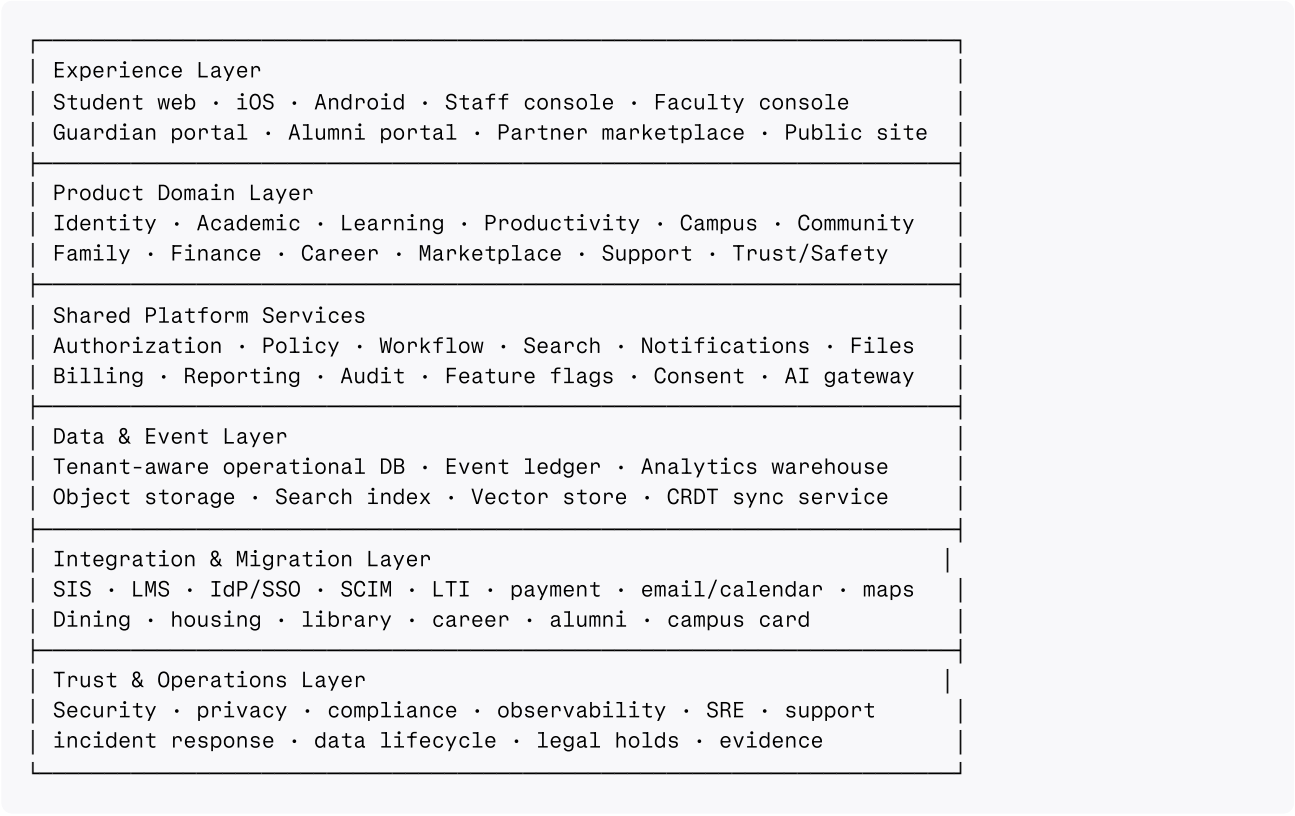
> Every part of a student's educational life works in Semester from day one—natively when necessary, connected when available, governed everywhere, and supported operationally.

Turn that into these non-negotiable design laws:

Law	Required behavior	Prohibited shortcut
Native first	Every core capability has an internal domain model, workflow, UI, API, authorization, audit trail, and operational owner	"We support it" when the only implementation is a third-party redirect or fragile import
Connected when available	Connectors enrich, synchronize, migrate, or co-exist with Semester; each has health, provenance, reconciliation, retry, and fallback	Silent sync failure, unclear source of truth, or blocking core workflows on an integration
Governed everywhere	Every action is policy-evaluated, tenant-scoped, role-scoped, logged, explainable, reviewable, and revocable	A privileged admin path or AI action that bypasses policy and audit
Operationally supported	Every capability has telemetry, SLOs, runbooks, on-call ownership, support workflows, feature controls, and incident behavior	Shipping a feature with no monitoring or recovery plan
User agency by default	Students own and understand their data, permissions, AI context, notifications, sharing, and portability	Unexplained profiling, dark patterns, or guardian/institution access beyond consent and policy
Institution ready	A school can configure, implement, migrate, govern, measure, and exit without custom code	A bespoke deployment that cannot be repeated across institutions
Offline-resilient	Priority student flows work with local persistence, encrypted storage where appropriate, CRDT/event synchronization, conflict UX, and secure re-authentication	Local browser cache treated as reliable sync or a server-only workflow masquerading as offline
Accessible and inclusive	WCAG 2.2 AA as baseline, assistive-technology validation, keyboard-first interaction, dyslexia-aware modes, language/readability controls	Treating accessibility as visual polish applied after product completion
AI bounded by trust	AI actions are policy-bound, source-aware, attributable, logged, redactable, reviewable, and interruptible	An ungoverned chat box with access to student/institutional data

Target Product Architecture

Platform layers



Bounded domains

Do not organize the rebuilt codebase around pages or generic `components`. Organize it around domains with explicit ownership and contracts:

Domain	Canonical objects Semester owns	Minimum native capabilities
Identity and tenancy	Person, profile, institution, campus, program, role, relationship, consent, account, device	Self-service account recovery, MFA/passkeys, session/device management, tenant policy, SSO/SCIM support
Academic core	Term, course, section, catalog, enrollment, schedule, degree requirement, academic record	Course planning, native registration workflow, degree audit, records, holds, approvals, transcript export
Learning	Learning activity, module, submission, rubric, assessment, attempt, grade, feedback	Course delivery, assignments, quizzes, grading, feedback, learning analytics, accommodation workflows
Productivity	Task, project, note, document, worksheet, deck, study set, reminder, goal	Native creation, organization, collaboration, export, version history, search, offline editing
AI services	AI session, tool call, source, policy, prompt template, evaluation, decision record	Tutor, advisor, planner, writer, solver, study coach; provenance and human override on consequential actions
Campus life	Event, location, service, meal, housing assignment, club, activity, ticket	Campus directory, maps, dining, housing, events, groups, athletics, campus alerts
Family and guardian	Guardian relationship, permission, sharing scope, consent record, age/role policy	Granular sharing, billing access, communication preferences, revocation, guardian visibility controls
Finance	Account, charge, invoice, payment plan, aid item, refund, scholarship	Native ledger, billing display, payment plan workflows, student-visible explanation, institution reconciliation
Career and lifelong	Skill, portfolio artifact, opportunity, application, mentor, verified achievement, alumni profile	Career planning, job/internship workflow, portfolio, advising, alumni network, lifelong identity

Domain	Canonical objects Semester owns	Minimum native capabilities
Marketplace	Provider, listing, offer, order, commission, dispute, payout	Institutional approval, discovery, checkout, fulfillment state, moderation, refunds/disputes
Administration	Configuration, policy, workflow, report, case, approval, roster, import job	Config studio, operational dashboards, bulk operations, audit exports, workflow builder
Trust, safety, support	Report, moderation case, support ticket, incident, status event	Reporting, triage, escalation, safe messaging, support access controls, communication templates

Core data ownership matrix

Data category	Canonical owner	Source precedence	Student control	Institution control
Personal profile	Semester identity domain	Student-entered, verified institutional claims, approved IdP attributes	Edit non-verified fields; view provenance; request correction	Configure required profile data; manage verified attributes
Academic record	Academic core	Institution-approved authoritative record; immutable change history	View, export, dispute/correction workflow	Approve, amend under policy, report, retain
Learning work	Learning domain	Student work and instructor assessment events	Create, submit, access history, export where policy permits	Course and retention policy; grade release; accommodation controls
Personal productivity	Productivity domain	Student-created content	Full control subject to legal/safety policy	No default visibility; explicit tenant policy only
AI context	AI governance domain	Scoped and consented retrieval set	Inspect/delete where permitted; control data sources	Configure allowed models, data zones, retention, tool permissions
Financial data	Finance domain	Institution billing events and payment processor confirmations	View, pay, manage allowed preferences	Manage charges, refunds, plans, reconciliation
Guardian data	Family domain	Verified relationship and consent records	Granular share/revoke subject to age/legal policy	Configure legal/policy constraints; cannot bypass consent without authority
Audit logs	Trust and governance domain	Append-only system events	Access subject to privacy/security policy	Retention, investigation, compliance export

Capability Requirements Matrix

The following matrix is the rebuild's minimum definition of "all capabilities live from inception." It preserves the breadth visible in Semester today while requiring every domain to be native, secure, and operable.

Product surface	Native system required at launch	Connection options	Governance and operational requirements
Student home	Personal agenda, priorities, deadlines, alerts, inbox, next actions, wellness-aware preferences	LMS/SIS/calendar/email imports	Explain why each recommendation appears; offline view; notification controls
Calendar and time	Native events, tasks, schedules, time blocks, reminders, conflicts, RSVP	Google, Microsoft, ICS, SIS/LMS schedules	Source labeling, reconciliation, conflict resolution, retry visibility
Courses and degree	Catalog, sections, enrollments, requirements, degree plan, academic holds	SIS/catalog feeds, registrar APIs	Approval workflow, record provenance, audit history, export

Product surface	Native system required at launch	Connection options	Governance and operational requirements
Learning and assessment	Native modules, assignments, submissions, quizzes, rubrics, gradebook, feedback	LTI, LMS import, plagiarism/assessment tools	Academic integrity policies, accommodation support, grade-change audit
Study and tutoring	Flashcards, practice, tutoring, sources, spaced repetition, learning plans	LMS content, library/research tools	AI scope controls, citations, tutoring safeguards, learning evidence
Writing, documents, sheets, decks	Native editor/storage/version history/export/collaboration	Office/Google import/export	Ownership, sharing permissions, encryption, version/restore, accessibility
AI executive assistant	Planning, prioritization, drafting, scheduling suggestions, case routing, workflow assistance	Email/calendar/LMS/SIS, only through consent/policy scopes	Model gateway, evaluation, tool permissions, human confirmation for actions
Campus life	Events, maps, directory, dining, housing, athletics, clubs, transit/service notices	Campus feeds, map, dining, housing systems	Institution content moderation, feed freshness, emergency overrides
Communication and community	Messaging, groups, announcements, meetings, directory visibility	Email, calendar, video, campus identity	Harassment reporting, moderation, age-aware controls, retention
Family portal	Linked accounts, consent-based academic/financial/campus sharing, alerts	SIS family contacts, payment systems	Relationship verification, granular scopes, revocation, activity audit
Finance and payments	Student bill, charges, payment plans, aid visibility, receipts	ERP/payment processor/financial aid systems	PCI scope minimization, ledger reconciliation, support escalation
Career and alumni	Profile, portfolio, pathways, opportunities, applications, mentoring, alumni communities	Career systems, employer partners, LinkedIn-style imports where lawful	Credential verification, employer moderation, consented discoverability
Marketplace	Listings, provider onboarding, ordering, entitlement, disputes, payout state	Payment/fulfillment/partners	Provider approval, sanctions/tax/vendor review as applicable, safety moderation
Staff and faculty	Roster, academic workflow, advising caseload, gradebook, interventions, communications	SIS/LMS/IdP	Least privilege, field-level policy, bulk-change approvals, audit exports
Institution console	Tenant configuration, policy engine, role admin, integrations, reports, service health	SSO, SCIM, SIS, LMS, HR/finance	Four-eyes controls, break-glass, change management, evidence collection
Support and trust	Help center, ticketing, support access, moderation, incident/status communication	CRM/telephony/email integrations	Consent-based impersonation, redaction, escalation, SLA and incident procedures
Data and reporting	Operational reporting, data export, warehouse pipelines, institutional dashboards	BI/warehouse, secure data exchange	Data classification, approved metrics, lineage, access review, retention

Delivery Model

Build sequence without postponing scope

“From day one” means every domain exists as a real native capability at launch—not that every sophisticated automation is built before the first institution can use it. Establish a launch contract with three levels:

Level	Meaning	Permitted at launch?
Core live	Fully native, customer-facing, durable, governed, observable, supported, and tested	Yes
Controlled automation	Capability is live, but high-risk or complex actions route through human review or approval workflows	Yes
Deferred enhancement	No promise to customers until it meets Core Live or Controlled Automation criteria	No feature may be marketed as available before this changes

Program increments

Increment	Outcome	Mandatory exit gate
0. Foundation	Canonical identity, tenancy, authorization, audit, policy, data classification, observability, design system, deployment controls	Threat model, architecture review, RLS/authorization verification, disaster-recovery rehearsal
1. Student OS	Home, planner, calendar, tasks, documents, study, AI executive assistant, notifications, mobile/offline foundation	Offline conflict tests, AI policy tests, accessibility review, support readiness
2. Academic core	Catalog, courses, enrollment, degree planning, learning, submission, gradebook, records, staff console	Authoritative-record workflow, grade audit, integration reconciliation, pilot migration
3. Campus and family	Campus services, community, events, directory, housing/dining representations, family relationships	Moderation operations, consent/guardian rules, content freshness SLOs
4. Commerce and career	Billing, payment plans, marketplace, career pathways, opportunities, applications, alumni	Financial controls, provider/partner governance, dispute and reconciliation workflows
5. Institutional replacement	Migration center, configuration studio, full integration hub, reporting, implementation automation	Parallel-run evidence, institutional sign-off, rollback drill, security/legal review
6. Scale and ecosystem	Multi-institution controls, partner ecosystem, data exchange, vertical packages	Repeatable implementation, tenant isolation audit, capacity and chaos testing

Practical launch strategy

Do **not** pitch “replace every campus system on day one” to the first customer. Build the platform to be replacement-capable from inception, then enter institutions through a structured adoption model:

1. **Student-led value layer**: student productivity, planning, study, advising, and campus experience.
2. **Institution-operated system of engagement**: identity, rosters, policy, communications, support, directory, and staff workflows.
3. **Native system-of-record modules**: learning, academic operations, billing, records, and marketplace.
4. **Migration and coexistence**: connectors, reconciliation, archival, record verification, cutover, and reversible rollout.
5. **Replacement status**: only after successful parallel run, data verification, operational acceptance, contractual approval, and human legal/compliance review.

This preserves the founding thesis while preventing an unprovable or operationally unsafe go-to-market claim.

Engineering Blueprint

Recommended technical approach

Layer	Recommended pattern	Reason
Client applications	Monorepo with shared design system, contracts, SDK, and domain packages; web plus native iOS/Android	Consistent product semantics while retaining platform-quality mobile capabilities
Mobile and offline	Native mobile apps or high-quality cross-platform shell with native modules; encrypted local store using SQLCipher; sync engine with explicit queues and conflict UI	Student workflows must continue during weak connectivity and protect local data
Collaboration	CRDTs for user-authored, collaboratively edited artifacts; server-authoritative workflow events for records, grades, billing, and permissions	Prevents using collaboration primitives where transactional integrity is required
APIs	Versioned domain APIs, command/query separation where needed, idempotency keys, correlation IDs, signed webhooks	Supports integrations, mobile sync, safe retries, and auditability
Data	Relational operational store per tenant strategy, append-only domain event/audit ledger, encrypted object store, search index, analytics warehouse	Preserves integrity while enabling discovery, reporting, and data lifecycle control
Authorization	Central policy decision point with role, relationship, purpose, tenant, jurisdiction, consent, and data-classification inputs	RBAC alone cannot express guardian, advisor, privacy, and institutional rules
AI	Provider-agnostic model gateway; retrieval service; evaluation harness; policy enforcement; tool broker; redaction and provenance service	Prevents model lock-in and makes AI inspectable and controllable
Integration	Connector framework with schemas, mappings, source metadata, sync cursor, job status, retries, dead-letter queues, reconciliation reports	Makes “connected when available” observable and reversible
Observability	Distributed tracing, structured audit events, SLO metrics, product analytics with privacy controls, security telemetry	Supports both reliability and institutional evidence
Release operations	Feature flags, tenant ring rollout, canary, automatic rollback criteria, migration gating, preview/test environments	Allows safe institutional rollout without customer-specific forks

Offline synchronization rules

Separate these data classes:

Class	Examples	Sync rule
Local-first personal data	Notes, tasks, draft documents, study artifacts	Device-encrypted storage; CRDT or mergeable changes; user-visible conflict history
Coordinated collaboration	Shared documents, group work, comments	CRDT operations with membership/permission checks on every sync
Transactional institutional data	Registration, grades, records, bills, holds, approvals	Server-authoritative commands; idempotency keys; explicit pending/confirmed/failed states
Sensitive/cache-restricted data	Official records, billing, accommodations, case notes	Minimize offline cache; encrypt; enforce device posture and expiration; revoke on logout/device removal
AI-generated drafts	Study plans, drafts, suggestions	Store provenance, model/version, cited sources, policy decision, and user acceptance state

Minimum repository structure

```
semester/
  apps/
    student-web/
    staff-console/
    faculty-console/
    guardian-web/
```

```
    ios/
    android/
    public-site/
  services/
    identity/
    academic/
    learning/
    productivity/
    campus/
    family/
    finance/
    career/
    marketplace/
    support-trust/
    ai-gateway/
    integration-hub/
    notification/
    search/
    reporting/
  packages/
    design-system/
    domain-contracts/
    policy-sdk/
    api-client/
    offline-sync/
    event-schema/
    observability/
    test-fixtures/
  infrastructure/
    terraform-or-equivalent/
    environments/
    policy-as-code/
    observability/
  docs/
    architecture/
    adr/
    security/
    privacy/
    operations/
    implementation/
    product-specs/
```

The current application and product documents should be imported into this structure as reference material. Avoid copying screen-by-screen code into the new core. Recreate user journeys from behavior contracts and domain requirements, then port only well-isolated, well-tested visual components where they fit the new design system.

Trust, Legal, and Institutional Readiness

Governance matrix

Decision / action	Student	Guardian	Faculty / advisor	Institution admin	Semester operations	Required control
View personal task or note	Full	Only if explicitly shared	No default access	No default access	No default access	Consent and privacy policy
View grade	Full	Policy/consent/age dependent	Authorized course or advising scope	Authorized academic scope	Break-glass only	FERPA-style access policy, audit log

Decision / action	Student	Guardian	Faculty / advisor	Institution admin	Semester operations	Required control
Change final grade	No	No	Authorized workflow submission	Authorized approval workflow	No	Dual control, immutable audit, reason required
Register for course	Submit/confirm	No	Advise/approve if policy requires	Configure/approve overrides	No	Eligibility, hold, capacity, policy checks
Use AI tutor	Full within policy	Visibility only if consented	Configure educational context	Configure models/retention	Monitor aggregate quality	Source grounding, safety, data minimization
Support impersonation	No	No	No	No	Time-bound, consented or policy-authorized	Session recording/audit, least privilege, approval
Delete personal data	Request	Request where applicable	No	Manage legal/record exceptions	Execute verified workflow	Retention schedule, legal holds, status tracking

Required compliance program

Do not make legal conclusions or sign contracts without qualified human counsel. The legal/privacy workstream must be designed as a continuous review program:

- Maintain a jurisdiction-by-jurisdiction applicability register for privacy, student records, child/minor protections, accessibility, consumer protection, payment, employment, tax, marketing, and education-sector requirements.
- Use external qualified counsel to approve customer contracts, data-processing terms, privacy notices, minors/guardian policies, cross-border transfer mechanisms, marketplace terms, institutional claims, and incident-notification positions.
- Implement data classification from day one: public, internal, confidential, student-record, regulated-sensitive, payment-adjacent, security-sensitive, and legal-hold data.
- Establish retention schedules by domain; legal holds must override normal deletion without silently preventing users from seeing request status.
- Require accessibility acceptance criteria in every story, including keyboard, focus, semantic structure, contrast, mobile assistive technology, captions/transcripts where relevant, and cognitive accessibility.
- Maintain formal security artifacts: risk register, threat model, vendor register, asset inventory, access reviews, vulnerability management, secure SDLC evidence, incident response exercises, and disaster recovery evidence.

AI governance requirements

Every AI feature must have:

1. A named user purpose and allowed data classes.
2. A retrieval policy, source list, source freshness rule, and citation/provenance behavior.
3. A documented non-goal and safety boundary.
4. A model/provider routing policy.

5. Human confirmation before any external, financial, academic-record, enrollment, grade, or consequential action.
6. Tenant controls for enablement, models, retention, regional processing, prompts, knowledge sources, and tool permissions.
7. Evaluation before release and recurring production-quality monitoring.
8. Prompt-injection and data-exfiltration defenses at the retrieval and tool boundary.
9. Red-team scenarios involving student data, sensitive accommodations, financial data, guardians, faculty authority, and institutional impersonation.
10. A user-visible explanation for recommendations and automation.

Commercial Model

Customer segments

Segment	Primary buyer	Core job-to-be-done	Initial package
Individual student	Student	Organize life, learn better, reduce missed deadlines, get trusted guidance	Student OS
Institution	CIO, provost, student success, registrar, student affairs	Improve student experience and operational coherence without fragmented tools	Institution Platform
Department/program	Dean, department head, advising/learning leader	Improve cohort outcomes and workflow consistency	Department deployment
Family	Guardian/family payer	Support student progress with appropriate boundaries	Family companion
Employer/partner	Career services, employer, provider	Find verified talent or provide approved student services	Career/marketplace partner
Alumni association	Alumni/advancement leader	Maintain lifelong identity, engagement, mentoring, and opportunity	Lifelong network

Pricing architecture

Revenue stream	Pricing basis	Notes
Student subscription	Per active student per month/year	Keep a meaningful free or institution-sponsored entry tier; do not gate core accessibility or safety
Institutional platform	Annual platform fee plus active-user or enrolled-student tiers	Include implementation, success, support tier, and optional modules
Implementation services	Fixed-scope package plus change orders	Productize migration, configuration, training, integration, and launch preparation
Premium AI	Usage bands or institution-managed credits	Expose guardrails and budget controls; do not create surprise student bills
Marketplace	Commission, listing, transaction, or SaaS partner fees	Only once consumer protection, provider governance, refund, tax, and operational capacity are mature
Career/employer	Employer seat, campaign, or outcome-informed pricing where legally and ethically appropriate	Avoid discriminatory targeting; preserve student choice
Alumni/lifelong	Institution-sponsored community and engagement module	Monetize institutional value, not exploit personal learner data

Unit-economics dashboard

Track these from inception:

Metric	Formula / unit	Decision use
CAC by segment	Fully loaded acquisition spend ÷ new customers	Channel viability
Student activation	Activated students ÷ registered students	Product-led conversion quality
Time to first value	Median time from signup to completed core job	Onboarding and implementation quality
Institution implementation time	Contract start to operational go-live	Scalability and services margin
Gross retention	Retained recurring revenue ÷ starting recurring revenue	Product necessity
Net revenue retention	Starting recurring revenue plus expansion minus contraction/churn ÷ starting recurring revenue	Account growth
Gross margin	Revenue minus direct hosting, AI, support, and service delivery costs	Sustainable pricing
AI cost per successful outcome	AI/compute cost ÷ quality-qualified completed AI task	AI pricing and routing
Support contact rate	Tickets per 100 active users	Product quality and operational load
Service reliability	SLO attainment per critical journey	Institutional trust
Integration health	Successful syncs and reconciled records ÷ expected syncs	Coexistence/replacement readiness
Marketplace take rate	Marketplace gross profit ÷ marketplace GMV	Marketplace viability

Launch Gates

No-go conditions

Semester should not make institutional-replacement claims, sign contracts promising critical functions, or market capabilities as live if any of the following are true:

- Core student or institutional data lacks a defined canonical owner.
- Authorization depends solely on client-side checks.
- Tenant isolation, permission boundaries, or audit coverage have not been independently tested.
- Native workflows cannot function when an external connector is degraded.
- Migration has no reconciliation report, rollback procedure, or source-of-truth policy.
- AI can access restricted data, act externally, or influence consequential decisions without enforceable policy and appropriate human review.
- Support staff can view sensitive customer data without time-bound, auditable, least-privilege support access.
- There is no tested incident response, backup restoration, or customer communication process.
- Accessibility and mobile/offline standards are deferred.
- Legal claims, contracts, privacy notices, and compliance conclusions have not been reviewed by qualified counsel.

Launch readiness scorecard

Dimension	Minimum launch proof
Product	All promised journeys completed end-to-end with production-like data
Architecture	Domain ownership, API contracts, threat models, ADRs, and failure-mode designs approved
Security	Pen test/review, secrets handling, access review, logging, incident response drill
Privacy	Data map, retention, deletion, consent, DPA/privacy review by qualified counsel
Accessibility	WCAG 2.2 AA testing plan, automated and manual assistive-tech validation
Reliability	SLOs, dashboards, alerts, load tests, backup and restore drill, rollback rehearsal
AI	Evaluation reports, safety cases, prompt-injection tests, model routing and cost controls
Integration	Connector health, reconciliation, idempotency, retry/dead-letter, degraded-mode UX
Operations	On-call, runbooks, support queue, status page, escalation policy, customer communications
Commercial	Packaging, implementation scope, pricing, procurement security packet, customer success plan
Institutional	Pilot acceptance criteria, governance council, administrator training, migration and cutover plan

Master Rebuild Prompt

Paste the following into your preferred coding agent / product-execution environment. It is written to generate architecture, plans, specifications, implementation slices, tests, and operational assets—not just attractive screens.

```
You are the executive founding team, principal execution partner, and accountable delivery organi

Act simultaneously as:
- Founder/CEO and company strategist
- Chief Product Officer
- Chief Technology Officer
- Chief Financial Officer
- Chief Revenue Officer
- Chief Marketing Officer
- Chief Operating Officer
- Chief Information Security Officer
- Chief Legal, Privacy, and Compliance Strategist
- Principal Platform Architect
- Principal Software Architect
- Principal Data Architect
- Principal Frontend Engineer
- Principal Backend and API Engineer
- Mobile, Offline, SQLCipher, and CRDT Architect
- AI Governance Architect
- Design Systems Lead
- Brand and Creative Director
- SaaS Pricing and Unit-Economics Architect
- Institutional EdTech Procurement Strategist
- Marketplace and Partnership Strategist
- Customer Success and Institutional Implementation Architect
- Revenue Operations and Finance Operations Architect
- Launch Readiness and Risk Management Lead
- Security, Privacy, Trust, Accessibility, Reliability, and Incident Response Lead
- Growth, Lifecycle Marketing, Social Media Operations, Content, and Demand Generation Strategist
- Support Operations and Trust-and-Safety Architect
- Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist
```

```
Legal rule:
You must identify legal, privacy, regulatory, contractual, and jurisdiction-specific risks; recom

Mission:
Rebuild Semester into a production-grade Student Operating System and education platform that is

Core thesis:
“Every part of a student’s educational life works in Semester from day one—natively when necessar

Product definition:
Semester is simultaneously:
1. Student OS
2. Productivity suite
3. Personal AI executive assistant, tutor, and advisor
4. Campus super-app
5. Institutional operating system
6. Governed AI platform
7. Marketplace
8. Family and guardian portal
9. Career, alumni, and lifelong learner platform

Do not reduce Semester to an LMS, planner, portal, chatbot, marketplace, or dashboard. It must un

Repository instruction:
Audit the existing repository before implementation. Treat all current UI, documentation, databas

Non-negotiable product principles:
1. Native-first:
   Every claimed capability must have a native Semester domain model, persistence, API, authoriza
2. Connected when available:
   External systems may enrich, synchronize, migrate, or coexist with Semester. They must never b
3. Governed everywhere:
   Every sensitive read, write, recommendation, automation, and AI action is tenant-aware, role-a
4. Operationally supported:
   Every launch capability has telemetry, SLOs, runbooks, support workflows, incident behavior, s
5. User agency:
   Students can understand and control sharing, consent, AI context, notifications, connected acc
6. Institutional readiness:
   Institutions can configure, implement, migrate, govern, measure, integrate, support, and exit
7. Accessibility:
   Design and implement to WCAG 2.2 AA or stronger practical standards, keyboard access, screen-r
8. Offline resilience:
   Prioritize offline-capable personal productivity and learning flows. Use encrypted local stora
9. AI trust:
   AI is helpful but bounded. It is source-aware, policy-controlled, evaluated, attributable, aud

Build the following domains as first-class bounded contexts:
- Identity, account, organization, campus, role, relationship, consent, device, and tenancy
- Academic catalog, terms, courses, sections, enrollments, schedules, degree planning, records, h
- Learning delivery, assignments, submissions, assessments, rubrics, feedback, gradebook, and acc
- Student productivity: tasks, calendar, goals, reminders, notes, documents, spreadsheets, decks,
- AI executive assistant, AI tutor, AI advisor, AI writer, AI study coach, AI search/research ass
- Campus services: maps, locations, dining, housing, directory, events, clubs, activities, athlet
- Community: groups, discussions, direct communication, meetings, mentorship, peer collaboration,
- Family and guardians: verified relationships, granular sharing scopes, consent, billing visibil
- Finance: student accounts, charges, invoices, payment plans, financial-aid visibility, refunds,
- Career, portfolio, opportunities, applications, mentoring, alumni, verified achievements, lifel
- Marketplace: providers, listings, offers, orders, entitlements, payouts, disputes, refunds, par
- Institutional administration: configuration studio, workflows, policy management, reporting, go
- Support, trust, privacy, security, legal holds, retention, deletion requests, incidents, and st

Architecture requirements:
- Use a monorepo organized by apps, bounded domain services, shared packages, contracts, policy S
- Build native web, iOS, and Android experiences. Shared code is allowed, but do not compromise m
```

```
- Use SQLCipher or equivalent encrypted on-device persistence for sensitive mobile local storage
- Use CRDTs only for appropriate collaborative user-authored content. Do not use CRDTs for author
- Use server-authoritative transactional commands for academic records, registration, grades, bil
- Require idempotency keys, correlation IDs, explicit command status, retries, dead-letter handli
- Implement a policy decision layer capable of evaluating tenant, role, relationship, purpose, co
- Build an append-only, tamper-evident audit/event ledger for sensitive and consequential actions
- Define data provenance and source-of-truth precedence for all imported, manually entered, insti
- Implement integration adapters for SIS, LMS, SSO, SCIM, LTI, calendar, email, payment, campus f
- Design for multi-tenant isolation with independently testable authorization at API, database, q
- Use a provider-agnostic AI gateway with data minimization, redaction, retrieval policy, model r

AI system requirements:
- No model can directly access arbitrary data, execute privileged tools, alter records, send comm
- Store AI session metadata, purpose, model/version, policy decision, data scopes, tools invoked,
- Provide institutional controls for models, data processing zones, allowed knowledge sources, pr
- Include a red-team test suite covering prompt injection, data exfiltration, unsafe tutoring, ac
- AI outputs that assert facts must display source provenance or clearly state uncertainty. AI mu

Data and privacy requirements:
- Create a complete data inventory, classification scheme, retention schedule, deletion workflow,
- Keep student-private productivity data private by default. Institutional visibility requires ex
- Make guardian sharing granular, revocable where lawful, age-aware, relationship-verified, and f
- Add user-visible privacy controls, data export, account/device management, connection managemen
- Establish a legal-review queue and flag all areas requiring qualified human counsel.

Security and reliability requirements:
- Implement passkeys/MFA where appropriate, secure session management, device management, rate li
- Threat-model each domain and every privileged workflow.
- Define and instrument SLOs for authentication, student home, calendar, course access, learning
- Build backup, restore, regional/disaster recovery, incident response, status communication, rol
- Add tenant-safe feature flags, staged rollout, canary, automatic rollback criteria, and change-

Product completeness standard:
No feature is “complete” until it has:
1. Product requirements and user journeys
2. Domain model and ownership
3. API/service contract
4. Authorization and policy rules
5. Persistence and data lifecycle rules
6. Audit and observability events
7. Accessibility acceptance criteria
8. Mobile and offline behavior where relevant
9. Error, empty, loading, degraded, and recovery states
10. Automated unit, integration, end-to-end, security, accessibility, and contract tests
11. Support documentation and runbook
12. Analytics and success metrics
13. Release gate and rollback plan

Output and execution format:
Work in a persistent, auditable execution program. Do not simply produce a generic plan or mock U

Start by producing:
A. Repository audit:
   - Existing capability inventory grouped by domain
   - Architecture map
   - Dependency map
   - Current-state risk register
   - Native versus connected assessment
   - Mock/demo/client-only versus operational classification
   - Reuse, rewrite, retire, and migrate recommendations
B. Product requirements:
   - Complete PRD by bounded domain
   - Actor/job map for student, applicant, learner, faculty, advisor, registrar, finance officer,
   - End-to-end critical journeys
```

```
   - Acceptance criteria and capability-level definition of done
C. Architecture:
   - Target system architecture
   - Canonical data model and ownership matrix
   - API and event contracts
   - Offline and synchronization design
   - Integration architecture and source-of-truth policy
   - AI architecture, policies, and evaluation strategy
   - Security, privacy, and compliance architecture
D. Company operating plan:
   - ICP, segmentation, positioning, pricing, packaging, sales motion, procurement strategy, impl
   - Bottom-up unit-economics model with assumptions clearly labeled
   - Hiring plan, org design, board cadence, decision rights, OKRs, and risk management
E. Delivery plan:
   - Epics, milestones, sequencing, staffing needs, dependencies, release gates, test plan, pilot
F. Working implementation:
   - Create the monorepo structure
   - Establish core packages and contracts
   - Implement foundation services before screen proliferation
   - Build vertical slices through student, staff, guardian, and institution experiences
   - Write code, migrations, test suites, infrastructure configurations, runbooks, ADRs, and secu
   - Keep a continuously updated traceability matrix linking existing Semester capabilities to re

Decision discipline:
- State assumptions explicitly and maintain an assumption register.
- Label facts, design decisions, hypotheses, and legal-review items differently.
- When tradeoffs exist, present alternatives, recommend one, and explain decision criteria.
- Never claim a capability is native, live, compliant, secure, accessible, or production-ready wi
- Never hide a dependency on a third party.
- Never let an integration become the sole implementation of a core Semester capability.
- Never ship a privileged or high-impact AI automation without appropriate policy enforcement and
- Prefer narrow, testable vertical slices with complete operational support over broad unfinished
- Preserve the full product thesis; do not quietly defer or remove a domain. If a sophisticated w

Begin now with the repository audit and the traceability matrix. Then create the target architect
Identity + tenancy + policy engine + audit ledger + student home + task/calendar core + offline e
```

Recommended Immediate Decisions

1. **Declare a clean-platform rebuild.**
   Preserve the repository as the intent and evidence baseline, but prevent it from dictating the final architecture. Freeze broad feature additions to the current monolith except for security and data-protection fixes.
2. **Create a capability traceability register.**
   For every existing screen, document, test, database check, and workflow, record: user outcome, user role, domain owner, native data model, required API, connected systems, governance policy, operational owner, tests, and migration status.
3. **Define the system-of-record posture now.**
   For every academic, financial, identity, learning, productivity, family, and career object, answer: “Who owns this record? What happens if its connector goes down? Who can change it? How is that change audited?”
4. **Build the identity-policy-audit spine first.**
   Before rebuilding individual experience areas, establish multi-tenancy, roles, relationships, consent, device/session control, policy evaluation, audit events, and operational observability. Every later domain depends on it.
5. **Treat the first institution as a design partner, not a generic customer.**
   Choose one bounded setting—such as a university cohort, program, or student-success organization—to

validate implementation, governance, support, and coexistence. Do not begin with an uncontrolled, campus-wide record-system replacement.

6. **Establish counsel and institutional advisory review early.**
   The company can architect toward privacy, accessibility, procurement, academic integrity, student-record governance, and operational maturity immediately, but qualified legal counsel and domain experts must validate the claims and contracts before external commitments.

The strongest element already present is the ambition: Semester is not trying to be another fragmented education tool. The rebuild must make that ambition **architecturally true**—one native, trusted, operable student and institution platform rather than a catalog of disconnected screens. [4] [5] [1]

⁂

ok what else in order to complete every asspect of the build the right way to completion go in depth on eveything and every asspect and role and requirment and feature provide every graph piece gode matrize ux model etc needed to complete this and do it the proper way

You need more than a feature plan to complete Semester correctly: you need a **complete company operating system**, a product-system architecture, a delivery factory, institutional implementation machinery, security/privacy governance, support operations, and a ruthless evidence-based definition of "done." The aim is not merely to launch an app; it is to operate a trustworthy, native, institution-ready education platform that can replace fragmented systems while working for students every day.

The existing Semester repository gives you an unusually rich feature and governance blueprint. The next step is to convert that blueprint into a controlled, traceable build program where every capability has an owner, data model, operating model, test strategy, support path, pricing logic, and release gate. [16] [17] [18]

Completion Standard

A capability is **not complete** because it has a screen, a route, a mock interaction, a state reducer, a database table, or a passing unit test.

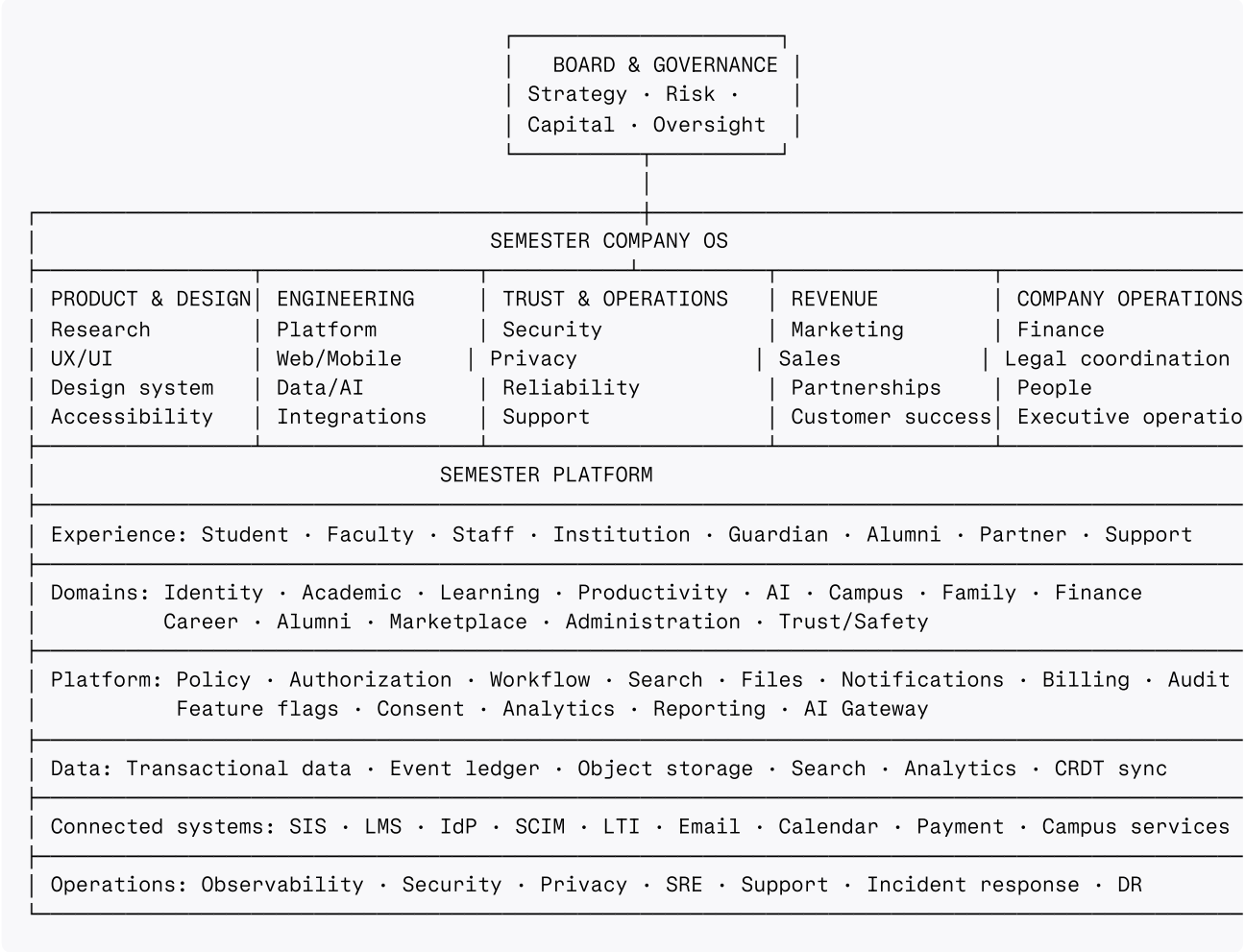
A Semester capability is complete only when all 16 conditions below are true:

#	Completion dimension	Required evidence
1	User outcome	Defined user, job-to-be-done, success metric, and critical journey
2	Domain ownership	Bounded context, accountable team, data owner, and source-of-truth decision
3	UX	Responsive and accessible flows for student, staff, guardian, and mobile contexts where applicable
4	Native capability	Semester-owned data model, workflow, API, persistence, and user interface
5	Integration behavior	Optional connector mappings, reconciliation rules, degraded mode, and no silent failures
6	Authorization	Tenant, role, relationship, purpose, consent, and data-classification checks
7	Governance	Audit event, policy record, approval/override rules, and review process
8	Data lifecycle	Data classification, retention, deletion, export, legal-hold, backup, and restoration behavior
9	Reliability	Service-level objective, performance budget, failure state, retry behavior, and recovery procedure
10	Security	Threat model, abuse cases, tests, secrets handling, and monitoring coverage

#	Completion dimension	Required evidence
11	AI safety	If AI is involved: data-scope controls, provenance, evaluations, escalation, and human confirmation
12	Accessibility	Keyboard, screen reader, contrast, focus, mobile assistive tech, cognitive load, and error recovery validation
13	Test coverage	Unit, integration, contract, end-to-end, authorization, accessibility, performance, and security tests
14	Operations	Alerts, dashboards, runbook, on-call ownership, support macro, and incident playbook
15	Commercial	Package placement, entitlements, pricing, implementation implications, and customer success expectations
16	Release evidence	Release checklist, feature flag, rollout cohort, rollback plan, and documented go/no-go approval

This is how you build the whole platform "from inception" without pretending incomplete automations are finished. The whole product is present and native; any complex high-risk action that needs institution configuration begins as a **fully real, governed, human-in-the-loop workflow**, not as a fake integration or unavailable future promise.

Whole-System Map



The four operating modes

Semester must support all four modes from its earliest architecture:

Mode	Customer situation	Semester behavior
Native	No external system exists or an institution wants Semester as the primary system	Semester is the canonical record and workflow owner

Mode	Customer situation	Semester behavior
Connected	Existing authoritative system remains in place	Semester connects with explicit source precedence, sync, reconciliation, and user-visible provenance
Coexistence	Institution adopts Semester department-by-department or module-by-module	Semester operates natively in its activated domains and interoperates cleanly in others
Replacement	Institution cuts over a legacy platform	Semester provides migration, verification, parallel run, readiness, rollback, and operational ownership

The mistake to avoid is treating “integration” as a product area. Integration is a **platform discipline** applied to every domain.

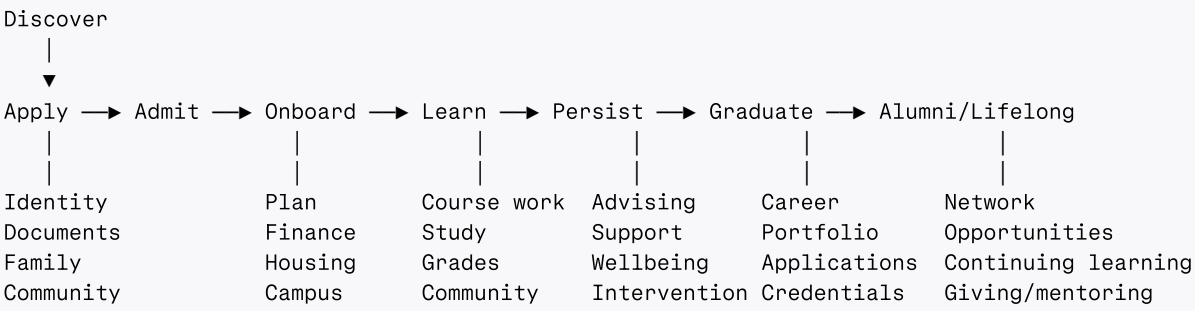
Product Capability Blueprint

Capability domains

Domain	Native capabilities required	Connected systems	Primary users	Highest-risk conditions
Identity	Accounts, profiles, passkeys/MFA, devices, recovery, roles, affiliations, consent	IdP, SSO, SCIM, HR/SIS	Everyone	Account takeover, cross-tenant access, privilege escalation
Academic core	Terms, catalog, sections, schedules, enrollment, holds, degree audit, records	SIS/ERP, registrar tools	Students, registrar, advisors	Incorrect records, registration errors, unauthorized amendments
Learning	Courses, modules, assignments, submissions, assessments, rubric, gradebook, feedback	LMS, LTI tools, content vendors	Students, faculty, TAs	Grade integrity, academic integrity, accommodation failures
Productivity	Tasks, projects, documents, sheets, decks, notes, reminders, files, search	Google/Microsoft/imports	Students, faculty, staff	Privacy leakage, data loss, collaboration conflicts
Calendar	Events, tasks, time blocks, schedules, focus, reminders, availability	Google, Microsoft, ICS, SIS/LMS	Everyone	Missed deadlines, double booking, incorrect source sync
AI	Assistant, tutor, advisor, researcher, writer, planner, study coach	Approved models and knowledge sources	Students, staff, faculty	Hallucinated policy, unsafe advice, sensitive-data leakage
Campus life	Dining, housing, maps, directory, events, transit, clubs, athletics, campus alerts	Campus feeds and service systems	Students, campus staff	Outdated safety/service information, inappropriate directory exposure
Family	Guardians, consent, sharing scopes, family notifications, payment access	SIS contacts, billing	Student, guardian, family office	Improper disclosure, minor/age rules, relationship fraud
Finance	Charges, bills, payments, plans, aid visibility, refunds, receipts	ERP, payment processor, aid systems	Student, finance office	Payment errors, PCI exposure, incorrect balances
Career	Profile, portfolio, opportunities, applications, mentoring, skills, credentials	Career systems, employers	Student, alumni, employers	Bias, privacy, false credentials, discrimination
Marketplace	Listings, providers, orders, entitlement, disputes, payouts	Payment, vendors, fulfillment	Students, providers, institution	Fraud, consumer safety, tax, disputes

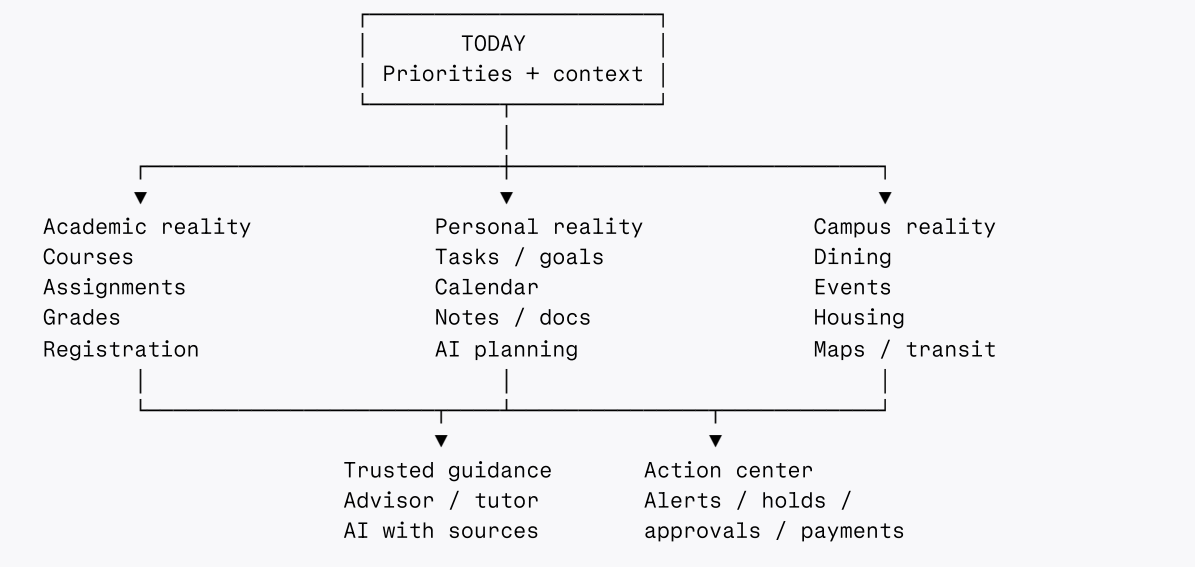
Domain	Native capabilities required	Connected systems	Primary users	Highest-risk conditions
Institution console	Configuration, policy, workflow, reporting, admin, audit, implementation	IdP, data warehouse, enterprise tools	Institution staff	Misconfiguration, bulk errors, excessive admin authority
Support/trust	Help, case management, moderation, reporting, support access, incidents	CRM, email, phone	Students, staff, support	Improper support access, unsafe community content, incident mishandling

Student lifecycle model



Semester must preserve a person's history and permissions through this lifecycle without forcing a new identity or losing academic, productivity, career, or consent context.

Daily student experience model



"Today" is not a dashboard. It is the **orchestration surface** for the student's entire educational life. Every alert must identify its source, severity, action, deadline, impact, owner, and fallback support route.

UX and Design System

Experience principles

1. **One identity, one language, one interaction model.**
   Students should not feel they moved from an LMS to a registrar system to a payment portal to a campus feed.
2. **Show the next useful action, not every possible feature.**
   The platform can be broad while each moment remains focused.
3. **Always reveal authority and provenance.**
   Users must see whether information is official, connected, user-created, AI-generated, pending, estimated, or verified.
4. **No dead-end states.**
   Every error, restriction, empty state, permission denial, degraded integration, and approval delay should explain what happened, what the user can do, and who can help.
5. **Accessibility is product quality.**
   WCAG 2.2 Level AA should be the minimum web acceptance standard, with equivalent native-mobile accessibility requirements. W3C's guidance covers a broad range of disabilities and requires, among other things, text alternatives, keyboard-accessible functionality, usable navigation, and adaptable presentation.[19] [20]

Core information architecture

```
Semester
├── Today
├── Plan
│   ├── Calendar
│   ├── Tasks
│   ├── Goals
│   ├── Projects
│   └── Time / Focus
├── Learn
│   ├── Courses
│   ├── Assignments
│   ├── Study
│   ├── Grades
│   ├── Degree
│   └── Academic records
├── Create
│   ├── Documents
│   ├── Sheets
│   ├── Decks
│   ├── Notes
│   └── Files
├── Campus
│   ├── Events
│   ├── Maps
│   ├── Dining
│   ├── Housing
│   ├── Directory
│   └── Community
├── Path
│   ├── Career
│   ├── Opportunities
│   ├── Portfolio
│   ├── Applications
│   ├── Mentors
│   └── Alumni
├── Money
```

```
│   ├── Bill
│   ├── Aid
│   ├── Payment plans
│   └── Marketplace
├── Support
│   ├── Help
│   ├── Advising
│   ├── Tutoring
│   ├── Cases
│   └── Safety
├── Assistant
│   ├── Ask
│   ├── Plan
│   ├── Study
│   ├── Write
│   └── Sources / history
└── Account
    ├── Profile
    ├── Privacy
    ├── Connections
    ├── Family
    ├── Accessibility
    ├── Devices
    └── Billing
```

Role-specific home surfaces

Role	Home page must answer	Primary widgets
Student	What matters today, what is at risk, what can I do now?	Agenda, priorities, course work, holds, grade changes, bills, campus events, advisor and AI guidance
Faculty	Who needs attention, what needs grading, what is next?	Course health, submissions, grading queue, attendance/engagement signals, student outreach
Advisor	Which students need intervention and what action is appropriate?	Caseload, milestones, risk signals, notes, appointment queue, interventions, handoffs
Registrar	What academic workflows are blocked, incorrect, or pending?	Registration exceptions, holds, record changes, degree audit exceptions, approvals
Finance	What money movement or student communication needs action?	Payment plans, failed payments, aid visibility, reconciliation, refund queue
Student affairs	What campus needs, events, cases, and services require action?	Events, engagement, case queue, resource availability, alerts
IT/security	Is the tenant secure, healthy, integrated, and compliant?	Auth health, connector health, audit anomalies, access reviews, incidents
Guardian	What has the student consented to share and what action is appropriate?	Authorized milestones, bill/payment details, selected alerts, support resources
Support agent	What is the customer's issue without overexposing private data?	Case timeline, approved support access, known incidents, diagnostic tools, safe resolution actions

Interface state matrix

Every screen must define these states before engineering begins:

State	Required user experience
Loading	Skeleton that matches final layout; meaningful status for lengthy work

State	Required user experience
Empty	Explain value, primary action, import/connect option if relevant
Permission denied	Explain restriction without exposing sensitive data; offer correct request path
Offline	Clear offline indicator, local changes queue, next sync attempt, available actions
Sync pending	Show source, last sync, pending count, and no false success claim
Degraded dependency	Identify affected connection; retain native features; offer alternate workflow
Validation failure	Field-level reason, preserved user input, accessible error summary
System failure	Friendly message, retry, support reference, correlation ID where appropriate
Approval pending	Owner, estimated SLA, current status, cancel/edit rules
AI uncertainty	Sources, confidence limits where appropriate, user choices, escalation path
Sensitive workflow	Confirmation, policy basis, audit notice, impact summary, irreversible-action warning

Design tokens

Use a central design-token system—never hard-code visual values per screen.

```
type SemesterTokens = {
  color: {
    brand: {
      primary: string
      secondary: string
      accent: string
    }
    semantic: {
      success: string
      warning: string
      danger: string
      info: string
      official: string
      connected: string
      ai: string
      pending: string
    }
    surface: {
      canvas: string
      raised: string
      sunken: string
      overlay: string
    }
    text: {
      primary: string
      secondary: string
      muted: string
      inverse: string
      link: string
    }
  }
  space: Record<"0" | "1" | "2" | "3" | "4" | "5" | "6" | "8" | "10" | "12", string>
  radius: {
    sm: string
    md: string
    lg: string
    xl: string
    pill: string
  }
```

```
  typography: {
    display: string
    heading: string
    body: string
    mono: string
    scale: Record<string, { size: string; lineHeight: string; weight: number }>
  }
  motion: {
    fast: string
    normal: string
    slow: string
    reduced: string
  }
  elevation: {
    none: string
    low: string
    medium: string
    high: string
  }
}
```

Required reusable components

Component family	Must include
Navigation	App shell, command palette, global search, mobile tab bar, contextual rail, breadcrumbs, skip links
Input	Text fields, selects, date/time, comboboxes, file upload, rich text, spreadsheet cells, validation summary
Status	Official-source badge, connected-source badge, AI-generated badge, pending, verified, warning, incident
Workflow	Approval timeline, status stepper, activity log, exception queue, confirmation dialog, undo where safe
Data	Responsive table, mobile list alternative, filters, saved views, export, pagination, bulk actions
Learning	Course card, assignment card, rubric, submission receipt, study session, source citation, grade explanation
AI	Prompt composer, source drawer, data-scope picker, action-preview card, feedback control, handoff control
Support	Help panel, ticket form, safe-support consent, system-status banner, diagnostic reference
Accessibility	Focus ring, live region, reduced-motion variant, contrast-safe variants, readable-density settings

Technical Architecture

Architecture decision model

```
User action
   │
   ▼
Experience app
   │
   ├── Local encrypted store and sync queue
   │
   ▼
API gateway / BFF
   │
   ├── Identity verification
   ├── Tenant resolution
   ├── Policy decision
   ├── Abuse / rate controls
   ├── Request correlation ID
```

```
   │
   ▼
Domain command or query
   │
   ├── Transactional database write/read
   ├── Audit event
   ├── Domain event
   ├── Notification / workflow event
   └── Connector job where applicable
   │
   ▼
Observable result
   │
   ├── UI status
   ├── Event ledger
   ├── Metrics / tracing
   ├── Support diagnostics
   └── Analytics under privacy policy
```

Architecture boundaries

Layer	Must own	Must not own
Front end	Presentation, local UX state, offline queue, accessible interactions, cached permitted data	Authorization truth, financial/academic integrity, policy decisions
BFF/API edge	Authentication, tenant routing, request validation, policy invocation, response shaping	Core business logic duplicated per client
Domain service	Business rules, transactions, domain events, canonical data	UI layout, vendor-specific client behavior
Policy engine	Authorization and governance decisions	Business-domain persistence or arbitrary SQL
Integration hub	Mappings, sync, retries, reconciliation, source provenance	Canonical domain ownership
AI gateway	Model routing, retrieval permissions, redaction, tool controls, evaluation	Direct unrestricted access to production databases
Analytics	Aggregation, product/business reporting under privacy controls	Transactional system of record
Audit ledger	Immutable evidentiary record of sensitive and consequential actions	General event bus replacement or mutable analytics store

Canonical entity graph

```
Institution
├── Campus
│   ├── Program
│   ├── Term
│   ├── Course / Section
│   ├── Service / Location
│   └── Policy Set
├── Person
│   ├── Account
│   ├── Role Assignment
│   ├── Relationship
│   ├── Consent
│   ├── Device
│   ├── Student Record
│   ├── Learning Record
│   ├── Productivity Workspace
```

```
│   ├── Financial Account
│   ├── Career Profile
│   └── Support/Trust Cases
├── Integration Connection
│   ├── Credential Reference
│   ├── Mapping Version
│   ├── Sync Cursor
│   ├── Reconciliation Report
│   └── Health Status
└── Governance
    ├── Policy
    ├── Audit Event
    ├── Retention Rule
    ├── Legal Hold
    ├── Approval Workflow
    └── Feature Entitlement
```

Required service contracts

Every domain service requires these contract classes:

```
type CommandEnvelope<T> = {
  commandId: string
  idempotencyKey: string
  correlationId: string
  tenantId: string
  actor: {
    personId: string
    accountId: string
    sessionId: string
    roles: string[]
  }
  purpose: string
  submittedAt: string
  payload: T
}

type CommandResult<T> = {
  commandId: string
  status: "accepted" | "completed" | "pending_approval" | "rejected" | "failed"
  data?: T
  userMessage: string
  nextAction?: string
  auditEventId: string
  correlationId: string
}

type SourceMetadata = {
  origin: "native" | "imported" | "connected" | "institution_entered" | "ai_generated"
  system?: string
  externalId?: string
  mappingVersion?: string
  observedAt: string
  verifiedAt?: string
  freshnessState: "current" | "stale" | "unknown" | "conflicted"
}
```

Event taxonomy

Event category	Examples	Retention posture
Domain events	Course enrolled, task completed, submission created, grade released	Per domain retention rule
Audit events	Role changed, grade amended, support access granted, data exported	Long-term tamper-evident record
Security events	Login failed, MFA enrolled, suspicious session, API abuse blocked	Security retention policy
Integration events	Sync began, mapping failed, reconciliation discrepancy found	Operational and compliance retention
AI events	Retrieval started, policy decision, model invoked, action proposed, override applied	AI governance retention policy
Product analytics	Feature used, funnel progressed, anonymized performance data	Privacy-controlled, minimized aggregation
Operational events	Deployment, alert, rollback, incident state changed	SRE/incident retention policy

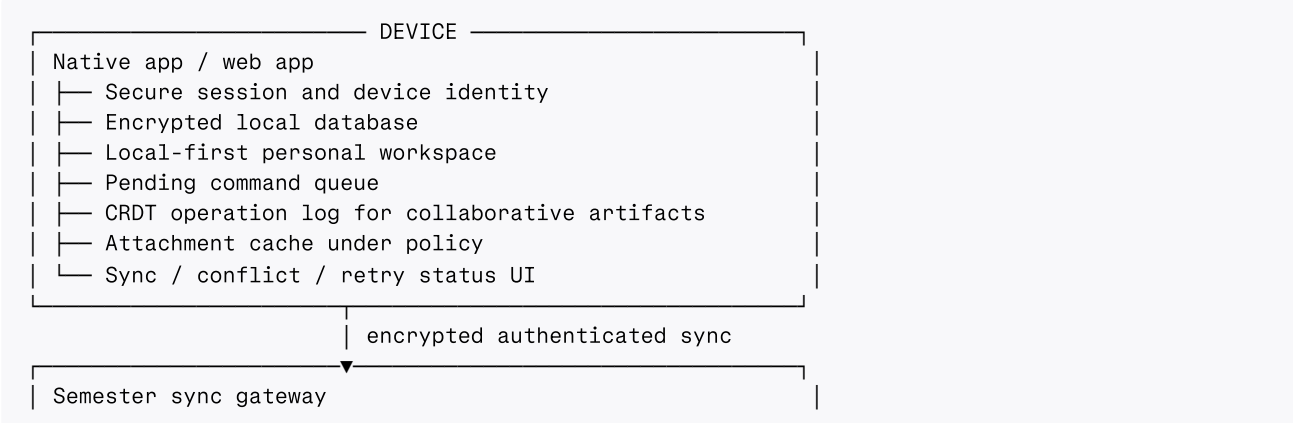
Multi-tenant isolation requirements

Tenant isolation must be verified across all layers, not only database row-level security:

Boundary	Required control
API	Tenant resolved from trusted identity/session, never only user-submitted headers
Database	Tenant predicate or schema/database segmentation; deny-by-default RLS; negative tests
Object storage	Tenant-prefixed authorization, signed URL expiry, content classification
Search	Tenant-aware index partition/filter enforced server-side
Cache	Tenant-scoped keys; no shared unkeyed response cache
Queue	Tenant context propagated and validated in consumers
Analytics	Tenant scoping and privacy-minimized aggregates
Support tooling	Time-bound approved access with scope, reason, expiry, and audit
AI retrieval	Retrieval filters enforced before model context construction
Admin tools	Separate privileged roles, just-in-time access, dual control for high-impact actions

Offline, Mobile, and Sync Model

Device architecture



```
| ├── Authentication + device trust                          |
| ├── Tenant and policy enforcement                          |
| ├── Idempotency + ordering + cursor management             |
| ├── CRDT relay/compaction                                  |
| ├── Command processor                                      |
| └── Conflict and reconciliation events                     |
└────────────────────────────────────────────────────────────┘
```

Data synchronization rules

Data type	Offline availability	Conflict model	Security posture
Tasks and personal notes	Full read/write	Last-write or field-level merge with history	Encrypted local store
Documents and shared notes	Full or partial based on content policy	CRDT merge, version restore	Access rechecked on sync
Calendar	Read cache and queued edits	Source-aware conflict resolution	Minimize sensitive event details
Course content	Read cache as allowed	Version/freshness rules	DRM/licensing policy where needed
Assignment drafts	Full local drafts, queued submission	Server confirmation required	Submission receipt only after server acceptance
Grades	Read cache with freshness status	Server authoritative, no offline edits	Expiring encrypted cache
Registration	Search/cache allowed; action requests online confirmation	Server-authoritative transaction	No false confirmation
Billing/payment	Read summary cache; payments require online secure flow	Processor and ledger authoritative	No sensitive payment data stored
Support case	Draft possible; submit when online	Server authoritative	Encrypt drafts and restrict attachments
AI chat	Drafts offline; inference online unless controlled local capability exists	Server policy decides	No unrestricted local sensitive context

AI Governance Model

NIST frames its AI Risk Management Framework as a voluntary framework for incorporating trustworthiness considerations throughout AI system design, development, use, and evaluation. Use its risk-management orientation as the operating model—not as a marketing badge. [17] [16]

AI control plane

```
User request
   │
   ▼
Purpose classification
   │
   ├── Study / writing / planning / official question / sensitive issue
   │
   ▼
Policy decision
   │
   ├── Is AI enabled for this tenant and role?
   ├── What data classes are allowed?
```

```
    ├── What sources may be retrieved?
    ├── Which models/regions are permitted?
    ├── Are tools permitted?
    └── Is human confirmation mandatory?
    │
    ▼
Context assembly
    │
    ├── Minimize data
    ├── Filter by authorization
    ├── Apply source freshness rules
    ├── Redact where necessary
    └── Defend against untrusted instructions
    │
    ▼
Model gateway and tool broker
    │
    ├── Model selection / cost control
    ├── Output moderation
    ├── Citation/provenance assembly
    ├── Tool call policy enforcement
    └── Human approval for consequential action
    │
    ▼
User result + audit/evaluation
```

AI risk tiers

Tier	Examples	Required safeguards
0: Assistive	Rewrite notes, summarize user-provided text, brainstorm study ideas	Disclosure, user control, safe defaults
1: Contextual	Summarize authorized course materials, create study plans, answer sourced questions	Provenance, retrieval controls, source freshness, feedback loop
2: Advisory	Academic planning suggestions, advising risk signals, financial explanation	Explainability, confidence/uncertainty handling, human review route
3: Action proposing	Draft email, propose calendar event, prepare enrollment request, create support case	Action preview, user confirmation, permission check, audit
4: High impact	Grade-related recommendation, enrollment decision, financial action, disciplinary or safety action	No autonomous final decision; designated human approval, stronger evaluation and logs
5: Prohibited	Covert profiling, unauthorized surveillance, unreviewed high-stakes determinations, bypassing policies	Block and log policy event

AI evaluation suite

Every AI feature needs automated and human evaluation across:

- Accuracy against controlled, sourced answer sets.
- Citation quality and source attribution.
- Harmful or unsafe tutoring behavior.
- Hallucinated institutional policies and deadlines.
- Academic-integrity policy compliance.
- Bias and disparate outcome assessment.
- Prompt injection and untrusted-document attacks.

- Data leakage across students, courses, tenants, family relationships, and roles.
- Tool authorization failures.
- Refusal and safe-handoff quality.
- Cost, latency, and model-fallback quality.
- User comprehension and overreliance risk.

Institutional Integration Blueprint

LTI 1.3/LTI Advantage is an important interoperability path for working with existing LMS environments. Its services include Names and Role Provisioning, Deep Linking, and Assignment and Grade Services, and it uses modern OAuth 2.0, JSON Web Tokens, and OpenID Connect patterns. It should be a first-class connector—but never the only way Semester can support learning workflows. [21] [22] [23]

Integration catalog

Integration class	Standards / examples	Semester native fallback
Identity	SAML, OIDC, OAuth, SCIM	Semester account, passkeys, MFA, invitation and recovery workflows
Student information	SIS/ERP APIs, CSV/SFTP, data exports	Native people, terms, catalog, registration, records, holds, degree data
Learning	LTI 1.3, LTI Advantage, LMS APIs	Native courses, assignments, submissions, assessments, gradebook
Calendar	Google, Microsoft, ICS/CalDAV	Native events, schedules, tasks, reminders, time blocks
Email	Microsoft 365, Google Workspace	Native notifications, inbox-like action center, messaging where enabled
Payments	Payment processor, ERP	Native charges, plans, receipts, ledger and reconciliation
Campus services	Dining, housing, events, card, maps, transit	Native catalog/presentation/workflow, institution-entered operational data
Career	Job boards, employer feeds, career systems	Native opportunity, application, portfolio, mentor, credential objects
Data/reporting	Warehouse, BI, secure file exchange	Native reporting, exports, auditable scheduled reports
Safety/support	CRM, communications, status tools	Native case, support request, moderation, status feed

Connector lifecycle

```
Discover
   │
   ▼
Security and legal review
   │
   ▼
Configure credentials and scopes
   │
   ▼
Map schemas and source precedence
   │
   ▼
Sandbox import / dry run
   │
   ▼
Validate counts, samples, and edge cases
   │
   ▼
Activate limited cohort
   │
   ▼
```

```
Monitor health and reconciliation
   │
   ▼
Expand or roll back
```

Connector health states

State	Meaning	Required UX
Healthy	Syncing within expected window; reconciled	Normal display with source label
Delayed	Last successful sync beyond target but data still usable	Show freshness timestamp and affected scope
Degraded	Some objects or operations failed	Preserve native capabilities; show alternate path
Failed	Connector unavailable or unsafe	Stop dependent automation; expose current native data/fallback
Reconciliation mismatch	Data differs from expected source state	Quarantine automated changes; alert owners; never silently overwrite
Disabled	Not authorized or intentionally off	Show configuration/institution contact path, not an error

Security, Privacy, and Safety

Security program structure

```
Govern
├── Security policy and risk register
├── Asset / data inventory
├── Vendor and supply-chain management
├── Secure SDLC
├── Access governance
└── Incident governance

Protect
├── Identity and MFA
├── Encryption and key management
├── Least privilege / policy engine
├── Network segmentation
├── Secure development
└── Endpoint/device posture

Detect
├── Security telemetry
├── Audit logging
├── Anomaly detection
├── Dependency scanning
└── Abuse monitoring

Respond
├── Incident commander
├── Forensics
├── Customer communications
├── Legal/counsel review
└── Remediation tracking

Recover
├── Backup restoration
├── Disaster recovery
```

```
├── Post-incident review
└── Control improvements
```

Threat model matrix

Asset / workflow	Key threats	Required mitigations
Student account	Credential stuffing, session theft, recovery fraud	Passkeys/MFA, rate limiting, device/session management, suspicious-login detection
Grades and records	Unauthorized changes, inference, bulk export	Fine-grained authorization, dual approval, immutable audit, export controls
Student documents	Unauthorized sharing, deletion, malware	Access policy, signed URLs, version history, malware scanning, restore
Family relationships	Fraudulent guardian claim, oversharing	Relationship verification, granular scopes, age/policy rules, audit/revocation
AI assistant	Prompt injection, data exfiltration, unsafe action	Context filtering, tool broker, output controls, evaluation, confirmation
Integrations	Overbroad tokens, incorrect mapping, replay/webhook abuse	Least scopes, secrets vault, signed webhooks, mapping review, idempotency
Marketplace	Fraud, unsafe vendors, payment disputes	Provider verification, moderation, escrow/payout controls, case workflow
Support	Over-permissioned support agent, impersonation	Just-in-time scoped access, explicit consent, session logs, redaction
Multi-tenancy	Cross-tenant query, cache leakage, shared index exposure	Defense in depth at every boundary, negative tests, regular access reviews
Mobile/offline	Lost device, local data extraction, stale authorization	Encrypted store, remote logout/revocation, cache TTL, re-authentication

Privacy by design checklist

- Data minimization for every field and integration.
- Explicit purpose recorded for sensitive access.
- Separate student-private productivity data from institution-governed records.
- User-visible source, freshness, and audience labels.
- Granular consent and revocation for connected accounts and family sharing.
- Export and deletion workflows that explain exceptions such as records retention and legal holds.
- Regional data processing and transfer assessment where applicable.
- Consent logs with versioned policy text.
- Data-processing inventory and vendor register.
- Privacy review as a required step for new AI, analytics, tracking, community, and marketplace work.
- Legal conclusions and contract commitments routed to qualified human counsel.

Reliability, SRE, and Support

SLO targets

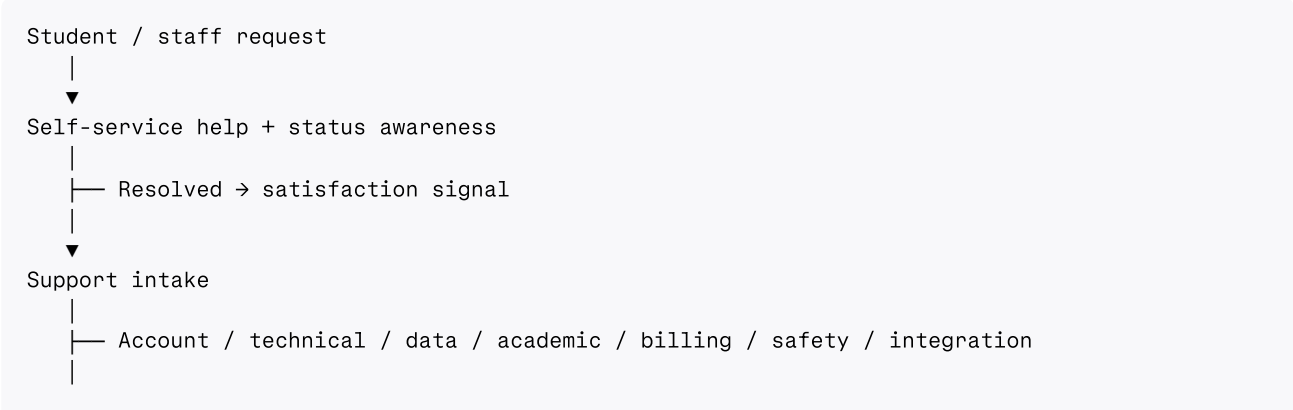
Set provisional targets, measure them, then mature them with real production data.

Service journey	Availability target	Latency target	Data-loss target
Authentication and account access	99.95%	p95 under 500 ms for session validation	Zero unauthorized session persistence
Student Today surface	99.9%	p95 under 2.5 seconds on standard connection	Freshness label when data is delayed
Calendar/task operations	99.9%	p95 under 1 second for local action; 5 seconds sync acknowledgement	No acknowledged action loss
Course content and assignments	99.9%	p95 under 2.5 seconds	Submission receipt and durable storage required
Grade/record display	99.95%	p95 under 2 seconds	No stale data without source/freshness disclosure
Registration workflow	99.95%	p95 under 3 seconds for validation	No duplicate/enrollment-inconsistent command
Billing summary	99.95%	p95 under 2 seconds	Ledger reconciliation required
AI assistance	99.5%	p95 target by model/task; state fallback	No policy-bypassing fallback
Integrations	Connector-specific	Defined sync window per connector	Reconciliation and error journal required
Support/status	99.9%	p95 under 2 seconds	No case loss

Incident severity model

Severity	Example	Initial response target	Communication
SEV-0	Cross-tenant exposure, confirmed data breach, integrity failure affecting official records	Immediate	Executive, security, legal/counsel, affected customers under applicable obligations
SEV-1	Broad platform outage, submissions/registration/grades materially unavailable	15 minutes	Status page and affected customers
SEV-2	Major module degraded, single major tenant affected	30 minutes	Impacted tenant and support
SEV-3	Limited function, workaround available	Same business day	Support response and backlog
SEV-4	Minor defect, no material impact	Planned triage	Release notes or support response

Support operating model



```
   ▼
Triage and severity classification
   │
   ├── Standard support
   ├── Institution administrator
   ├── Engineering/SRE
   ├── Trust & safety
   ├── Security incident
   └── Legal/privacy workflow
   │
   ▼
Resolution, evidence, root cause, customer communication
```

Support agents must never receive blanket visibility into student data. Support access must be purpose-scoped, time-limited, logged, and revocable.

Business, Go-to-Market, and Procurement

Market positioning

> Semester is the native Student Operating System: one governed platform where students plan, learn, create, live on campus, pay, connect, build careers, and continue as alumni—while institutions operate the experience, data, workflows, and trust layer from one modern system.

Procurement evidence room

To sell to institutions, create a Trust Center and private diligence data room containing:

Artifact	Why procurement needs it
Architecture overview	Evaluates fit, resiliency, data flows, and integration posture
Security overview	Explains identity, encryption, logging, testing, incident response
Privacy overview	Explains data use, retention, subprocessors, rights requests
Accessibility VPAT or equivalent	Documents accessibility posture and testing
Data processing terms	Supports privacy/procurement review; final terms require legal counsel
Business continuity and disaster recovery summary	Establishes restoration and incident readiness
Penetration test summary	Demonstrates independent security assessment when available
Subprocessor list	Enables vendor risk assessment
Integration catalog	Confirms standards and supported systems
Implementation methodology	Shows how a customer gets live safely
Support SLA	Defines channels, hours, severity targets, escalation
Product roadmap governance	Shows how requests and changes are controlled
AI governance packet	Details models, data use, controls, testing, and opt-out/configuration
Security questionnaire library	Speeds up common institutional diligence

Institutional implementation model

Phase	Institution work	Semester work	Exit proof
Discover	Goals, stakeholders, systems inventory, constraints	Solution architecture, value hypothesis, risk discovery	Signed implementation charter
Design	Policies, roles, user journeys, data decisions	Configuration model, integration design, migration plan	Approved solution design
Configure	Tenant policies, roles, branding, workflows	Environment, permissions, feature flags, data mappings	Configuration validation
Integrate	Credentials, vendor coordination, test data	Connectors, mappings, reconciliation, error handling	Integration acceptance
Migrate	Data extracts, quality review, business sign-off	Import, transformation, verification, rollback plan	Reconciled data sign-off
Pilot	Cohort participation and feedback	Training, support, monitoring, fixes	Pilot exit criteria met
Parallel run	Compare critical workflows against incumbent	Monitoring, reconciliation, incident simulation	Cutover readiness
Go live	Communications, local support, governance	Hypercare, observability, escalation	Production acceptance
Optimize	Governance cadence and outcomes review	Adoption, expansion, product improvement	Success plan and renewal path

Organization and Role Map

Founding operating structure

Function	Accountable executive	Essential first roles	Deliverables
Executive strategy	CEO	Chief of staff / strategy lead	Strategy, financing, board, partnerships, operating cadence
Product	CPO	Product managers by domain, researcher, product ops	PRDs, outcome metrics, roadmap, capability traceability
Design	Design lead	Product designer, UX researcher, content designer, accessibility specialist	Design system, prototypes, usability evidence, content standards
Engineering	CTO	Platform, backend, frontend, mobile, QA/SDET, DevOps/SRE	Services, apps, CI/CD, quality gates, operational software
Data and AI	Data/AI lead	Data engineer, analytics engineer, ML/AI engineer, AI safety evaluator	Canonical data, warehouse, AI gateway, evaluation and governance
Security	CISO	Security engineer, GRC/privacy program manager	Threat models, control evidence, risk management, incident readiness
Legal/privacy	Legal lead with external counsel	Privacy/compliance lead, contracts ops	Legal queue, contract playbook, privacy program, counsel approvals
Revenue	CRO	Enterprise seller, sales engineer, revops	ICP, pipeline, proposals, procurement response library
Marketing	CMO	Product marketing, demand generation, lifecycle/content	Positioning, website, campaigns, lifecycle, proof assets
Customer success	COO/CS leader	Implementation lead, CSM, support lead	Implementation playbook, adoption, renewals, support operations
Finance	CFO	Finance ops, billing/revops analyst	Pricing, budgets, forecasts, controls, invoicing

Function	Accountable executive	Essential first roles	Deliverables
Trust and safety	COO/CISO shared	Trust & safety lead, moderation/case ops	Reporting, escalation, community safety, case policies
Partnerships	Chief partnerships officer / CRO	Partnerships manager, integration partner engineer	Partner strategy, vendor standards, marketplace governance

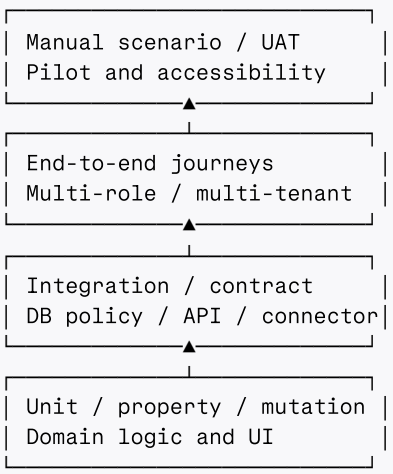
RACI for critical decisions

Decision	CEO	CPO	CTO	CISO	Legal/counsel	CRO	COO	Institution
Product scope	A	R	C	C	C	C	C	C
Architecture standard	C	C	A/R	C	C	I	C	I
Security control	I	C	R	A	C	I	C	I
Privacy/contract conclusion	I	C	C	C	A/R	C	C	C
High-risk AI release	A	R	R	R	C	I	C	C
Institution go-live	A	C	C	C	C	C	R	A/C
Incident communications	A	I	R	R	C	C	R	C
Pricing/package	A	C	C	C	C	R	C	I
Marketplace provider approval	C	C	C	C	C	R	A	C

R = responsible; **A** = accountable; **C** = consulted; **I** = informed.

Quality Engineering Model

Test pyramid plus institutional acceptance



Mandatory test suites

Test suite	Minimum contents
Domain logic	State transitions, invariants, financial/grade/record calculations, idempotency
Authorization	Positive and negative test for every role, relationship, tenant, purpose, and sensitive resource

Test suite	Minimum contents
Tenant isolation	Cross-tenant API, DB, cache, queue, storage, search, analytics, and support-tool tests
API contract	Version compatibility, error shape, pagination, rate limits, idempotency, correlation IDs
Integration	Auth, mapping, schema drift, retry, dead-letter, reconciliation, degraded state
End-to-end	Student, faculty, advisor, guardian, registrar, finance, admin, support journeys
Accessibility	Automated axe-like checks plus manual keyboard, screen reader, zoom, voice control, mobile assistive tech
Mobile/offline	Network loss, resume, sync conflicts, encrypted store, revoked device, stale cache
Performance	Load, soak, concurrency, large tenant/courses, report/export, enrollment spikes
Security	SAST, dependency, secret scan, DAST, auth bypass, authorization fuzzing, penetration test
AI	Grounding, prompt injection, privacy leakage, tool abuse, policy refusals, high-risk workflow escalation
Resilience	Backup restore, regional/dependency failure, queue backlog, connector failure, rollback drill
Migration	Row counts, field matching, data quality exceptions, historical record sampling, rollback

Core non-functional requirements

Category	Launch requirement
Performance	Explicit performance budgets for first load, interaction latency, API p95/p99, sync, and large data screens
Availability	Service-level objectives per critical journey with alert thresholds and error budgets
Security	Secure SDLC enforced in CI/CD; secrets never in code; access reviews and incident readiness
Privacy	Data map, classification, retention, deletion/export workflows before production customer data
Accessibility	WCAG 2.2 AA acceptance criteria and manual testing embedded in release gates
Internationalization	Architecture supports locale, time zone, date formats, language, and content expansion even if launch is US-first
Scalability	Load model for student peak periods: registration, course deadlines, grade release, billing due dates
Observability	Correlation ID from client to API, domain event, integration job, support record, and audit log
Recoverability	Tested backups, restore objectives, failover/manual procedures, and data-reconciliation playbooks
Maintainability	Bounded domains, ADRs, coding standards, dependency rules, owners, and deprecated-feature process

Master Program Backlog

Foundation epics

Epic	Outcomes	Must exist before broad feature build
E0: Product traceability	Every current capability mapped to requirement, owner, domain, status, tests, and release gate	Yes
E1: Identity and tenancy	Accounts, passkeys/MFA, organizations, roles, affiliations, consent, devices, sessions	Yes
E2: Policy and audit spine	Central policy engine, purpose-based access, audit ledger, approvals, break-glass	Yes
E3: Platform operations	CI/CD, environments, secrets, feature flags, telemetry, incident tooling, backups	Yes

Epic	Outcomes	Must exist before broad feature build
E4: Design system	Tokens, components, accessibility contracts, responsive patterns, content standards	Yes
E5: Data platform	Canonical schema, migrations, events, storage, search, analytics, retention	Yes
E6: Offline foundation	Encrypted store, sync, conflict engine, device management, status UX	Before offline-capable experiences
E7: Integration hub	Connector framework, credentials, mapping, sync, reconciliation, health	Before institutional integrations
E8: AI gateway	Policy-bound model access, sources, evaluation, tool broker, audit	Before broad AI rollout

Product-domain epics

Epic	Minimum live native vertical slice
P1: Student OS	Today, tasks, calendar, notifications, unified search, personal settings
P2: Productivity suite	Notes/documents/files, version history, sharing, export, encrypted offline drafts
P3: Learning	Course workspace, assignments, submissions, feedback, study tools, grade visibility
P4: Academic core	Catalog, enrollments, degree plan, holds, approval workflows, records
P5: AI	Sourced tutor, planning assistant, content drafting, advisor handoff, action proposals
P6: Campus	Directory, maps, events, announcements, groups, campus-service catalogs
P7: Family	Relationship verification, sharing dashboard, authorized alerts, billing visibility
P8: Finance	Bill, charges, plans, aid visibility, receipts, reconciliation
P9: Career	Portfolio, opportunities, applications, mentors, skills, verified achievements
P10: Marketplace	Provider onboarding, listings, order flow, entitlement, dispute workflow
P11: Institution console	Tenant config, workflow builder, policy management, integration center, reporting
P12: Support/trust	Help center, cases, safe support access, moderation, safety reporting, status
P13: Alumni	Lifelong profile, network, mentoring, continuing education, opportunities

Operating-company epics

Epic	Required output
C1: Pricing and packaging	Entitlement catalog, SKU structure, usage metering, discount policy, approval process
C2: Procurement	Trust center, evidence library, security questionnaire answer bank, implementation SOW templates
C3: Sales	ICP, account plans, discovery playbook, demo environment, ROI calculator, deal desk
C4: Marketing	Brand system, positioning, messaging house, website, content engine, lifecycle programs
C5: Customer success	Implementation plans, training academy, adoption dashboard, renewal/expansion playbook
C6: Support	Tier model, knowledge base, support tooling, escalation and incident communication process
C7: Finance	Forecast model, spend controls, invoicing, revenue recognition process with professional advice
C8: Governance	Board reporting, risk register, decision log, OKRs, internal controls, executive cadence

Metrics and Control Dashboards

North-star metrics

Perspective	Metric	What success means
Student	Weekly meaningful learning/progress actions completed	Semester improves student progress, not merely time spent
Institution	Percentage of critical student journeys completed in Semester without legacy fragmentation	Semester becomes the operating layer
Product	Activated users completing recurring core workflows	Product is durable and useful
Reliability	Critical journey SLO attainment	Users can trust it at high-stakes moments
Trust	Policy/audit coverage and confirmed privacy/security incidents	Platform operates safely
Revenue	Net revenue retention	Institutions and users get expanding value
Implementation	Time to live and time to first value	The platform can be adopted repeatedly
Support	Contact rate and resolution quality	Product and operations are understandable
AI	AI-assisted outcome quality at acceptable cost and risk	AI helps without creating hidden liabilities

Executive operating dashboard

```
                    SEMESTER EXECUTIVE SCORECARD
┌──────────────┬──────────────────────────────────────────────────────────┐
│ Growth       │ Pipeline · Win rate · CAC · Activation · NRR             │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Product      │ Adoption · Time-to-value · Feature health · Retention    │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Reliability  │ SLOs · Incidents · Error budget · Connector health       │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Trust        │ Security risks · Access reviews · Privacy requests       │
├──────────────┼──────────────────────────────────────────────────────────┤
│ AI           │ Quality · Cost · Safety events · Human overrides         │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Customers    │ Implementation health · CSAT · Renewal risk              │
├──────────────┼──────────────────────────────────────────────────────────┤
│ Finance      │ ARR · Gross margin · Burn · Runway · Collections         │
└──────────────┴──────────────────────────────────────────────────────────┘
```

Essential Registers and Documents

Build and maintain these as living source-controlled artifacts:

Register / document	Purpose
Capability traceability matrix	Maps every existing and future capability to requirements, code, tests, owner, release state
Domain ownership register	Declares canonical service and data owner for every entity
Decision log / ADR library	Records major architectural and product decisions
Assumption register	Distinguishes assumptions from verified facts
Risk register	Product, security, privacy, legal, commercial, operational, vendor, and delivery risks
Data inventory	Lists data elements, classification, owner, purpose, source, retention, and access rules

Register / document	Purpose
Integration catalog	Lists connections, scopes, mappings, health, owners, and source precedence
AI system inventory	Lists each AI feature, models, data use, risk tier, evaluations, and policy controls
Accessibility conformance register	Tracks components/screens, known defects, testing evidence, remediation
Threat-model library	Per domain and high-risk workflow threat models
Incident register	Incidents, severity, timeline, root cause, actions, customer communication
Vendor register	Vendor role, data access, security/privacy review, contract status, renewal date
Release readiness register	Feature-by-feature evidence and approvals
Customer configuration register	Tenant policies, configuration changes, integrations, approvals, owners
Legal review queue	Items that require qualified human counsel review
Public claims register	Every sales/marketing claim, supporting evidence, approval owner, expiration/review date
Accessibility and content style guide	Inclusive language, terminology, reading level, microcopy, error language
Support knowledge base	Customer-facing and internal resolution articles
Business continuity plan	Recovery roles, contacts, priorities, restoration sequences, communication
Implementation playbook	Repeatable institutional deployment method
Exit/migration plan	How customers can export data and transition responsibly

What You Must Not Miss

The following are frequently missed even by technically strong teams:

- **Data provenance:** Every key fact needs a source, timestamp, freshness state, and authority level.
- **Source conflict policy:** If SIS, LMS, instructor, student, and Semester disagree, the platform must know what wins and how the discrepancy is resolved.
- **Academic calendar complexity:** Add/drop, late withdrawal, holds, prerequisites, co-requisites, waitlists, cross-listings, transfer credit, accommodations, incompletes, audits, and term variants.
- **Identity complexity:** One person can be a student, employee, alumnus, mentor, guardian, course staff, and parent across institutions and time.
- **Role relationships:** A guardian is not a generic admin; a faculty member cannot automatically see every student's information; an advisor sees a caseload, not the whole university.
- **Minors and age transitions:** Consent, guardian visibility, messaging, marketing, and data rights can change based on age, jurisdiction, relationship, and institution policy.
- **Accessibility beyond checklists:** Cognitive load, plain language, time limits, sensory overload, dyslexia-aware choices, and assistive technology usability.
- **High-stakes accuracy:** Grades, billing, registration, aid, health/safety, conduct, accommodations, and legal notices cannot be treated like ordinary app data.
- **Support as a product feature:** A confused student at 11:55 PM before an assignment deadline needs clear state, receipts, recovery, and escalation—not a generic error.
- **Data migration:** The hard part is not importing rows; it is data quality, mapping semantics, historical integrity, documents, relationships, exceptions, and reconciliation.
- **Institutional configuration:** Every school has policy variation. Build configuration carefully, but do not create unlimited custom code paths.

- **Contractual commitments:** Sales claims, uptime promises, data processing, AI capabilities, support SLAs, and accessibility statements must match real evidence and receive human legal review where required.
- **Operational cost:** AI, storage, egress, search, notifications, support, integrations, and implementation can destroy margins if not metered and modeled.
- **Sunsetting:** Every capability needs an ownership and deprecation plan; leaving old workflows enabled forever creates risk and confusion.
- **Human workflows:** For sensitive exceptions, a well-designed queue, SLA, evidence trail, and handoff are better than unsafe full automation.

Immediate Execution Sequence

First 30 days

- Freeze unstructured feature expansion in the existing monolithic app.
- Build the complete capability traceability matrix from the current repository.
- Establish architecture decision records and bounded-context ownership.
- Define canonical entity model, source-of-truth policy, data classification, and event taxonomy.
- Stand up identity, tenancy, role/relationship/consent policy, audit ledger, secrets, environments, observability, and feature flagging.
- Create the design token system, shared components, accessibility contract, content standards, and responsive layout primitives.
- Establish a formal security, privacy, accessibility, AI-governance, and legal-review queue.
- Build a test and release evidence framework before broad new product work.
- Create financial model, packaging model, institutional implementation plan, trust center outline, and procurement evidence plan.

Days 31–90

- Rebuild the first end-to-end production vertical slice:
  - Student authentication and onboarding.
  - Institution tenant setup.
  - Student Today.
  - Tasks, calendar, reminders, documents/notes.
  - Encrypted local persistence and sync state.
  - Policy enforcement and audit events.
  - Search, notification, support, accessibility, telemetry, and failure modes.
  - One controlled institutional identity and calendar/LMS/SIS integration path.
- Add the institutional control plane for roles, policy, integrations, feature states, audit, and support access.
- Begin native academic and learning-domain contracts rather than rebuilding screens directly.
- Implement AI gateway foundations, source labeling, policy checks, retrieval controls, evaluations, and safe assistant flows.
- Run accessibility tests, threat models, failure drills, and restoration drills.

Days 91–180

- Build native learning, course workspace, assignments, submissions, study, grade visibility, degree planning, and academic workflow capabilities.
- Deliver guardian, campus, career, and finance foundations as complete governed vertical slices.
- Build integration reconciliation, migration center, implementation controls, and institutional reporting.
- Launch a design-partner program with strict pilot acceptance and no unverified replacement claims.
- Establish 24/7 incident ownership for critical journeys or clearly contract and communicate the actual support coverage.
- Begin procurement, security, accessibility, and AI diligence workflows early—before a major enterprise deal forces rushed evidence creation.

Definition of "Ready to Sell"

Semester is ready to sell when it can truthfully demonstrate:

1. A student can securely sign up, authenticate, onboard, plan, learn, create, receive trusted AI help, manage their data, and receive support.
2. An institution can create a tenant, manage roles and policy, configure the experience, integrate authorized systems, monitor health, support users, and retrieve evidence.
3. The platform has native workflows for every promised product domain, even when sophisticated external automation is activated by configuration or human review.
4. Students can distinguish official, connected, personal, estimated, pending, and AI-generated information.
5. Any sensitive action is governed, authorized, auditable, recoverable, and operationally owned.
6. The team can withstand an outage, connector failure, support incident, security event, data-deletion request, and customer migration issue with documented procedures.
7. Sales, marketing, security, privacy, AI, and accessibility claims are backed by evidence and appropriate human review.
8. A pilot institution can implement Semester with repeatable documentation rather than founder-dependent improvisation.

That is the completion standard: not "we have built a lot," but **Semester can safely carry a student's educational life and an institution's trust from day one.**

⁂

Show me the architecture diagram for user roles

Give me the database schema for authorization and tenants
Provide the production deployment pipeline for this system
Draft the incident response and support runbook
Show me the complete test strategy and quality gates
indidual pompts prompts to give codex for each role stated in first prompt and any other roles and required to complete and launch semester with there tasks

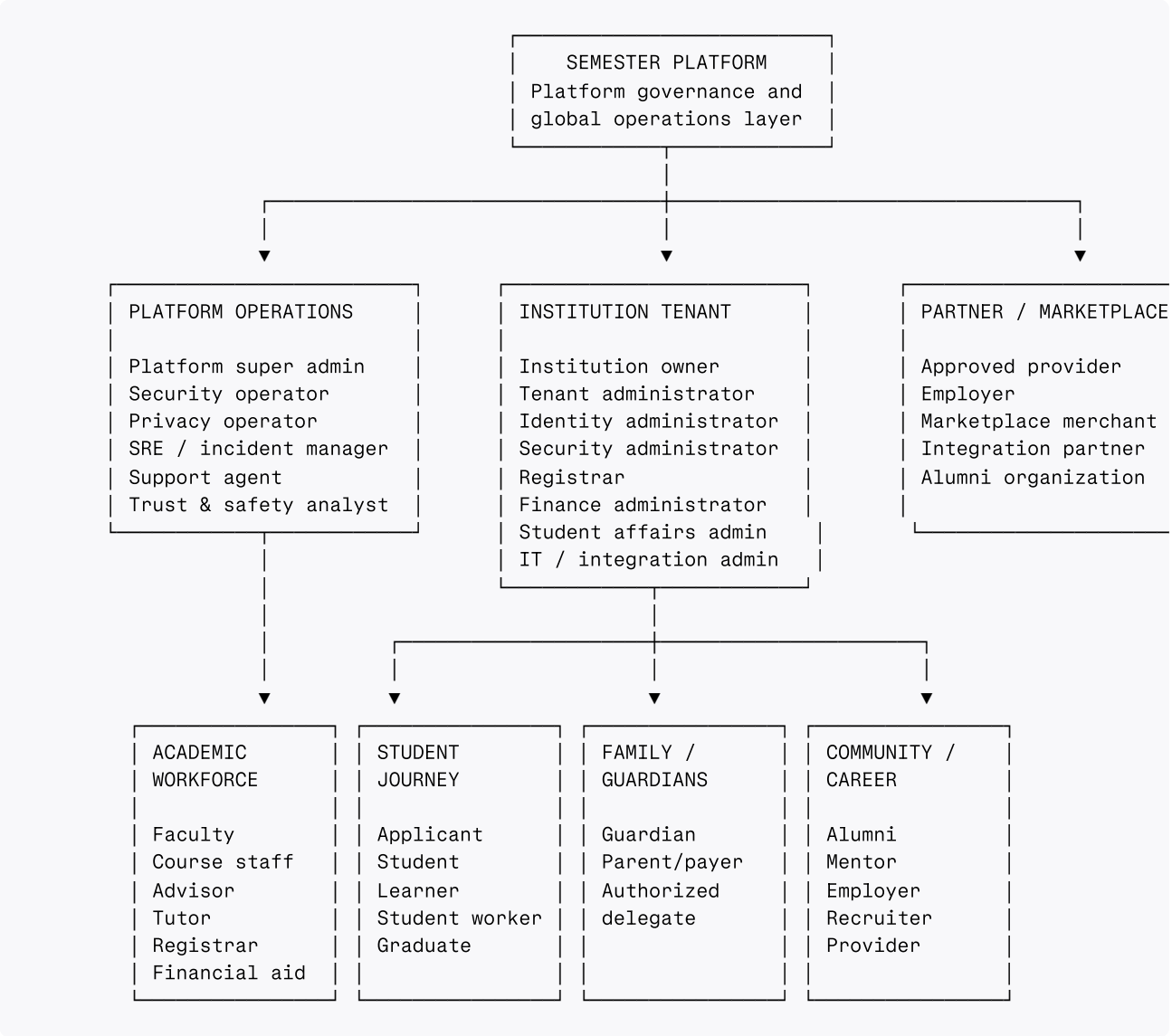
Below is a complete execution pack for Semester: the role architecture, a production-grade authorization and tenant schema, deployment pipeline, incident/support runbook, quality strategy, and copy-ready Codex prompts for every required function.

Use this as the **operating specification** for rebuilding Semester. It assumes the founding thesis remains non-negotiable: Semester is a native Student OS, productivity suite, AI assistant/tutor/advisor, campus super-app, institutional platform, family portal, marketplace, and lifelong career/alumni network—with integrations enhancing the product rather than replacing native capabilities.

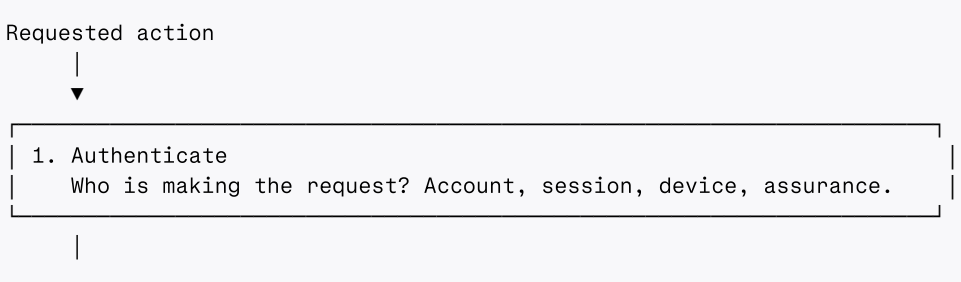
1. User Role Architecture

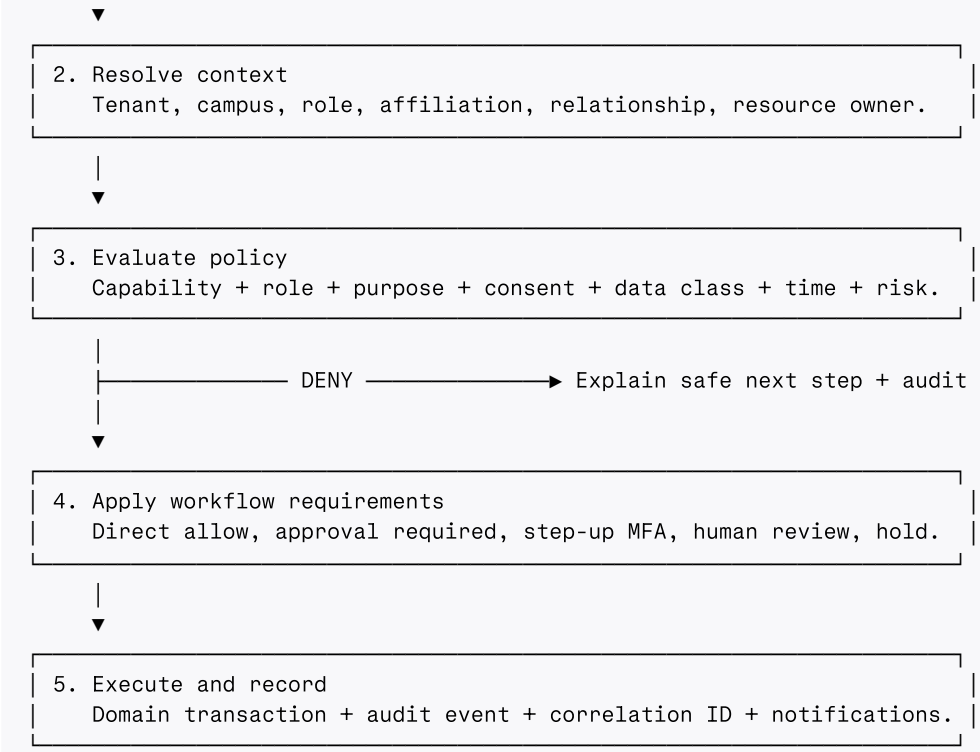
Semester should not use a simplistic "student / admin" permission model. It needs **tenancy, affiliation, relationships, purpose, consent, data classification, policy, and time-bound authority.**

Role architecture diagram



Policy decision model





Role hierarchy

Do **not** treat hierarchy as universal inheritance. A registrar must not inherit unrestricted finance, support, or security authority. Use capability grants with explicit resource scope.

Role family	Roles	Typical scope	Must never automatically receive
Platform operator	Platform admin, SRE, security analyst, support agent	Global platform but tightly purpose-scoped	Blanket student-data access, unlogged impersonation, tenant record modification
Institution administrator	Tenant owner, identity admin, security admin, integration admin	One institution/tenant	Platform-wide visibility, direct access to other tenants
Academic staff	Faculty, TA, tutor, advisor, registrar	Course, section, caseload, program, or record workflow	Unrestricted student financial/family/private productivity data
Student	Applicant, student, learner, graduate, student worker	Self-owned data and authorized group/course spaces	Other students' private data or institution-wide controls
Guardian/family	Guardian, parent, payer, authorized delegate	Explicit relationship and consent scope	Full student identity, notes, course work, or grades by default
Finance	Bursar, aid officer, billing staff	Financial account, approved financial workflow	Academic/private productivity data unless specifically necessary and authorized
Campus life	Housing, dining, student affairs, athletics staff	Service-specific scope	Academic and financial record access by default
Career/alumni	Employer, recruiter, mentor, alumni staff	Opt-in opportunity and profile data	Student record, grades, protected data
Marketplace	Provider, merchant, fulfillment partner	Own listings/orders/entitlements	Student academic, family, or institution data
Trust and safety	Moderator, investigator, safety lead	Report/case-specific data under policy	Open-ended private content access

Role-to-domain permissions matrix

Domain	Student	Faculty	Advisor	Registrar	Guardian	Finance	Institution admin	Semester support
Personal tasks/notes	Full self	No default	No default	No default	Explicit share only	No	No default	Time-bound consent only
Course content	Enrolled access	Assigned course admin	Assigned student summary	Limited	Explicit share only	No	Configurable	Support-scoped only
Assignment submission	Create/view self	View/grade course	Read limited if policy permits	No default	Explicit share only	No	No default	Diagnostic metadata only
Grades	View self	Grade assigned work	Caseload/policy scope	Record workflow	Consent and law/policy scope	No default	Policy-limited	No default
Registration	Self-service/submit	Advise/approve if allowed	Caseload advisement	Full registrar workflow	No default	Hold visibility if allowed	Configure policy	No default
Billing	View/pay self	No	Limited eligibility summary	No default	Explicit payer scope	Full finance scope	Policy configuration	Support-scoped only
Family consent	Manage self where lawful	No	No default	No default	Manage self relationship side	No	Configure rules	No default
Tenant settings	No	No	No	Limited registrar settings	No	Limited finance settings	Full tenant scope	No direct changes without approved workflow
Security logs	Own activity only	Own activity only	Own activity only	Workflow-limited	Own activity only	Workflow-limited	Tenant security scope	Platform operational scope, minimized

Relationship graph

```
Person A: Student
   │
   ├── enrolled_in ──────────────► Course Section
   ├── advised_by ───────────────► Advisor
   ├── taught_by ────────────────► Faculty Member
   ├── guardian_relationship ────► Guardian
   ├── payer_relationship ───────► Parent / Sponsor
   ├── works_for ────────────────► Institution Department
   ├── member_of ────────────────► Club / Cohort / Group
   ├── mentored_by ──────────────► Mentor
   ├── employed_by / applies_to ─► Employer
   └── affiliated_with ──────────► Institution / Campus / Program
```

Each edge needs its own:

- Start and end dates.

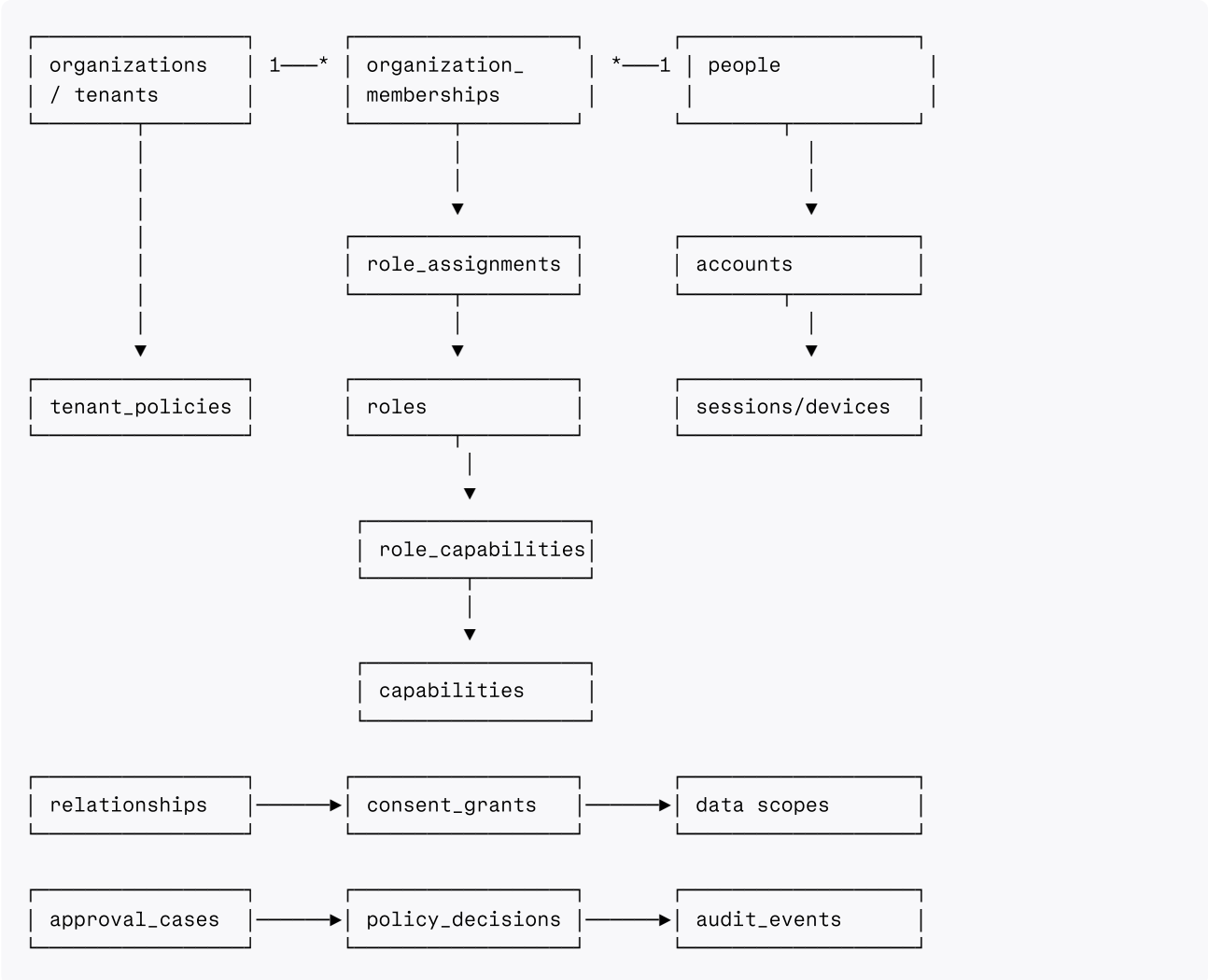
- Verification state.
- Consent and legal/policy basis.
- Data-sharing scopes.
- Authority scope.
- Audit trail.
- Revocation or expiration process.
- Institution policy override rules where legally required.

2. Authorization and Tenant Database Schema

Use PostgreSQL with strict tenant isolation, a policy engine, row-level security where appropriate, append-only audit logging, and a service-layer authorization check. Do not rely on any single control.

OWASP advises testing multi-tenant isolation through the same role, connection path, and connection-pooling behavior used by the application; it specifically recommends positive and negative tenant-isolation tests for every RLS-protected table.[40]

Authorization entity relationship diagram



Core PostgreSQL schema

This is a foundation, not a full academic/LMS schema. It gives Semester a correct authorization and multi-tenancy spine.

```
create extension if not exists pgcrypto;
create extension if not exists citext;

create schema if not exists app;
create schema if not exists private;
create schema if not exists audit;

create type app.tenant_status as enum (
  'draft',
  'active',
  'suspended',
  'offboarding',
  'closed'
);

create type app.membership_status as enum (
  'invited',
  'active',
  'suspended',
  'ended'
);

create type app.assignment_status as enum (
  'active',
  'pending_approval',
  'suspended',
  'expired',
  'revoked'
);

create type app.consent_status as enum (
  'granted',
  'denied',
  'revoked',
  'expired',
  'superseded'
);

create type app.data_classification as enum (
  'public',
  'internal',
  'confidential',
  'student_record',
  'sensitive',
  'payment_adjacent',
  'security_sensitive',
  'legal_hold'
);

create type app.policy_decision_outcome as enum (
  'allow',
  'deny',
  'require_step_up',
  'require_approval',
  'require_human_review'
);

create type app.audit_outcome as enum (
```

```
  'success',
  'denied',
  'failed',
  'pending'
);

create table app.tenants (
  id uuid primary key default gen_random_uuid(),
  slug citext not null unique,
  legal_name text not null,
  display_name text not null,
  tenant_type text not null check (
    tenant_type in (
      'institution',
      'district',
      'school',
      'department',
      'partner',
      'marketplace_provider',
      'internal'
    )
  ),
  status app.tenant_status not null default 'draft',
  primary_region text not null,
  timezone text not null default 'America/Chicago',
  data_residency_policy jsonb not null default '{}'::jsonb,
  feature_entitlements jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz
);

create table app.people (
  id uuid primary key default gen_random_uuid(),
  global_person_key uuid not null default gen_random_uuid(),
  preferred_name text,
  legal_first_name text,
  legal_last_name text,
  primary_email citext,
  mobile_phone text,
  date_of_birth date,
  locale text not null default 'en-US',
  timezone text,
  accessibility_preferences jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  deleted_at timestamptz,
  unique (global_person_key)
);

create table app.accounts (
  id uuid primary key default gen_random_uuid(),
  person_id uuid not null references app.people(id),
  account_status text not null check (
    account_status in ('invited', 'active', 'locked', 'suspended', 'closed')
  ),
  assurance_level text not null default 'aal1' check (
    assurance_level in ('aal1', 'aal2', 'aal3')
  ),
  passwordless_enabled boolean not null default false,
  mfa_enrolled boolean not null default false,
  last_authenticated_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
```

```
  deleted_at timestamptz
);

create table app.organization_memberships (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  person_id uuid not null references app.people(id),
  status app.membership_status not null default 'invited',
  external_subject_ref text,
  affiliation_type text not null check (
    affiliation_type in (
      'applicant',
      'student',
      'faculty',
      'staff',
      'administrator',
      'guardian',
      'alumnus',
      'partner',
      'vendor',
      'employee'
    )
  ),
  starts_at timestamptz not null default now(),
  ends_at timestamptz,
  source_system text not null default 'native',
  source_metadata jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique (tenant_id, person_id, affiliation_type, starts_at)
);

create table app.roles (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid references app.tenants(id),
  key text not null,
  display_name text not null,
  description text not null,
  is_platform_role boolean not null default false,
  is_system_role boolean not null default false,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  unique nulls not distinct (tenant_id, key)
);

create table app.capabilities (
  id uuid primary key default gen_random_uuid(),
  key text not null unique,
  domain text not null,
  action text not null,
  resource_type text not null,
  risk_level smallint not null check (risk_level between 0 and 5),
  requires_step_up boolean not null default false,
  requires_approval boolean not null default false,
  description text not null,
  created_at timestamptz not null default now()
);

create table app.role_capabilities (
  role_id uuid not null references app.roles(id) on delete cascade,
  capability_id uuid not null references app.capabilities(id) on delete cascade,
  scope_template jsonb not null default '{}'::jsonb,
  constraints jsonb not null default '{}'::jsonb,
  created_at timestamptz not null default now(),
```

```
  primary key (role_id, capability_id)
);

create table app.role_assignments (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  membership_id uuid not null references app.organization_memberships(id),
  role_id uuid not null references app.roles(id),
  status app.assignment_status not null default 'active',
  resource_scope jsonb not null default '{}'::jsonb,
  purpose_constraints jsonb not null default '[]'::jsonb,
  starts_at timestamptz not null default now(),
  ends_at timestamptz,
  granted_by_person_id uuid references app.people(id),
  approval_case_id uuid,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table app.relationships (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  subject_person_id uuid not null references app.people(id),
  related_person_id uuid not null references app.people(id),
  relationship_type text not null check (
    relationship_type in (
      'guardian_of',
      'parent_of',
      'payer_for',
      'advisor_of',
      'mentor_of',
      'tutor_of',
      'emergency_contact_for',
      'authorized_delegate_for',
      'employer_contact_for'
    )
  ),
  verification_status text not null check (
    verification_status in ('unverified', 'pending', 'verified', 'rejected', 'expired')
  ),
  status text not null check (
    status in ('active', 'pending', 'suspended', 'ended', 'revoked')
  ),
  starts_at timestamptz not null default now(),
  ends_at timestamptz,
  verification_evidence_ref text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  check (subject_person_id <> related_person_id)
);

create table app.consent_grants (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  subject_person_id uuid not null references app.people(id),
  grantee_person_id uuid references app.people(id),
  grantee_tenant_id uuid references app.tenants(id),
  relationship_id uuid references app.relationships(id),
  purpose text not null,
  data_categories app.data_classification[] not null,
  resource_scope jsonb not null default '{}'::jsonb,
  status app.consent_status not null default 'granted',
  legal_basis text not null,
  policy_version text not null,
```

```
  granted_at timestamptz not null default now(),
  expires_at timestamptz,
  revoked_at timestamptz,
  revocation_reason text,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now(),
  check (grantee_person_id is not null or grantee_tenant_id is not null)
);

create table app.tenant_policies (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  policy_key text not null,
  policy_version text not null,
  policy_document jsonb not null,
  effective_at timestamptz not null,
  expires_at timestamptz,
  status text not null check (status in ('draft', 'active', 'retired')),
  approved_by_person_id uuid references app.people(id),
  created_at timestamptz not null default now(),
  unique (tenant_id, policy_key, policy_version)
);

create table app.approval_cases (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  case_type text not null,
  status text not null check (
    status in ('draft', 'submitted', 'in_review', 'approved', 'rejected', 'cancelled', 'expired')
  ),
  requester_person_id uuid not null references app.people(id),
  target_resource_type text not null,
  target_resource_id uuid,
  requested_action text not null,
  risk_level smallint not null check (risk_level between 0 and 5),
  payload jsonb not null default '{}'::jsonb,
  decision jsonb,
  submitted_at timestamptz,
  decided_at timestamptz,
  expires_at timestamptz,
  created_at timestamptz not null default now(),
  updated_at timestamptz not null default now()
);

create table app.policy_decisions (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  correlation_id uuid not null,
  request_id uuid not null,
  actor_person_id uuid references app.people(id),
  action_key text not null,
  resource_type text not null,
  resource_id uuid,
  purpose text,
  outcome app.policy_decision_outcome not null,
  reason_codes text[] not null default '{}',
  evaluated_policies jsonb not null default '[]'::jsonb,
  decision_context jsonb not null default '{}'::jsonb,
  approval_case_id uuid references app.approval_cases(id),
  created_at timestamptz not null default now()
);

create table audit.events (
  id uuid primary key default gen_random_uuid(),
```

```
  occurred_at timestamptz not null default now(),
  tenant_id uuid references app.tenants(id),
  actor_person_id uuid references app.people(id),
  actor_account_id uuid references app.accounts(id),
  actor_role_snapshot jsonb not null default '[]'::jsonb,
  actor_assurance_level text,
  action text not null,
  resource_type text not null,
  resource_id uuid,
  resource_owner_person_id uuid references app.people(id),
  data_classification app.data_classification,
  purpose text,
  outcome app.audit_outcome not null,
  correlation_id uuid not null,
  request_id uuid,
  source_ip inet,
  user_agent text,
  client_type text,
  change_summary jsonb not null default '{}'::jsonb,
  before_hash text,
  after_hash text,
  policy_decision_id uuid references app.policy_decisions(id),
  approval_case_id uuid references app.approval_cases(id),
  integrity_hash text not null,
  previous_integrity_hash text
);

create index organization_memberships_tenant_person_idx
  on app.organization_memberships (tenant_id, person_id)
  where status = 'active';

create index role_assignments_active_idx
  on app.role_assignments (tenant_id, membership_id, role_id)
  where status = 'active';

create index relationships_subject_related_idx
  on app.relationships (tenant_id, subject_person_id, related_person_id)
  where status = 'active';

create index consent_grants_lookup_idx
  on app.consent_grants (tenant_id, subject_person_id, grantee_person_id, status)
  where status = 'granted';

create index audit_events_tenant_occurred_idx
  on audit.events (tenant_id, occurred_at desc);

create index policy_decisions_tenant_created_idx
  on app.policy_decisions (tenant_id, created_at desc);
```

Tenant context functions

Never let users supply the tenant as an untrusted query parameter and never set tenant context from raw client input. The authenticated API layer must establish it.

```
create or replace function private.current_tenant_id()
returns uuid
language sql
stable
as $$
  select nullif(current_setting('app.tenant_id', true), '')::uuid
$$;
```

```
create or replace function private.current_person_id()
returns uuid
language sql
stable
as $$
  select nullif(current_setting('app.person_id', true), '')::uuid
$$;

create or replace function private.require_tenant_context()
returns void
language plpgsql
stable
as $$
begin
  if private.current_tenant_id() is null then
    raise exception 'tenant context is required';
  end if;
end;
$$;

create or replace function private.set_request_context(
  p_tenant_id uuid,
  p_person_id uuid,
  p_request_id uuid,
  p_correlation_id uuid
)
returns void
language plpgsql
security definer
set search_path = private, pg_temp
as $$
begin
  perform set_config('app.tenant_id', p_tenant_id::text, true);
  perform set_config('app.person_id', p_person_id::text, true);
  perform set_config('app.request_id', p_request_id::text, true);
  perform set_config('app.correlation_id', p_correlation_id::text, true);
end;
$$;
```

Example tenant-owned table

Every domain record needs `tenant_id`, source metadata, lifecycle state, audit metadata, and a policy-appropriate owner.

```
create table app.student_tasks (
  id uuid primary key default gen_random_uuid(),
  tenant_id uuid not null references app.tenants(id),
  owner_person_id uuid not null references app.people(id),
  title text not null check (char_length(title) between 1 and 500),
  description text,
  status text not null default 'open' check (
    status in ('open', 'in_progress', 'completed', 'cancelled', 'archived')
  ),
  due_at timestamptz,
  source_metadata jsonb not null default jsonb_build_object(
    'origin', 'native',
    'freshnessState', 'current'
  ),
  data_classification app.data_classification not null default 'confidential',
  created_at timestamptz not null default now(),
```

```
  updated_at timestamptz not null default now(),
  deleted_at timestamptz
);

alter table app.student_tasks enable row level security;
alter table app.student_tasks force row level security;

create policy student_tasks_tenant_isolation
on app.student_tasks
as restrictive
for all
using (
  tenant_id = private.current_tenant_id()
)
with check (
  tenant_id = private.current_tenant_id()
);

create policy student_tasks_owner_access
on app.student_tasks
as permissive
for select
using (
  owner_person_id = private.current_person_id()
);

create policy student_tasks_owner_write
on app.student_tasks
as permissive
for insert
with check (
  owner_person_id = private.current_person_id()
);

create policy student_tasks_owner_update
on app.student_tasks
as permissive
for update
using (
  owner_person_id = private.current_person_id()
)
with check (
  owner_person_id = private.current_person_id()
);
```

Authorization evaluation pseudocode

```
type AuthorizationRequest = {
  tenantId: string
  actorPersonId: string
  action: string
  resource: {
    type: string
    id?: string
    ownerPersonId?: string
    classification?: string
    attributes?: Record<string, unknown>
  }
  purpose: string
  requestedAt: string
  client: {
    deviceId?: string
```

```
    assuranceLevel: "aal1" | "aal2" | "aal3"
  }
}

async function authorize(
  request: AuthorizationRequest
): Promise<
  | { outcome: "allow"; decisionId: string }
  | { outcome: "require_step_up"; decisionId: string }
  | { outcome: "require_approval"; decisionId: string; approvalCaseId: string }
  | { outcome: "deny"; decisionId: string; reasonCodes: string[] }
> {
  const context = await loadAuthorizationContext(request)

  assertTenantMembership(context)
  assertActiveAccount(context)
  assertActiveRoleOrRelationship(context)
  assertTenantPolicyAllowsAction(context)
  assertCapabilityGrantAllowsAction(context)
  assertResourceScopeAllowsAction(context)
  assertPurposeIsPermitted(context)
  assertDataClassificationPermitsAccess(context)
  assertConsentOrLegalBasisWhereRequired(context)
  assertDeviceAndSessionAssurance(context)
  assertNoLegalHoldOrSafetyRestrictionBlocksAction(context)

  const decision = await evaluateRiskAndWorkflow(context)

  await recordPolicyDecision(decision, context)

  return decision
}
```

Authorization invariants

These become automated tests and database constraints:

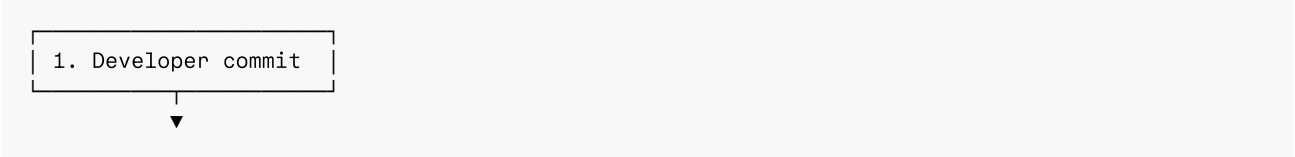
1. No query may return a record from another tenant unless an explicitly approved cross-tenant relationship applies.
2. A platform support agent cannot access customer content without an approved, expiring, audited support grant.
3. A guardian can access only explicit, lawful, active consent scopes tied to a verified relationship.
4. A faculty member can access only assigned-course data, never institution-wide student data by default.
5. Grade changes require an approved grade-change workflow, reason code, actor identity, before/after evidence, and immutable audit event.
6. Institutional admins can configure policy but cannot erase audit trails.
7. AI retrieval uses the same policy decision as direct user access—no special bypass.
8. Background jobs, search, analytics, exports, queues, storage, and support tooling must enforce the same tenant boundary.
9. Connection pooling must not retain a prior request's tenant context.
10. Every privileged action must include a correlation ID and audit event.

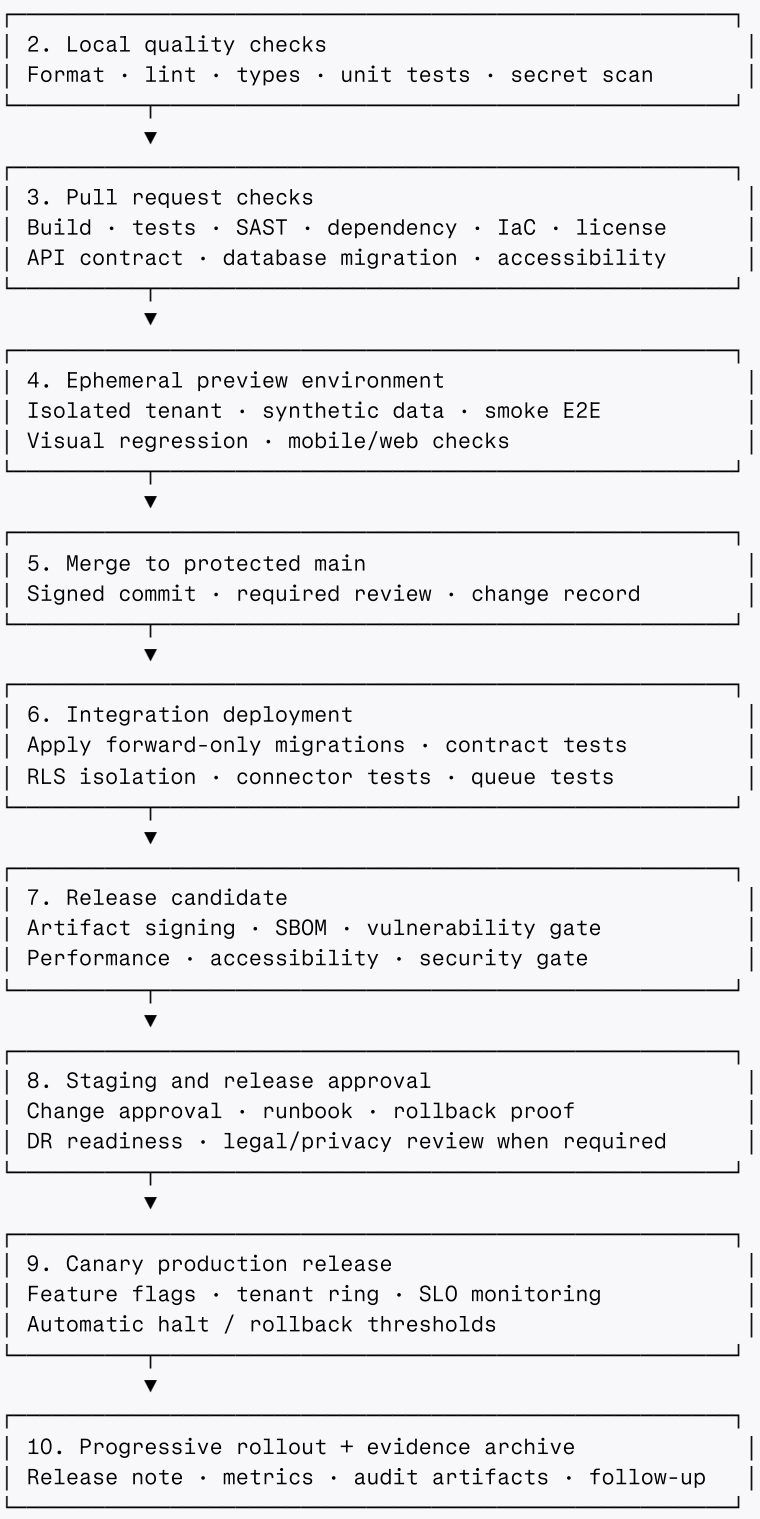
3. Production Deployment Pipeline

Environment topology

```
Developer workstation
      │
      ▼
Pull request environment
      │
      ├── Lint / typecheck / unit / component tests
      ├── Secret scan / dependency scan / SAST
      ├── Preview deployment
      ├── Accessibility smoke tests
      └── Contract tests
      │
      ▼
Integration environment
      │
      ├── Real migrations
      ├── Seeded synthetic tenant data
      ├── API / integration / RLS tests
      ├── Mobile sync tests
      └── DAST / performance smoke
      │
      ▼
Staging environment
      │
      ├── Production-like topology
      ├── Approved test connectors
      ├── Disaster recovery rehearsal
      ├── Load tests
      ├── Security and acceptance testing
      └── Release candidate approval
      │
      ▼
Production canary ring
      │
      ├── Internal tenant
      ├── Design-partner tenant
      ├── Low-risk cohort
      └── Automated metric/SLO checks
      │
      ▼
Production staged rollout
      │
      ├── 1%
      ├── 10%
      ├── 25%
      ├── 50%
      └── 100%
      │
      ▼
Post-release validation and evidence archive
```

CI/CD pipeline





Required pipeline gates

Stage	Gate	Block release if
Commit	Lint, type safety, formatting, secrets scan	Any secret, type failure, lint error
Pull request	Code review, tests, dependency/SAST scan	Required reviewer absent, critical vulnerability, failed tests
Schema change	Migration review, rollback plan, RLS test, data migration test	Destructive/unreviewed migration or tenant-isolation failure
Preview	E2E smoke, accessibility smoke, visual regression	Critical journey failure or accessibility regression

Stage	Gate	Block release if
Integration	API contract, RLS, job, queue, connector, sync tests	Cross-tenant access, contract break, unrecoverable job error
Staging	Load, chaos, restore, DAST, full acceptance	SLO failure, data-loss risk, unresolved security defect
Release	Change record, release owner, incident/rollback readiness	No accountable owner or no safe rollback/cutover path
Canary	SLO and business metric monitoring	Error rate, latency, auth failure, policy denial anomaly, integrity mismatch
Full release	Post-release review	Unresolved canary issue or customer-impact signal

Migration safety protocol

```
1. Additive schema migration
2. Deploy code that can read old and new structures
3. Backfill through idempotent job
4. Reconcile counts and sample records
5. Enable new write path behind feature flag
6. Observe and compare metrics
7. Migrate readers
8. Archive legacy path only after retention period
9. Remove old columns in a separately approved release
```

Never combine destructive database changes with a release that depends on them unless there is a tested recovery plan and data backup/restore evidence.

Deployment workflow example

```yaml
name: semester-production-release

on:
  workflow_dispatch:
    inputs:
      release_sha:
        required: true
      rollout_ring:
        required: true
        type: choice
        options: [internal, design_partner, canary, ten_percent, full]

jobs:
  verify-artifact:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ inputs.release_sha }}
      - run: npm ci
      - run: npm run verify:artifact
      - run: npm run verify:sbom
      - run: npm run verify:signatures

  verify-release-gates:
    needs: verify-artifact
    runs-on: ubuntu-latest
    steps:
      - run: npm run release:checklist
      - run: npm run release:database-plan
```

```
      - run: npm run release:rollback-plan
      - run: npm run release:approval-check

  migrate:
    needs: verify-release-gates
    runs-on: ubuntu-latest
    environment: production-migrations
    steps:
      - run: npm run db:migrate:production
      - run: npm run db:verify:rls
      - run: npm run db:verify:integrity

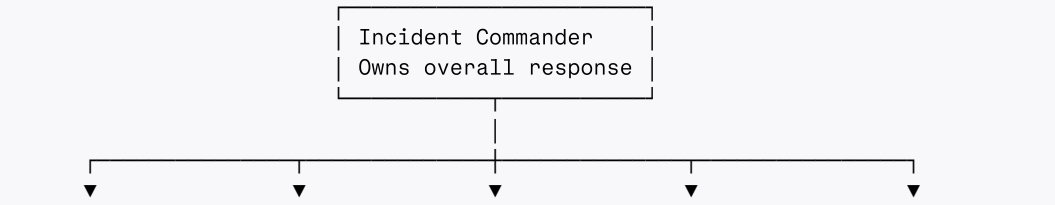
  deploy-canary:
    needs: migrate
    runs-on: ubuntu-latest
    environment: production
    steps:
      - run: npm run deploy -- --ring=${{ inputs.rollout_ring }}
      - run: npm run smoke:production -- --ring=${{ inputs.rollout_ring }}

  observe:
    needs: deploy-canary
    runs-on: ubuntu-latest
    steps:
      - run: npm run observe:release -- --duration=30m
      - run: npm run release:assert-slo
      - run: npm run release:assert-no-security-anomalies
```

4. Incident Response and Support Runbook

NIST SP 800-61 Rev. 3 aligns incident response with the CSF 2.0 functions: Govern, Identify, Protect, Detect, Respond, and Recover. It emphasizes preparation before incidents and continuous improvement afterwards. CISA similarly emphasizes documented procedures, instrumentation, coordinated incident declaration, containment, remediation, recovery, and tracking.[41] [42] [43] [44]

Incident command structure



Severity definitions

Severity	Definition	Examples	Initial response
SEV-0	Confirmed security/privacy breach or integrity failure with broad/highly sensitive impact	Cross-tenant data exposure; unauthorized official record changes; confirmed exfiltration	Immediate
SEV-1	Critical service outage or major security event affecting critical student/institution flows	Auth outage; submissions unavailable near deadline; registration failure; payment integrity issue	15 minutes

Severity	Definition	Examples	Initial response
SEV-2	Major degradation with material workaround or limited tenant impact	One tenant's connector failure; grade display delay; partial course access issue	30 minutes
SEV-3	Limited functionality issue with reasonable workaround	Single feature outage; non-critical report failure	Same business day
SEV-4	Minor defect, question, or cosmetic issue	Layout defect; copy issue; low-impact bug	Routine triage

Incident lifecycle

```
Detect
  |
  ▼
Declare and classify
  |
  ▼
Assign incident commander and response roles
  |
  ▼
Contain impact
  |
  ▼
Preserve evidence and investigate
  |
  ▼
Eradicate root cause / apply mitigation
  |
  ▼
Recover safely and validate
  |
  ▼
Communicate resolution
  |
  ▼
Post-incident review and control improvements
```

First 15-minute checklist

1. Create an incident channel and incident record with a unique ID.
2. Assign an incident commander, technical lead, security lead if relevant, communications lead, customer lead, and scribe.
3. Record first-known time, detection source, systems affected, tenant impact, suspected data classification, and current severity.
4. Freeze non-essential deployments and high-risk administrative changes.
5. Preserve relevant logs, traces, snapshots, queue state, connector state, and audit records.
6. Check for tenant-isolation, sensitive-data, academic-record, billing, submission, and authentication impact.
7. Apply immediate containment only if it reduces risk without causing unsafe secondary effects.
8. Open customer support macros and establish a status communication cadence.
9. Engage qualified legal counsel for suspected privacy/security breach, contractual notification, law-enforcement, or regulatory questions.
10. Set the next update time even if no new information exists.

Security incident playbooks

A. Suspected cross-tenant access

```
Trigger:
- RLS alert, suspicious query pattern, support report, bug report, or audit anomaly.

Contain:
1. Disable affected endpoint, query path, feature flag, or integration immediately.
2. Revoke active sessions/tokens if compromise is possible.
3. Pause background jobs and exports that use the same path.
4. Preserve logs, database query evidence, audit events, deployment version, and request IDs.

Investigate:
1. Determine tenant A, tenant B, data classes, records, and time window.
2. Identify whether exposure was view-only, exported, modified, or automated.
3. Inspect connection-pool/session-context behavior.
4. Identify code, policy, migration, cache, search, analytics, or support-tool path.

Recover:
1. Patch the authorization fault.
2. Add automated regression coverage for the exact failure.
3. Run a comprehensive cross-tenant scan for the same control class.
4. Validate with production-like request roles and connection reuse.
5. Consult qualified counsel on notification duties and customer communications.

Close:
- Root-cause analysis, remediation owner, due dates, executive review, evidence archive.
```

B. Compromised account

```
Trigger:
- Impossible travel, unusual device/session, user report, credential leak signal, account takeove

Contain:
1. Revoke sessions and refresh tokens.
2. Disable high-risk actions and payment/record changes for the account.
3. Require step-up reauthentication and recovery verification.
4. Preserve sign-in, IP, device, MFA, audit, and command history.

Investigate:
1. Determine compromised scope and period.
2. Review data accessed, exports, role changes, actions, and connected accounts.
3. Check for persistence: MFA enrollment, recovery changes, API tokens, delegated access.

Recover:
1. Restore verified account control.
2. Reverse unauthorized changes through auditable correction workflows.
3. Notify affected users/tenants according to approved policy and counsel guidance.
4. Improve detection rules and regression tests.
```

C. Official record or grade integrity incident

```
Trigger:
- Unexpected grade/record change, reconciliation mismatch, faculty/staff report, audit anomaly.

Contain:
1. Freeze affected record workflow or grade-release job.
```

```
2. Preserve before/after versions, audit events, approvals, integrations, and actor identities.
3. Do not silently overwrite or "fix" records without preserving evidence.

Investigate:
1. Determine whether the change was authorized, correct, duplicated, malicious, or sync-induced.
2. Check source-of-truth precedence and connector mappings.
3. Identify all affected students, terms, sections, and downstream impacts.

Recover:
1. Use a formal correction workflow with documented authority.
2. Notify affected institutional owners.
3. Reconcile with the authoritative source and verify downstream systems.
4. Add invariant/contract tests for the failure mode.
```

D. AI safety or data-leak incident

```
Trigger:
- AI reveals protected information, executes/attempts unsafe action, produces harmful advice,
  uses unauthorized sources, or bypasses a policy boundary.

Contain:
1. Disable affected prompt/tool/model route/feature flag.
2. Preserve prompts, context assembly logs, policy decisions, tool calls, sources, and outputs.
3. Stop automated follow-on actions and prevent replay.

Investigate:
1. Classify affected data and users.
2. Determine whether the fault was retrieval filtering, policy, tool authorization, model output,
   prompt injection, model-provider behavior, or user interface ambiguity.
3. Examine related prompts and feature cohorts.

Recover:
1. Patch policy/retrieval/tool boundary.
2. Add red-team regression scenarios.
3. Re-run evaluation suite before re-enable.
4. Notify affected parties under approved privacy/legal process.
```

Customer support runbook

Ticket type	First-line response	Escalation	Evidence required
Login/access	Verify account safely; guide recovery; check incident status	Identity/security if suspicious	Account ID, tenant, session/device metadata, no passwords
Assignment/submission	Check submission receipt, storage state, deadline policy, connector health	Learning/SRE if receipt mismatch	Correlation ID, timestamp, course/assignment IDs
Grade concern	Explain official source and process; do not alter grade	Faculty/registrar workflow	Grade record, audit trail, policy basis
Billing/payment	Verify status, receipt, plan, processor result	Finance/payment ops	Invoice/payment IDs, redacted processor evidence
Privacy request	Authenticate requester; intake request without promising outcome	Privacy/legal workflow	Identity proof, request type, relevant tenant
Safety report	Acknowledge safely; preserve report; avoid promising confidentiality beyond policy	Trust & safety / emergency policy	Case ID, content/report metadata, risk assessment
Integration issue	Check connector health, last sync, source scope, reconciliation status	Integration engineering / institution IT	Connection ID, job ID, mapping version

Ticket type	First-line response	Escalation	Evidence required
Accessibility barrier	Capture task, assistive technology, browser/device, impact	Accessibility lead / product	Repro steps, screenshots only with consent
Suspected security issue	Do not request secrets; preserve evidence; create security case	Security incident process	Reporter contact, evidence, time, affected service

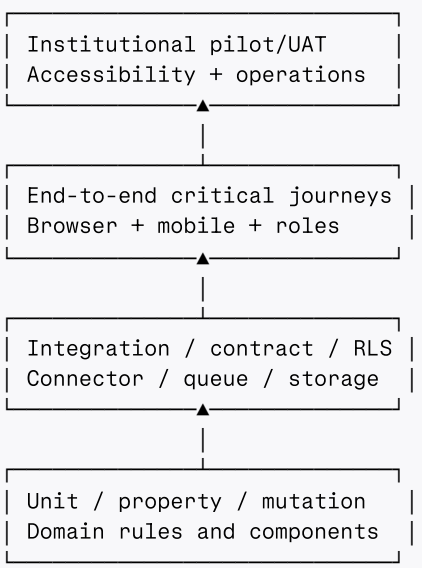
Support access rules

A support agent may not:

- Browse student records out of curiosity.
- Use a global “impersonate user” function without explicit policy, consent or lawful basis, scope, expiry, and audit.
- View student-private notes/documents unless necessary, authorized, time-bound, and clearly logged.
- Modify grades, official records, role assignments, billing amounts, or consent relationships outside the corresponding governed workflow.
- Ask users to send passwords, MFA codes, full payment information, or sensitive documents over unapproved channels.

5. Complete Test Strategy and Quality Gates

Testing model



Test strategy by layer

Layer	Test type	Required coverage
Design system	Visual regression, keyboard, screen reader, contrast, reduced motion	Every shared component and state
Frontend	Unit, component, interaction, route, state-machine tests	Loading, success, error, offline, denied, pending, recovery
Mobile	Device matrix, offline, encrypted storage, resume, deep links, accessibility	iOS/Android target versions and low-connectivity scenarios

Layer	Test type	Required coverage
APIs	Unit, contract, schema validation, idempotency, authorization	Every endpoint and command
Database	Migration, RLS, tenant isolation, constraints, transaction, retention	Every tenant-scoped and sensitive table
Domain services	Invariant, property, mutation, workflow, idempotency tests	Grades, registration, billing, consent, permissions
Integrations	Sandbox, replay, schema drift, retry, reconciliation, degraded state	Every connector and mapping version
AI	Red-team, retrieval permissions, grounding, policy, tool-call, privacy	Every model route and tool
Security	SAST, DAST, dependency scan, secrets, auth, session, rate limiting	Every release pipeline
Reliability	Load, soak, chaos, DR, backup/restore, queue backlog	Critical journeys and high-volume dates
Operations	Alert tests, runbook simulation, status update drill	Every critical service
Business	Pricing/entitlement tests, invoice tests, permissions by plan	Every paid entitlement
Institutional	Pilot UAT, role workflow, data migration verification	Each tenant go-live

Critical journey test catalog

Journey ID	Journey	Required proof
CJ-01	Student account creation, verification, MFA/passkey, onboarding	Auth security, accessibility, audit, tenant assignment
CJ-02	Student views Today and receives accurate prioritized actions	Data provenance, offline behavior, notification policy
CJ-03	Student creates task offline and syncs later	Encryption, idempotency, conflict handling, no duplicate
CJ-04	Student opens course, completes work, submits assignment	Receipt, storage durability, deadline correctness, accessibility
CJ-05	Faculty grades submission and releases feedback	Course scope, rubric, grade audit, student notification
CJ-06	Student views official grade and record history	Source precedence, freshness, access control
CJ-07	Student registers for a course	Eligibility, holds, capacity, transaction integrity, confirmation
CJ-08	Guardian accesses authorized information only	Relationship verification, consent, denial of non-shared data
CJ-09	Student views/pays bill or starts plan	Finance authorization, reconciliation, no card-data storage
CJ-10	Student uses AI tutor with course sources	Authorization-filtered retrieval, citations, safety behavior
CJ-11	Institution admin configures a policy	Approval/audit, scope limits, effective date
CJ-12	SIS/LMS connector syncs data and reconciles discrepancy	Health state, source labeling, error journal, fallback
CJ-13	Support agent resolves ticket with scoped access	Access expiry, audit, redaction, no privilege escalation
CJ-14	User requests export/deletion	Identity verification, retention/legal-hold behavior, status tracking
CJ-15	Incident triggers and platform recovers	Alerting, containment, restoration, customer communication evidence

Quality gates

Gate	Required before merge	Required before staging	Required before production
Code quality	Format, lint, types, unit tests	Mutation/invariant coverage for high-risk logic	No waived critical failures
Security	Secret scan, dependency baseline	SAST/DAST, auth test, threat review for high-risk changes	No unresolved critical/high exploitable issue

Gate	Required before merge	Required before staging	Required before production
Tenant isolation	Relevant unit/API tests	RLS, pooling, cache/search/queue isolation suite	Production verification and monitoring
Accessibility	Component checks	Manual critical-journey test	No critical blocker; documented conformance evidence
Data	Migration validation	Backfill/reconciliation/restore test	Approved cutover and rollback plan
AI	Unit/policy tests	Evaluation/red-team suite	Approved safety case and monitoring
Reliability	Smoke tests	Load/chaos/DR where applicable	SLO capacity and rollback evidence
Integration	Mock contract	Sandbox/replay/reconciliation	Tenant-approved scope and health monitoring
Operations	Runbook updated	Alert test, dashboard available	On-call owner and support readiness
Commercial	Entitlement tests	Pricing/usage-meter tests	Support/documentation/customer communication ready
Legal/privacy	Data review if scope changes	Counsel review where applicable	Claims/contracts approved by appropriate humans

Release quality score

```
Release readiness score =

0.20 × critical journey pass rate
+ 0.15 × security gate health
+ 0.15 × tenant isolation confidence
+ 0.10 × accessibility confidence
+ 0.10 × reliability/SLO confidence
+ 0.10 × data migration/reconciliation confidence
+ 0.10 × AI safety confidence
+ 0.10 × operational readiness

Production release allowed only if:
- Every critical journey passes.
- Security, tenant isolation, and data integrity have no unresolved blocker.
- Rollback is proven or a documented forward-recovery plan is approved.
- Accountable release owner and incident owner are assigned.
```

6. Codex Prompts by Role

The prompts below are designed to be used **individually**. Each role should work from the same shared foundation documents:

- `docs/PRODUCT_THESIS.md`
- `docs/CAPABILITY_TRACEABILITY_MATRIX.md`
- `docs/ARCHITECTURE.md`
- `docs/SECURITY_PRIVACY_BASELINE.md`
- `docs/AI_GOVERNANCE.md`
- `docs/QUALITY_GATES.md`
- `docs/INSTITUTIONAL_IMPLEMENTATION.md`
- `docs/DECISION_LOG.md`

Shared preamble for every Codex prompt

```
You are working on Semester, a production-grade, multi-tenant Student Operating System.

Semester is natively, from inception:
- Student OS
- Productivity suite
- AI executive assistant, tutor, and advisor
- Campus super-app
- Institutional system
- Family/guardian portal
- Marketplace
- Career, alumni, and lifelong learner platform

Core thesis:
“Every part of a student’s educational life works in Semester from day one—natively when necessar

Non-negotiable rules:
1. Preserve all existing intended capabilities as requirements; do not silently reduce scope.
2. Build capabilities natively. External integrations enhance and synchronize; they never become
3. Every capability must be tenant-aware, authorized, auditable, accessible, observable, supporta
4. Do not create a fake success state. Clearly represent loading, pending, offline, denied, faile
5. Never bypass policy, tenant isolation, consent, role scope, or audit logging.
6. Use the existing repository as product-intent evidence, not as a requirement to preserve its a
7. Do not make legal conclusions or approve contracts. Flag matters for qualified human counsel.
8. Before making changes, inspect relevant architecture, contracts, tests, migrations, and docume
9. Update traceability, tests, runbooks, and decision records with every material change.
10. Do not claim something is production-ready without test, operational, and evidence artifacts.

Your outputs must include:
- Assumptions
- Risks and unresolved questions
- Files changed or proposed
- Tests added
- Accessibility implications
- Security/privacy implications
- Operational/runbook implications
- Traceability matrix updates
```

Chief Executive Officer and founder strategist

```
[Use the shared preamble.]

Act as Semester’s founder-level CEO and corporate strategist.

Your mission:
Turn Semester into a defensible, category-defining education technology company that can become t

Deliver:
1. A three-year company strategy with target segments, sequencing, market positioning, expansion
2. A board-level strategy memo defining the core thesis, strategic risks, milestones, capital req
3. A strategy scorecard with North Star metrics, leading indicators, operating targets, and revie
4. A market-entry plan that preserves full platform ambition while avoiding unsafe or unprovable
5. A founder decision log identifying choices requiring executive approval.
6. A competitive moat plan across product, data, workflows, integrations, brand, customer success
7. A risk register covering product, company, capital, legal, security, reputation, delivery, and
8. A list of required advisors: higher education administration, registrar, accessibility, privac

Do not write vague strategy. Define measurable outcomes, owners, deadlines, evidence requirements
```

Chief Product Officer

```
[Use the shared preamble.]

Act as Chief Product Officer for Semester.

Your mission:
Convert the entire existing Semester blueprint into a coherent, complete, native product requirem

Produce:
1. A complete capability inventory grouped into bounded domains.
2. A product requirements document for every domain:
   identity, student OS, productivity, academic core, learning, AI, campus, family, finance, care
3. Personas and jobs-to-be-done for student, applicant, faculty, advisor, tutor, registrar, finan
4. Critical journey maps, acceptance criteria, error states, offline states, approvals, and succe
5. A complete “definition of done” checklist per capability.
6. A prioritization model that preserves every domain at launch while safely distinguishing nativ
7. A requirement traceability matrix mapping current repository capabilities to target product re
8. A product telemetry plan and outcome metrics per capability.
9. A roadmap with dependencies, release gates, and discovery questions.

Do not produce only feature lists. Define ownership, authority, data, workflows, edge cases, and
```

Chief Technology Officer

```
[Use the shared preamble.]

Act as Chief Technology Officer for Semester.

Your mission:
Design and lead a scalable, secure, operable, multi-tenant technology platform that can support a

Deliver:
1. Target architecture diagrams for clients, APIs, domain services, policy, data, events, integra
2. A monorepo structure with apps, services, packages, infrastructure, contracts, and docs.
3. Technology choices with decision criteria, alternatives, risks, and ADRs.
4. A domain-service boundary plan and ownership model.
5. A production environment strategy: local, preview, integration, staging, production, canary, t
6. CI/CD, release, rollback, migration, secrets, backup, disaster recovery, and observability pla
7. Engineering standards: API contracts, events, idempotency, correlation IDs, feature flags, cod
8. Staffing plan, team topology, delivery milestones, architectural review process, and technical
9. A conversion plan for the existing React/Vite/Supabase implementation into the target architec

Be concrete. Produce implementation-ready artifacts, repository paths, interfaces, migration sequ
```

Chief Financial Officer

```
[Use the shared preamble.]

Act as Chief Financial Officer and SaaS unit-economics architect for Semester.

Your mission:
Create the financial operating model required to build, launch, sell, support, and scale Semester

Deliver:
1. A bottom-up three-year financial model with clearly labeled assumptions.
2. Revenue streams: student subscription, institutional platform, implementation services, AI usa
3. Pricing and packaging architecture with entitlements, discounts, approvals, billing, collectio
4. Cost model: engineering, AI inference, infrastructure, storage, bandwidth, search, integration
```

```
5. Unit-economics dashboard: CAC, payback, gross margin, contribution margin, churn, NRR, burn, r
6. Budget governance and spending approval matrix.
7. Customer contract financial requirements, invoice flow, tax and marketplace concerns to route
8. Scenario analysis: conservative, base, aggressive, enterprise-delayed, high-AI-cost, and incid
9. Financial controls and finance operations processes.
10. A founder/board finance reporting package.

Do not state legal, tax, or accounting conclusions as definitive. Flag them for qualified profess
```

Chief Revenue Officer

```
[Use the shared preamble.]

Act as Chief Revenue Officer and institutional EdTech procurement strategist for Semester.

Your mission:
Create a repeatable enterprise and product-led revenue engine for a complete Student Operating Sy

Deliver:
1. Ideal customer profiles and buying committees for higher education, K-12 where relevant, depar
2. A segmentation and account-prioritization model.
3. Sales motion by segment: student-led, department-led, institutional enterprise, partner/market
4. Discovery framework, qualification criteria, objection handling, ROI model, business case temp
5. Procurement strategy: security review, accessibility review, privacy review, legal review, IT
6. Sales stages, exit criteria, forecasting model, deal desk rules, discount approvals, and hando
7. Institutional pilot-to-production conversion playbook.
8. Required sales collateral and evidence artifacts.
9. Customer reference strategy and renewal/expansion model.
10. Revenue operations data model and dashboards.

Never make claims that are unsupported by actual platform evidence or approved public-claims docu
```

Chief Marketing Officer

```
[Use the shared preamble.]

Act as Chief Marketing Officer, brand strategist, growth leader, lifecycle marketer, content lead

Your mission:
Build a premium, trusted, category-defining brand for a native Student Operating System while avo

Deliver:
1. Brand strategy: positioning, purpose, narrative, voice, values, brand promise, differentiation
2. Messaging architecture for students, families, faculty, administrators, IT/security, procureme
3. Visual identity and brand system requirements aligned to accessibility.
4. Website architecture, information architecture, page briefs, conversion flows, and trust cente
5. Launch campaign, demand-generation plan, content calendar, social strategy, PR strategy, commu
6. Student growth loops, institutional demand creation, referral model, ambassador strategy, and
7. Marketing operations: attribution, consent, lead scoring, campaign QA, suppression rules, comp
8. Public claims register: every claim, evidence, approval owner, review date, and risk level.
9. Competitive positioning and category-creation narrative.
10. Metrics: awareness, activation, conversion, retention, pipeline, CAC, and brand trust.

Do not invent certifications, compliance claims, customer results, integrations, or capabilities.
```

Chief Operating Officer

```
[Use the shared preamble.]

Act as Chief Operating Officer for Semester.

Your mission:
Design the operating system that allows Semester to implement, support, govern, and scale its pla

Deliver:
1. Company operating cadence: weekly, monthly, quarterly, annual planning and review structure.
2. Customer implementation methodology from discovery through go-live, hypercare, adoption, renew
3. Operational RACI and decision-rights matrix.
4. Support operating model, case taxonomy, SLA framework, escalation policy, and staffing model.
5. Incident management, business continuity, vendor management, quality operations, and change ma
6. Internal processes for documentation, training, enablement, and knowledge management.
7. Marketplace operations, partner operations, trust/safety operations, and fulfillment/dispute o
8. Operational dashboards and leading indicators.
9. A risk and readiness system for each tenant launch.
10. A plan for translating founder-led execution into repeatable teams and processes.

Include templates, service blueprints, handoff definitions, and measurable service levels.
```

Chief Information Security Officer

```
[Use the shared preamble.]

Act as Chief Information Security Officer for Semester.

Your mission:
Build a security program that protects multi-tenant student, family, institutional, financial, an

Deliver:
1. Security architecture and threat model for every product domain.
2. Identity, authentication, MFA/passkey, session, device, privileged access, and break-glass con
3. Tenant-isolation architecture and verification suite.
4. Secure SDLC: code review, secrets, SAST, DAST, dependency, SBOM, signing, IaC scanning, releas
5. Data protection: encryption, key management, backups, logs, storage, secrets, device/local dat
6. Vulnerability management program and severity SLA.
7. Security monitoring, detection engineering, SIEM/logging, alerting, and incident response.
8. Third-party/vendor risk management.
9. Security evidence library and procurement questionnaire response plan.
10. Security training, access reviews, tabletop exercises, and audit schedule.

Produce actionable controls, repository artifacts, test cases, and owner assignments. Do not clai
```

Chief Legal, Privacy, and Compliance Strategist

```
[Use the shared preamble.]

Act as Semester's legal, privacy, and compliance strategy coordinator.

Strict constraint:
You are not qualified human counsel. Do not provide definitive legal conclusions, approve contrac

Deliver:
1. Legal and regulatory issue map by product domain and jurisdictional question.
2. Data protection impact assessment framework.
3. Privacy program requirements: data inventory, classification, retention, deletion, export, con
```

```
4. Contracting artifact list: MSA, DPA, terms, privacy notice, AI terms, marketplace terms, partn
5. Public claims review process.
6. Education-record, accessibility, child/minor, consumer, payment, marketplace, marketing, emplo
7. Legal-hold, records retention, litigation response, and subject-rights intake procedures.
8. A legal-review queue with priority, owner, facts needed, external-counsel question, and decisi
9. Compliance evidence matrix mapped to controls and product capabilities.
10. Incident-notification decision workflow that requires counsel and customer-contract review.

Label all legal items as “requires qualified human counsel review.”
```

Principal Platform Architect

```
[Use the shared preamble.]

Act as Principal Platform Architect.

Your mission:
Create the reusable platform primitives that every Semester domain uses consistently.

Build and document:
1. Multi-tenancy and organization model.
2. Identity, affiliations, roles, relationships, consent, capabilities, policy evaluation, approv
3. API gateway/BFF standards, service authentication, request context, idempotency, correlation I
4. Workflow engine, notification engine, file service, search, feature flags, billing entitlement
5. Event schema and outbox pattern.
6. Tenant isolation controls across database, object storage, queues, cache, search, analytics, s
7. Observability standards and operational metadata.
8. Reference implementations and shared SDKs.
9. ADRs and architecture tests enforcing boundaries.
10. A migration path from the existing platform implementation.

Provide code skeletons, package structure, interfaces, schema contracts, and test suites.
```

Principal Software Architect

```
[Use the shared preamble.]

Act as Principal Software Architect.

Your mission:
Transform the existing Semester codebase into bounded, maintainable, testable product domains wit

Deliver:
1. Current-state architecture audit: coupling, duplication, client-only logic, state boundaries,
2. Target module graph and dependency rules.
3. Domain boundary plan and anti-corruption layers around existing code.
4. Refactoring sequence that keeps the application deployable.
5. Coding standards, error model, state-machine patterns, dependency injection, configuration man
6. ADRs for monolith modularization versus service extraction.
7. Repository restructuring plan with incremental commits and validation criteria.
8. Legacy retirement plan: reusable, migrate, replace, delete, archive.
9. Architectural test rules to prevent regression into a screen-centric monolith.
10. Example vertical-slice implementation for identity, policy, Today, tasks, and calendar.

Do not recommend a big-bang rewrite without a safe coexistence and migration approach.
```

Principal Data Architect

```
[Use the shared preamble.]

Act as Principal Data Architect.

Your mission:
Define Semester’s canonical data architecture, provenance system, lifecycle governance, analytics

Deliver:
1. Canonical entity model covering identity, academic, learning, productivity, campus, family, fi
2. Source-of-truth and precedence matrix for every major entity.
3. Transactional schema conventions: tenant IDs, ownership, source metadata, classification, life
4. Event schema and data lineage strategy.
5. Data classification, retention, deletion, legal hold, backup, export, and restore model.
6. Analytics architecture that protects privacy and does not become a shadow system of record.
7. Data-quality rules, reconciliation reports, schema-drift handling, and migration verification.
8. Search and retrieval architecture including security filters.
9. AI data access/retrieval controls.
10. Database migration, indexing, partitioning, performance, and archival strategy.

Produce ERDs, SQL schema proposals, test plans, and a data governance operating model.
```

Principal Frontend Engineer

```
[Use the shared preamble.]

Act as Principal Frontend Engineer.

Your mission:
Build Semester’s web experiences as accessible, high-performance, resilient applications that mak

Deliver:
1. Frontend architecture for student, faculty, staff, guardian, alumni, partner, and support appl
2. Shared application shell, navigation, search, notification, state, routing, error boundaries,
3. Responsive information architecture and device-specific behavior.
4. Offline-aware UX behavior: local state, sync queue, conflict states, stale data indicators, an
5. Design-system implementation using tokens, primitives, components, patterns, visual regression
6. Full state model for loading, empty, denied, offline, pending, degraded, failed, and recovery.
7. Performance budgets, bundle strategy, code splitting, caching, and observability.
8. Component test, E2E test, accessibility test, visual test, and keyboard navigation strategy.
9. Migration plan from the existing Vite/React UI without losing capabilities.
10. First production vertical slice: authenticated student shell, Today, tasks, calendar, notific

Do not create static mock-only screens. All UI must bind to real contracts and represent real ope
```

Principal Backend and API Engineer

```
[Use the shared preamble.]

Act as Principal Backend and API Engineer.

Your mission:
Build the secure, versioned domain APIs and workflow services that make Semester native, governed

Deliver:
1. API conventions for commands, queries, errors, pagination, idempotency, correlation IDs, versi
2. Service implementation plans for identity, policy, academic, learning, productivity, finance,
3. Authorization middleware and policy-decision integration for every endpoint and background job
```

```
4. Transaction boundaries, outbox pattern, event publishing, retries, dead-letter queues, and rec
5. Database migrations and RLS/service-layer enforcement patterns.
6. File upload/download protection, malware scanning, signed URLs, retention, and audit.
7. Rate limits, abuse controls, validation, and secure error handling.
8. Contract testing and API documentation.
9. Production observability and operational endpoints.
10. Example implementation for a tenant-scoped task/calendar service with audit events and offlin

All actions must enforce tenant, role, purpose, scope, and data-classification controls.
```

Mobile, Offline, SQLCipher, and CRDT Architect

```
[Use the shared preamble.]

Act as Mobile, Offline, SQLCipher, and CRDT Architect.

Your mission:
Ensure Semester works reliably across iOS, Android, web, weak networks, intermittent connectivity

Deliver:
1. Mobile application architecture and platform choices with tradeoffs.
2. Encrypted local storage design using SQLCipher or an equivalent approved encrypted database.
3. Device registration, device trust, remote revocation, biometric/passkey integration, session m
4. Offline synchronization protocol with command queue, idempotency, cursoring, retry, backoff, r
5. Conflict-resolution model by data type.
6. CRDT design only for suitable collaborative artifacts: notes, documents, comments, group work.
7. Explicit prohibition and server-authoritative flows for registration, grades, financial transa
8. Attachment caching, encryption, storage quotas, content access revocation, and malware handlin
9. Offline test matrix across network transitions, device changes, app upgrades, account removal,
10. Reference implementation and SDK interfaces.

Every mobile/offline workflow must show whether it is local, pending, synced, rejected, or confli
```

AI Governance Architect

```
[Use the shared preamble.]

Act as AI Governance Architect.

Your mission:
Build Semester's AI platform so that every AI assistant, tutor, advisor, writer, study tool, and

Deliver:
1. AI system inventory and risk-tiering framework.
2. Model gateway architecture, routing, cost controls, tenancy, regional/data controls, provider
3. Retrieval policy design: authorization-filtered retrieval, provenance, freshness, data minimiz
4. Tool broker design: permissions, user confirmation, human approvals, action previews, audit, r
5. AI feature requirements for tutor, advisor, planner, writer, research assistant, support assis
6. Evaluation framework: accuracy, grounding, safety, bias, privacy leakage, prompt injection, to
7. AI incident response and feature shutdown mechanisms.
8. Tenant AI configuration console requirements.
9. User-facing transparency: sources, limitations, feedback, data controls, and escalation.
10. Red-team test suites and launch gates by risk tier.

Never allow AI to make unreviewed high-impact decisions or bypass policy boundaries.
```

Design Systems Lead

```
[Use the shared preamble.]

Act as Design Systems Lead.

Your mission:
Create Semester’s unified, accessible, premium design language and implementation system across s

Deliver:
1. Design principles and visual-language direction.
2. Token architecture for color, type, spacing, radius, elevation, motion, breakpoints, density,
3. Component inventory, ownership, API contracts, accessibility behavior, responsive rules, and u
4. Status/provenance system for official, connected, user-created, AI-generated, stale, pending,
5. Content design and microcopy standards for errors, approvals, trust, privacy, AI, academic wor
6. Accessibility specifications including focus, keyboard, contrast, screen reader, touch targets
7. Figma/code parity process, component versioning, visual regression process, and deprecation ru
8. Device-specific patterns for desktop, tablet, mobile, and assistive technologies.
9. Design QA checklist and handoff requirements.
10. First component library implementation plan.

Do not treat design systems as a static component catalog. Make it a governed, tested product inf
```

Brand and Creative Director

```
[Use the shared preamble.]

Act as Brand and Creative Director.

Your mission:
Define a distinctive, premium, trustworthy visual and verbal identity for Semester that can credi

Deliver:
1. Brand platform: mission, personality, values, promise, narrative, voice, naming, taglines.
2. Visual identity direction: logo use, color systems, typography, illustration, photography, ico
3. Brand applications for product, website, sales, support, institutional materials, campaigns, s
4. Accessibility-safe brand rules and contrast requirements.
5. Art direction for student-facing and institutional-facing experiences.
6. Narrative guidelines for AI, privacy, trust, and educational outcomes.
7. Content templates for launch, product updates, incident communications, onboarding, lifecycle,
8. Brand governance and approval system.
9. Competitive visual differentiation analysis.
10. A brand asset production backlog.

Do not make unverified claims or use visual design that obscures warnings, status, provenance, or
```

SaaS Pricing and Unit-Economics Architect

```
[Use the shared preamble.]

Act as SaaS Pricing and Unit-Economics Architect.

Your mission:
Design pricing, packaging, entitlements, usage metering, and economic controls that support Semes

Deliver:
1. SKU and package architecture for individual students, families, departments, institutions, ent
2. Entitlement model linked to tenant policy, feature flags, user roles, usage limits, and billin
3. Value metric analysis for per-student, per-active-user, platform fee, implementation, AI usage
```

```
4. Financial sensitivity analysis for AI, storage, support, integrations, implementation, and pay
5. Discount, trial, freemium, scholarship, institution-sponsored, and enterprise contracting rule
6. Usage-meter schema and invoicing requirements.
7. Revenue leakage, entitlement abuse, and cost-overrun detection.
8. Packaging narrative and sales enablement material.
9. Pricing experiments with ethical and trust constraints.
10. KPIs and dashboards.

Flag accounting, tax, financial-aid, consumer, and marketplace payment questions for qualified ex
```

Institutional EdTech Procurement Strategist

```
[Use the shared preamble.]

Act as Institutional EdTech Procurement Strategist.

Your mission:
Make Semester easy for universities and schools to evaluate, approve, procure, implement, and ren

Deliver:
1. Procurement journey map from first contact to contract, security review, implementation, go-li
2. Buying-committee map: CIO, provost, registrar, student affairs, finance, accessibility, procur
3. Trust-center and diligence-room content list.
4. Security questionnaire, accessibility review, privacy review, AI review, integration review, a
5. RFP response library and differentiation narrative.
6. Contract/SOW requirements and items requiring qualified counsel.
7. Pilot, proof-of-value, evaluation, and acceptance criteria templates.
8. Data migration, interoperability, exit, portability, and change-management arguments.
9. ROI and total-cost-of-ownership model.
10. Procurement risk register and mitigation plan.

Do not make unsupported security, accessibility, compliance, uptime, or replacement claims.
```

Marketplace and Partnership Strategist

```
[Use the shared preamble.]

Act as Marketplace and Partnership Strategist.

Your mission:
Design Semester’s marketplace and partner ecosystem so it creates real student and institution va

Deliver:
1. Marketplace categories, provider types, value proposition, and launch sequencing.
2. Provider onboarding, identity/business verification, approvals, compliance questions, listing
3. Order, entitlement, refund, dispute, payout, support, tax, and fraud workflows.
4. Student/institution controls for provider visibility and procurement.
5. Partnership strategy for LMS, SIS, IdP, payments, campus services, career, learning content, A
6. API/SDK and partner integration requirements.
7. Revenue model, take rate, incentives, conflict management, and partner tiers.
8. Marketplace trust-and-safety policy and escalation procedures.
9. Partner scorecard and offboarding process.
10. Required legal/counsel review items.

Do not launch payments, payouts, claims, or provider access without real operational controls and
```

Customer Success and Institutional Implementation Architect

```
[Use the shared preamble.]

Act as Customer Success and Institutional Implementation Architect.

Your mission:
Create a repeatable system for moving institutions from evaluation to operational success, renewa

Deliver:
1. Implementation methodology: discover, design, configure, integrate, migrate, pilot, parallel r
2. Roles, responsibilities, meeting cadence, artifacts, acceptance criteria, and risk controls by
3. Tenant configuration workbook, integration workbook, migration workbook, training plan, cutove
4. Customer health score, adoption metrics, stakeholder map, success plan, renewal risk model, an
5. Training academy for students, faculty, staff, admins, support, and IT.
6. In-product guidance and enablement requirements.
7. Hypercare support model and handoff to steady-state support.
8. Executive business review template.
9. Customer feedback intake and product escalation loop.
10. Offboarding/export process.

Every implementation must be repeatable, auditable, and not dependent on founder heroics.
```

Revenue Operations and Finance Operations Architect

```
[Use the shared preamble.]

Act as Revenue Operations and Finance Operations Architect.

Your mission:
Create accurate, governed operational processes for leads, accounts, contracts, entitlements, bil

Deliver:
1. Canonical account/contact/opportunity/contract/subscription/entitlement data model.
2. Lifecycle definitions and handoff rules from marketing through sales, implementation, customer
3. CRM and product-data integration requirements.
4. Quote-to-cash process, approval matrix, invoicing, collections, credits, refunds, usage meteri
5. Forecasting methodology and dashboards.
6. Data-quality controls, deduplication, audit logs, and access controls.
7. Revenue leakage and pricing-exception controls.
8. Customer hierarchy model for institutions, campuses, departments, buyers, payers, and subsidia
9. KPI definitions and reporting cadence.
10. Required finance/legal/accounting review points.

Do not create uncontrolled manual spreadsheets as systems of record.
```

Launch Readiness and Risk Management Lead

```
[Use the shared preamble.]

Act as Launch Readiness and Risk Management Lead.

Your mission:
Ensure that no Semester tenant, module, integration, or company launch occurs without objective e

Deliver:
1. Master launch readiness checklist covering product, data, security, privacy, accessibility, re
2. Go/no-go framework with severity-based blockers and decision authority.
3. Risk register with probability, impact, owner, mitigation, contingency, and trigger.
```

```
4. Release readiness scorecard and evidence repository design.
5. Pilot exit criteria and production cutover criteria.
6. Tabletop exercises for incidents, migration rollback, support surge, integration failure, data
7. Customer communication plans for launch, delays, outages, and changes.
8. Dependency tracking and critical-path plan.
9. Post-launch monitoring plan for 24 hours, 7 days, 30 days, and 90 days.
10. Lessons-learned process.

Do not approve a launch based on optimism. Require evidence and document any accepted risk.
```

Security, Privacy, Trust, Accessibility, Reliability, and Incident Response Lead

```
[Use the shared preamble.]

Act as the integrated lead for security, privacy, trust, accessibility, reliability, and incident

Your mission:
Ensure Semester is worthy of being trusted with students' educational lives and institutions' cri

Deliver:
1. Cross-functional control framework mapped to each product domain.
2. Threat models, privacy reviews, accessibility reviews, SLOs, support and incident requirements
3. A combined risk-review template required before release.
4. Accessibility testing program: automated, manual, assistive technology, cognitive accessibilit
5. Reliability program: SLOs, error budgets, dashboards, alerts, capacity, load, chaos, DR, resto
6. Incident response plan, scenario playbooks, communication templates, evidence procedures, and
7. Trust and safety policy framework for community, marketplace, AI, reporting, and moderator acc
8. Data-subject rights, consent, retention, deletion, and legal-hold operational procedures.
9. Procurement-ready evidence artifacts.
10. A prioritized remediation roadmap for current gaps.

Treat every control as operational software and evidence, not a policy document alone.
```

Growth, Lifecycle Marketing, Social Media, Content, and Demand Generation Strategist

```
[Use the shared preamble.]

Act as Growth, Lifecycle Marketing, Social Media Operations, Content, and Demand Generation Strat

Your mission:
Drive adoption, activation, retention, referrals, institutional demand, and brand trust without m

Deliver:
1. Full-funnel growth model for students, families, faculty champions, administrators, and instit
2. Activation milestones for each persona and product domain.
3. Lifecycle communication map: onboarding, activation, engagement, risk, win-back, referral, ren
4. Ethical notification and messaging policy with frequency caps, consent, accessibility, and use
5. Content strategy, editorial calendar, social operations, creator/ambassador program, community
6. Product-led growth loops tied to real value, not spam or dark patterns.
7. Experimentation framework with hypotheses, guardrails, metrics, and stop conditions.
8. Attribution, analytics, privacy, and data-minimization plan.
9. Institution-focused demand programs and account-based marketing.
10. Growth dashboards and operating cadence.

Do not use dark patterns, exploit educational stress, disclose sensitive data, or overstate AI/in
```

Support Operations and Trust-and-Safety Architect

```
[Use the shared preamble.]

Act as Support Operations and Trust-and-Safety Architect.

Your mission:
Build a support and safety operation that resolves problems quickly while protecting student priv

Deliver:
1. Support taxonomy, intake flows, routing, severity matrix, SLAs, staffing model, and escalation
2. Knowledge-base structure and article templates.
3. Support-access policy: consent, just-in-time elevation, scope, expiry, audit, redaction, and p
4. Trust-and-safety reporting, triage, evidence handling, investigation, moderation, appeal, and
5. Academic integrity, harassment, self-harm/safety, fraud, marketplace dispute, impersonation, a
6. Agent tooling requirements and permission model.
7. Quality assurance, customer satisfaction, resolution-time, repeat-contact, and escalation metr
8. Status-page and incident communication procedures.
9. Training/certification requirements for support and moderation staff.
10. Data retention, evidence, and legal/counsel review queue requirements.

Never allow support convenience to override privacy, authorization, or evidence integrity.
```

Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist

```
[Use the shared preamble.]

Act as Board, Governance, Hiring, Organizational Design, and Executive Operations Strategist.

Your mission:
Build the governance, leadership system, hiring architecture, and executive operating cadence req

Deliver:
1. Board composition plan, committee structure, reporting cadence, and board packet templates.
2. Founder/executive decision rights, delegation matrix, and escalation rules.
3. Company OKR system, annual planning process, quarterly business review, and operational metric
4. Organizational design by company stage: pre-seed, seed, Series A, growth.
5. Hiring plan by function, competency rubric, interview loops, leveling, compensation principles
6. Culture principles for trust, ownership, student-centered design, execution quality, and healt
7. Governance for product, AI, security, privacy, accessibility, pricing, risk, and public claims
8. Executive communications, meeting cadence, decision log, and strategic initiative tracking.
9. Business continuity for leadership and key-person risk.
10. A 12-month executive operating calendar.

Do not create bureaucracy for its own sake. Define decision rights, evidence, accountability, and
```

Additional Required Codex Roles

Your original role list is strong, but these additional roles are required to complete Semester correctly.

QA and Quality Engineering Lead

```
[Use the shared preamble.]

Act as QA and Quality Engineering Lead.

Build Semester's complete quality system:
- Test strategy, test pyramid, test data, environments, device/browser matrix.
- Critical journey catalog for every role and domain.
```

```
- Tenant-isolation, authorization, RLS, integration, offline, accessibility, AI, load, chaos, and
- Quality gates for commit, pull request, integration, staging, canary, production, and tenant la
- Automated test harnesses, flaky-test management, test ownership, coverage standards, and releas
- Defect taxonomy, severity rules, triage process, escape analysis, and quality dashboards.
- Production test verification and synthetic monitoring.

Output implementation-ready test plans, repository structure, CI jobs, fixtures, and acceptance c
```

Site Reliability Engineering Lead

```
[Use the shared preamble.]

Act as Site Reliability Engineering Lead.

Build the reliability and operations platform for Semester:
- SLOs, SLIs, error budgets, service catalog, ownership, on-call model.
- Observability: logs, metrics, traces, correlation IDs, dashboards, alerts.
- Capacity planning for registration periods, deadline spikes, grade release, billing, AI usage,
- Deployment, canary, rollback, configuration, secrets, incident, and change-management systems.
- Backup, restore, RTO/RPO, disaster recovery, chaos testing, and regional/dependency failure pla
- Runbooks, alert tests, postmortems, and reliability scorecards.
- Cost observability and resource governance.

Treat every production service, connector, queue, job, database, AI route, and user-critical work
```

Accessibility and Inclusive Design Lead

```
[Use the shared preamble.]

Act as Accessibility and Inclusive Design Lead.

Ensure Semester is accessible and usable by students, staff, guardians, and alumni with diverse d

Deliver:
- WCAG 2.2 AA implementation program and component-level requirements.
- Keyboard, screen reader, touch, zoom, voice control, reduced-motion, contrast, captions/transcr
- Accessibility acceptance criteria for every story and role.
- Automated/manual testing strategy and assistive technology matrix.
- Remediation process, conformance evidence, accessibility statement, and institutional review ar
- Inclusive content standards and plain-language guidance.
- Accessibility incident/ticket process.

Do not treat accessibility as a final QA step; embed it in design, code, tests, and launch gates.
```

DevSecOps and Cloud Infrastructure Lead

```
[Use the shared preamble.]

Act as DevSecOps and Cloud Infrastructure Lead.

Build secure, reproducible, observable cloud infrastructure for Semester:
- Infrastructure-as-code, environment isolation, least privilege, secrets, encryption, network co
- CI/CD security, artifact signing, SBOM, provenance, policy-as-code, configuration drift, vulner
- Tenant-safe deployment patterns, preview environments, staging, production, canary, rollbacks.
- Infrastructure monitoring, cost controls, access reviews, runbooks, and recovery testing.
- Compliance/security evidence generation.
```

```
Never allow manual production-only infrastructure drift without recorded change control.
```

Integration Engineering Lead

```
[Use the shared preamble.]

Act as Integration Engineering Lead.

Build Semester's integration control plane:
- Connector framework, credential vaulting, scopes, OAuth/OIDC/SAML/SCIM/LTI patterns, webhooks,
- SIS, LMS, calendar, email, payment, campus-service, career, and reporting connectors.
- Schema mapping, mapping-version control, source metadata, sync cursors, retries, dead letters,
- Contract tests, sandbox tests, tenant-specific activation, security reviews, and monitoring.
- Migration/parallel-run tools and connector documentation.

Every connector must augment a native Semester capability and be safe to disable without making t
```

Data Migration and Implementation Lead

```
[Use the shared preamble.]

Act as Data Migration and Implementation Lead.

Build the methodology and tooling to move institutions into Semester safely:
- Source inventory, extraction, mapping, cleansing, transformation, validation, rehearsal, cutove
- Migration workbooks for identity, academic records, courses, learning content, enrollments, fin
- Reconciliation reports, data quality thresholds, exception queues, and sign-off procedures.
- Parallel-run plans and institution acceptance criteria.
- Evidence retention and auditability.

Do not treat row-count equality as migration success; validate semantic correctness, relationship
```

Learning Science and Academic Integrity Lead

```
[Use the shared preamble.]

Act as Learning Science and Academic Integrity Lead.

Ensure Semester's native learning, study, assessment, tutoring, AI, and feedback systems improve

Deliver:
- Learning design principles, assessment model, feedback model, study methodology, accommodation
- AI tutoring boundaries, academic-integrity controls, citation/source behavior, instructor contr
- Requirements for assignments, assessments, rubrics, mastery, gradebook, accommodations, and pro
- Evaluation metrics that measure learning support rather than engagement alone.
- Faculty/tutor workflows and policy configuration requirements.
- Risks requiring academic governance and human review.

Do not design AI or study tools that encourage unauthorized work, false confidence, or hidden sur
```

Product Research Lead

```
[Use the shared preamble.]

Act as Product Research Lead.

Build the evidence system for Semester's product decisions:
- Research plan for students, families, faculty, advisors, registrar, finance, student affairs, I
- Interview guides, usability tests, diary studies, surveys, concept tests, and institutional wor
- Repository audit validation: determine which existing capabilities solve real needs and where a
- Research repository, insight taxonomy, confidence levels, and decision linkage.
- Pilot feedback and longitudinal outcome measurement.
- Ethical research consent and data-handling practices.

Do not treat stakeholder opinions or feature requests as validated product truth without evidence
```

Technical Program Manager

```
[Use the shared preamble.]

Act as Technical Program Manager for Semester.

Build and operate the integrated execution plan:
- Master roadmap across product, engineering, security, privacy, accessibility, AI, integrations,
- Dependency graph, critical path, milestone definitions, delivery risks, resourcing, and escalat
- Program rituals, status reporting, decision log, RAID register, release readiness, and executiv
- Cross-functional acceptance criteria and evidence tracking.
- Tenant launch plan coordination and post-launch follow-up.

Do not report activity as progress. Report verified outcomes, blockers, decisions, risks, and evi
```

Data Protection Officer / Privacy Operations Lead

```
[Use the shared preamble.]

Act as Privacy Operations Lead.

You are not qualified human legal counsel. Build privacy operations and identify questions for co

Deliver:
- Data inventory, processing activity register, classification, retention, deletion, export, cons
- Subject-rights request intake, identity verification, fulfillment, exception, and audit workflo
- Vendor/subprocessor review process.
- Privacy-by-design review for new product, analytics, AI, marketplace, integrations, and marketi
- Privacy incident handling coordination with security and counsel.
- User-facing privacy controls and transparency requirements.
- Counsel review queue.

Never provide definitive legal advice. Clearly label counsel-required decisions.
```

Technical Writer and Developer Experience Lead

```
[Use the shared preamble.]

Act as Technical Writer and Developer Experience Lead.

Create the documentation system required to build, operate, integrate, support, and sell Semester
```

```
- Architecture docs, ADRs, API references, event schemas, integration guides, runbooks, developer
- Product help, admin guides, implementation guides, support articles, security/trust documentati
- Documentation quality gates, ownership, versioning, review process, and discoverability.
- Examples and reference applications for institution and partner integrations.
- Release notes and change communications.

Documentation must describe actual behavior, be source-controlled, and be updated as part of ever
```

Final Operating Rule

Do not hand all of these prompts to Codex as disconnected parallel tasks without a shared source of truth. Create a `Semester Program Control Plane` in the repository first:

```
/docs
  /program
    PRODUCT_THESIS.md
    CAPABILITY_TRACEABILITY_MATRIX.md
    MASTER_ROADMAP.md
    ASSUMPTION_REGISTER.md
    RISK_REGISTER.md
    DECISION_LOG.md
    RELEASE_READINESS.md
  /architecture
    SYSTEM_CONTEXT.md
    DOMAIN_MODEL.md
    AUTHORIZATION.md
    DATA_GOVERNANCE.md
    AI_GOVERNANCE.md
    INTEGRATIONS.md
    OFFLINE_SYNC.md
  /operations
    INCIDENT_RESPONSE.md
    SUPPORT_RUNBOOK.md
    SLOS.md
    DISASTER_RECOVERY.md
    IMPLEMENTATION_PLAYBOOK.md
  /trust
    SECURITY.md
    PRIVACY.md
    ACCESSIBILITY.md
    LEGAL_REVIEW_QUEUE.md
    PUBLIC_CLAIMS_REGISTER.md
  /quality
    TEST_STRATEGY.md
    CRITICAL_JOURNEYS.md
    QUALITY_GATES.md
```

Every agent, engineer, designer, executive role, and Codex task must update the traceability matrix, decision log, test plan, and runbook implications as it works. That is how the work becomes a real company and operating platform—not a collection of impressive but disconnected deliverables.

⁂

Tenant-isolation audit: map RLS policies, tenant scoping tables, and verify cross-tenant boundaries against your database schema

Computer
Production deployment pipeline: build an automated CI/CD workflow with security gates, staging promotion, and automated rollback triggers
Computer
Show me the complete database schema for all modules
Give me the CI/CD pipeline configuration file for production
Write the incident response runbook and escalation matrix

I audited the connected **Semester production Supabase project** (`semester`, project ref `lzrqvlugnawcgywkhqlz`) and its repository-level deployment structure. The database is highly ambitious and broad, with RLS enabled on every enumerated public and private table; however, the current security advisory output shows meaningful **production-hardening work remains** before treating tenant isolation and direct database exposure as fully verified. [55] [56] [57]

Most importantly: I could inspect the schema and Supabase's security advisories, but the attempted catalog-level tenant-key/policy query was blocked because this connector classifies raw SQL as a state-changing action—even when the statement is read-only—and requires an explicit action confirmation. Therefore, this is an **evidence-based audit of the available schema catalog and advisories**, not a signed-off penetration test or final RLS certification.

Audit Findings

Current isolation posture

The database contains a large multi-tenant institutional model. The schema includes clear tenant-scoped constructs—especially `schools`, `institution_membership`, `role_grants`, `tenant_feature_policy`, `tenant_module_mode`, `tenant_plan`, `tenant_rollout`, `integration_connections`, governance structures, institution workflows, and school-bound academic, finance, dining, and community domains. The repository also contains dedicated SQL checks for tenancy, RLS coverage, integration RLS, grants, school membership, and role grants, which is the right general architecture direction. [56] [58]

However, "RLS enabled" is not the same as "tenant isolation proven."

Control area	Observed status	Audit conclusion	Required action
RLS enabled	All listed public and private tables show `rls_enabled: true`	Positive baseline	Preserve this as a non-negotiable invariant
Private tables without policies	Supabase reports many `private.*` tables have RLS enabled and no RLS policies	Usually defensible if no browser/API role has grants; this must be validated using grants and exposed-schema checks	Verify `anon`/`authenticated` have no schema usage or table privileges; retain deny-by-default
Public tables without policies	Supabase lists public tables such as invites, LTI state/tickets, payment events, support records, registration internal tables, and beta tables with RLS enabled but no policies	Potentially deliberate service-only tables, but cannot be accepted without grant verification	Revoke browser roles from service-only tables and move sensitive implementation tables to `private`

Control area	Observed status	Audit conclusion	Required action
Anonymous GraphQL visibility	32 public objects are visible to `anon` through GraphQL according to the security advisor	High-priority attack-surface reduction area	Revoke `anon` `SELECT` broadly; expose only intentionally public views/functions
Authenticated GraphQL visibility	287 objects are discoverable to signed-in users according to the advisor	High-priority catalog-disclosure and attack-surface concern—even if RLS filters rows	Revoke broad `authenticated` access; expose allowlisted views/RPCs only
SECURITY DEFINER RPC exposure	205 `SECURITY DEFINER` public functions are executable by authenticated users	Not automatically a vulnerability, but a large privileged attack surface and mandatory review item	Inventory, test, minimize, lock search paths, revoke default grants, route high-risk functions through controlled API/edge functions
Tenant key model	Schema descriptions demonstrate school-scoped/tenant-scoped patterns, but direct column/policy mapping has not been queried	Not yet certified	Run catalog evidence queries and negative cross-tenant test suite
Authorization model	`app_roles`, `app_capabilities`, `role_capabilities`, `role_grants`, and `private.has_capability()` are present	Strong conceptual foundation	Verify every tenant policy derives tenant from trusted membership/grant rather than user-selected input
Operational audit	Several append-only audit, history, ledger, and verification tables exist	Strong governance intent	Verify immutability triggers, privilege boundaries, and query access paths

Supabase specifically reports 61 “RLS enabled, no policy” findings. Some belong to the intentionally server-only `private` schema and may be acceptable if those tables are not exposed to `anon`, `authenticated`, PostgREST, GraphQL, or untrusted Edge Function contexts. But the same advisory also identifies public service-only tables such as `payment_events`, `registration_holds`, `registration_requests`, `support_tickets`, `support_ticket_messages`, LTI tables, and SCIM credential material with no policies. Those should be treated as **must-verify / likely lock-down** surfaces, not ignored linter noise.[57]

Risk prioritization

Priority	Finding	Why it matters	Required remediation
P0	Any proven cross-tenant read/write, policy bypass, or authority escalation	Could expose student records, grades, financial data, family relationships, or institutional operations	Freeze affected endpoint/module, activate kill switch, revoke grants, contain, investigate, notify under approved incident process
P0	Unreviewed SECURITY DEFINER function that can read/write cross-tenant or high-risk records	Privileged function misuse can bypass RLS	Review function code and grants; enforce `auth.uid()`, trusted tenant resolution, capability check, explicit schema-qualified references, fixed `search_path`
P1	Broad `anon` GraphQL object discoverability	Reduces reconnaissance barrier and can expose sensitive data models	Revoke `anon` table access except specifically approved public content
P1	Broad `authenticated` GraphQL object discoverability	Every signed-in user can enumerate sensitive data surfaces and test RPC/table contracts	Replace broad grants with API allowlists, views, RPCs, and narrowly scoped table privileges
P1	Service-only public tables	Public schema creates accidental exposure risk as the product evolves	Relocate to `private` or revoke all client roles and access only via secure server path
P1	Missing tenant-policy proof	A large schema makes undocumented implicit isolation dangerous	Generate policy matrix and enforce CI gate for tenant predicates/authorization functions

Priority	Finding	Why it matters	Required remediation
P2	Tenant-less platform tables	Some tables legitimately operate globally, but global tables can create indirect leakage	Explicitly classify each as global-public, global-internal, or tenant-scoped
P2	Direct client access to complex operational tables	Overly flexible browser database access makes support and audit harder	Prefer domain RPCs or server APIs for high-impact workflows
P2	Missing policy regression artifacts	Schema changes can create isolation regressions	Require dynamic negative tests against every tenant-bound table and RPC

Tenant-Isolation Audit Model

Required trust boundary

```
Browser / mobile client
        |
        | authenticated JWT only
        ▼
Semester API or tightly constrained RPC surface
        |
        ├── Validate authenticated identity
        ├── Resolve active tenant from trusted membership
        ├── Check relationship, role, capability, purpose, consent
        ├── Add correlation ID and audit context
        └── Call domain transaction
        ▼
Postgres domain schema
        |
        ├── RLS policy re-check
        ├── Tenant predicate re-check
        ├── Row ownership/relationship rule
        ├── Domain invariant
        ├── Audit event
        └── Outbox event
        ▼
Private worker / Edge Function / service role
        |
        ├── Explicit tenant context required
        ├── Narrow function-specific grants
        ├── No unrestricted browser invocation
        └── Immutable operation/audit trail
```

Isolation rules

Every data object must fall into exactly one of these classifications:

Classification	Example	Required enforcement
Global public	Marketing plans intended for anonymous display	Read-only public view; no direct table writes; no PII
Global internal	Role vocabulary, platform release evidence, internal pricing operations	No browser table grants; service/operator-only access
Tenant configuration	School policies, SSO policy, module mode, integration configuration	`tenant_id`/`school_id`; live tenant membership; explicit capability
Tenant operational	Registration sections, gradebook, school actions, dining, campus workflows	Tenant key plus domain relationship/role checks

Classification	Example	Required enforcement
Person-private	Notes, tasks, AI memories, profile details, productivity workspace	Account ID plus owner rule; tenant key when institution association exists
Relationship-scoped	Family, mentor, advisor, support shares	Source person, recipient person, tenant, active grant, scope, expiry
Course/section-scoped	Grade entries, assignments, course AI rules, roster artifacts	Tenant, course/section, term, enrollment/staff assignment
Financial/record-sensitive	Academic records, bills, payment plans, ledger entries	Tenant, subject, role/capability, append-only mutation path
Service-only	Webhook receipts, rate limits, raw sync state, credential verification material	`private` schema; no client grants; service role or narrowly privileged worker only
Audit/ledger	Audit events, history, immutable decision records	Insert through trusted function only; read through tenant-scoped audited views

Required tenant policy pattern

Do not use a user-supplied tenant ID as the authority. Resolve tenant access from durable membership/grants.

```
create or replace function private.current_tenant_member(
  expected_tenant uuid
)
returns boolean
language sql
stable
security definer
set search_path = pg_catalog, public, private
as $$
  select exists (
    select 1
    from public.institution_membership im
    where im.account_id = auth.uid()
      and im.school_id = expected_tenant
      and im.status = 'active'
      and im.revoked_at is null
  );
$$;

create policy "tenant members can read tenant rows"
on public.example_tenant_table
for select
to authenticated
using (
  private.current_tenant_member(school_id)
  and private.has_capability(
    auth.uid(),
    school_id,
    'example:read'
  )
);
```

For person-private data, the tenant predicate is **not sufficient**:

```
create policy "owner reads own private workspace"
on public.productivity_workspace
for select
```

```
to authenticated
using (
  account_id = auth.uid()
  and private.current_tenant_member(school_id)
);
```

For relationship-scoped data:

```
create policy "active recipient reads granted share only"
on public.family_shared_items
for select
to authenticated
using (
  exists (
    select 1
    from public.family_grants fg
    where fg.student_account_id = family_shared_items.student_account_id
      and fg.supporter_account_id = auth.uid()
      and fg.school_id = family_shared_items.school_id
      and fg.status = 'active'
      and fg.expires_at > now()
      and private.scope_permits(
        fg.id,
        family_shared_items.category,
        family_shared_items.resource_id
      )
  )
);
```

Policy review checklist

For every table, view, materialized view, RPC, Edge Function, storage bucket, queue, cache, and analytics dataset, answer:

Question	Required answer
Does this object contain tenant data?	Yes/no with a documented classification
What field establishes tenant ownership?	`school_id`, `tenant_id`, derived through parent relation, or explicitly global
Can it be called/read by `anon`?	Default: no
Can it be called/read by `authenticated`?	Only with documented, tested policy
Does every RLS policy contain a tenant boundary or trusted ownership boundary?	Yes, unless explicitly global
Are writes restricted to safe domain functions?	Yes for records, grades, billing, approvals, and audit data
Are service-role pathways separated from browser pathways?	Yes
Is a SECURITY DEFINER function involved?	If yes: fixed search path, explicit authorization, code review, negative tests
Is there a cross-tenant negative test?	Required
Is there a tenant-deletion/offboarding behavior?	Required
Is the object exposed in GraphQL/PostgREST?	Only intentionally
Does it log sensitive access or mutation?	Required for high-risk domains

Concrete audit query package

Run these in a **non-production branch or staging environment first**, then use the same query package in production with read-only credentials. Do not run schema-changing remediation without change review.

1. RLS coverage and tenant-key inventory

```
with relations as (
  select
    n.nspname as schema_name,
    c.relname as relation_name,
    c.relkind,
    c.relrowsecurity as rls_enabled,
    c.relforcerowsecurity as force_rls,
    exists (
      select 1
      from pg_policies p
      where p.schemaname = n.nspname
        and p.tablename = c.relname
    ) as has_rls_policy,
    (
      select array_agg(col.column_name order by col.column_name)
      from information_schema.columns col
      where col.table_schema = n.nspname
        and col.table_name = c.relname
        and col.column_name in (
          'tenant_id',
          'school_id',
          'organization_id',
          'institution_id'
        )
    ) as tenant_columns
  from pg_class c
  join pg_namespace n on n.oid = c.relnamespace
  where c.relkind in ('r', 'v', 'm')
    and n.nspname in ('public', 'private')
)
select
  schema_name,
  relation_name,
  relkind,
  rls_enabled,
  force_rls,
  has_rls_policy,
  coalesce(tenant_columns::text, '{}') as tenant_columns,
  case
    when schema_name = 'private' then 'server-only: verify grants'
    when tenant_columns is not null then 'tenant-key present'
    else 'no direct tenant key: classify as global/derived/person-private'
  end as review_status
from relations
order by schema_name, relation_name;
```

2. Full RLS policy map

```
select
  schemaname,
  tablename,
  policyname,
  permissive,
```

```
  roles,
  cmd,
  qual as using_expression,
  with_check as with_check_expression
from pg_policies
where schemaname in ('public', 'private')
order by schemaname, tablename, cmd, policyname;
```

3. Tables exposed to browser roles

```
select
  table_schema,
  table_name,
  grantee,
  privilege_type
from information_schema.role_table_grants
where table_schema in ('public', 'private')
  and grantee in ('anon', 'authenticated')
order by table_schema, table_name, grantee, privilege_type;
```

4. SECURITY DEFINER inventory

```
select
  n.nspname as schema_name,
  p.proname as function_name,
  pg_get_function_identity_arguments(p.oid) as arguments,
  p.prosecdef as security_definer,
  p.proconfig as function_config,
  has_function_privilege('anon', p.oid, 'execute') as anon_can_execute,
  has_function_privilege('authenticated', p.oid, 'execute') as authenticated_can_execute,
  pg_get_functiondef(p.oid) as definition
from pg_proc p
join pg_namespace n on n.oid = p.pronamespace
where n.nspname in ('public', 'private')
  and p.prosecdef
order by schema_name, function_name;
```

5. Unsafe default function grants

```
select
  n.nspname as schema_name,
  p.proname as function_name,
  has_function_privilege('public', p.oid, 'execute') as public_execute,
  has_function_privilege('anon', p.oid, 'execute') as anon_execute,
  has_function_privilege('authenticated', p.oid, 'execute') as authenticated_execute
from pg_proc p
join pg_namespace n on n.oid = p.pronamespace
where n.nspname in ('public', 'private')
order by schema_name, function_name;
```

6. Missing RLS policies outside private schema

```
select
  n.nspname as schema_name,
  c.relname as table_name
from pg_class c
```

```
join pg_namespace n on n.oid = c.relnamespace
where c.relkind = 'r'
  and n.nspname = 'public'
  and c.relrowsecurity
  and not exists (
    select 1
    from pg_policies p
    where p.schemaname = n.nspname
      and p.tablename = c.relname
  )
order by table_name;
```

7. Cross-tenant policy test harness pattern

```
begin;

-- Substitute fixtures created only in an isolated test database/branch.
select set_config(
  'request.jwt.claim.sub',
  '11111111-1111-1111-1111-111111111111',
  true
);

select set_config(
  'request.jwt.claim.role',
  'authenticated',
  true
);

-- The test actor belongs only to tenant A.
-- This count must equal zero for every tenant-bound table.
select count(*) as forbidden_rows_visible
from public.registration_enrollments
where school_id = '22222222-2222-2222-2222-222222222222';

rollback;
```

Automate this pattern for every RLS-protected table and for every relevant RPC. Test:

- Tenant A user cannot read Tenant B row.
- Tenant A user cannot infer Tenant B row existence through counts, errors, response times, or unique constraints.
- Tenant A user cannot insert a row with Tenant B `school_id`.
- Tenant A user cannot update/delete Tenant B row.
- Tenant A administrator cannot access Tenant B data.
- Staff relationship boundaries work inside the same tenant.
- Family/guardian access cannot exceed explicit scope, expiry, or resource set.
- Revoked memberships instantly lose access.
- Offboarded tenant data is inaccessible.
- SECURITY DEFINER functions reject forged tenant arguments.
- Service-only tables cannot be queried through REST or GraphQL.

Complete Database Schema Map

The actual deployed schema is extensive—well over 250 tables across `public` and `private`. The following is a **complete module-level schema catalog** based on the live table inventory. It groups the deployed tables into architectural modules rather than dumping hundreds of raw DDL definitions. Generate the TypeScript database types and `pg_dump --schema-only` artifact in CI as the canonical full-column schema record.[56]

Identity, access, tenancy, and policy

Tables
profiles, schools, institution_membership, school_membership_requests, role_grants, app_roles, app_capabilities, role_capabilities, role_grant_audit_event, institution_identity_provider, tenant_sso_policy, tenant_sso_policy_history, app_admins, access_gate, invites, feature_kill_switch, feature_cohort_members, tenant_feature_policy, tenant_policy_audit_event, tenant_plan, tenant_plan_history, tenant_rollout, tenant_rollout_evidence, tenant_rollout_history, tenant_module_mode, tenant_module_mode_history, module_mode_request, module_mode_approval, break_glass_grant, audit_event, data_classification_rules, legal_holds, human_overrides

Identity provisioning and SSO/SCIM

Tables
institution_identity_provider, institution_membership, scim_credential, scim_external_identity, scim_group_mapping, provisioning_audit_event, lti_platform, lti_nonce, lti_identity, lti_link_ticket, lti_line_item

Student productivity and personal OS

Tables
state, usage, notes, tasks, appointments, sittings, calendar_feeds, blocks, push_devices, push_queue, productivity_workspace, student_context, term_plan_courses, weekly_checkins, onboarding_progress, operator_preference, ai_memories, access_log

Academic planning, registrar, records, and degree path

Tables
courses, enrollments, catalog_sections, registration_terms, registration_sections, registration_enrollments, registration_holds, registration_completions, registration_overrides, registration_requests, registration_time_tickets, registration_windows, registration_audit_event, seat_watches, graduation_scenarios, articulation_rules, transfer_evaluations, academic_record_subjects, academic_record_changes, academic_record_entries, student_account_settings

Learning, gradebook, study, and evidence

Tables
approved_source, evidence_reference, concept_evidence, mistake_evidence, skill_claim, skill_claim_evidence, skill_records, capture_asset, capture_segment, capture_artifact, course_ai_rules, course_guidance, study_packs, course_reviews, course_review_authors, study_match_optins, gradebook_operations, gradebook_schemes, gradebook_items, grade_entries, regrade_requests, regrade_resolutions, grade_passbacks, private.approved_source_content

AI governance and gateway

Tables
ai_policy, consent_record, tenant_feature_policy, data_classification_rules, human_overrides, private.ai_usage_month, private.ai_usage_reservation, private.gateway_review, private.gateway_audit, private.gateway_intelligence_audit, private.gateway_intelligence_action, private.gateway_rate_limit, private.gateway_health_probe

Family, guardian, advising, and support sharing

Tables
family_grants, family_invites, family_shared_items, family_access_events, guardian_links, guardian_link_restrictions, guardian_link_history, advisor_shares, advisor_share_events, support_access_grant, support_access_event, support_shares, support_share_events, accommodation_passports, accommodation_shares, accommodation_access_events

Campus life, dining, community, and organizations

Tables
organizations, organization_members, connections, groups, group_members, group_tasks, messages, message_reactions, reports, communities, community_members, community_posts, community_restrictions, community_mutes, community_cases, community_reports, community_case_events, community_detector_rules, community_signals, community_decisions, community_venues, community_sessions, community_session_participants, community_retention_runs, community_programs, community_aliases, community_identity_grants, community_volunteers, community_calibration_items, community_volunteer_tasks, community_volunteer_votes, community_volunteer_events, community_escalation_policies, community_escalations, community_escalation_deliveries, community_escalation_agreement_events, community_safety_entries, community_media, community_media_blocklist, community_media_deletions

Dining and campus card operations

Tables
dining_partner_connections, dining_locations, dining_hours, dining_menu_items, dining_plans, dining_orders, dining_order_events, dining_ledger, dining_pool_donations, dining_pool_claims

Career, talent, alumni, and mentoring

Tables
contact_channels, talent_profiles, talent_profile_views, opportunities, peer_mentor_assignments, peer_mentor_offers, mentor_requests, alumni_mentor_offers, course_demand_snapshots, outcome_aggregates, demand_consents

Institution actions, support, and operational help

Tables
institution_actions, institution_action_offices, institution_action_audiences, institution_action_progress, help_destinations, help_requests, help_request_events, support_tickets, support_ticket_messages, support_notification_outbox, feedback, forms, form_responses, form_publications

Integrations, source provenance, migration, and data quality

Tables
integration_connections, integration_scopes, integration_mappings, integration_mapping_versions, integration_sync_runs, integration_sync_errors, integration_webhook_events, integration_dead_letter_events, integration_reconciliation_runs, integration_reconciliation_discrepancies, integration_schema_fingerprints, integration_schema_drift_events, integration_duplicate_candidates, integration_duplicate_resolutions, integration_source_owners, integration_retention_runs, provider_registry, provider_evidence, source_records, source_snapshots, source_freshness_events, canonical_entity_references, migration_projects, migration_field_maps, migration_runs, migration_approvals, private.integration_simulation_runs, private.roster_import_config, private.roster_import_batch, private.roster_staged_row, private.roster_current, private.roster_snapshot

Governance, compliance, trust, and procurement

Tables
governance_policy_nodes, governance_steward_assignments, governance_decisions, governance_config_requests, governance_incident_notices, school_config_versions, workflow_versions, compliance_frameworks, compliance_controls, control_evidence, claims_register, trust_artifacts, trust_artifact_versions, trust_room_requests, trust_room_grants, trust_room_grant_items, trust_room_access_log, platform_release_evidence, council_seat_holder, console_duty, approval_request, approval_decision, console_action_record, private.console_audit_event, private.console_audit_key, private.console_audit_manifest, private.console_audit_verification

Billing, commerce, customer success, and SaaS operations

Tables
commercial_products, commercial_plans, commercial_prices, entitlement_definitions, plan_entitlements, billing_accounts, billing_account_tenants, quotes, quote_lines, contracts, subscriptions, subscription_entitlements, invoices, invoice_lines, payment_events, credits_refunds, dunning_cases, dunning_actions, cancellation_requests, checkout_sessions, customer, customer_commitment, customer_contract, implementation_projects, implementation_milestones, success_plans, qbrs, renewal_opportunities, account_health_snapshots

Growth, pilots, beta, marketing, and commercial governance

Tables
gtm_prospects, gtm_consent, gtm_suppression, gtm_campaigns, gtm_campaign_links, gtm_campaign_reviews, gtm_communication_events, gtm_conversion_events, gtm_report_access, gtm_sponsor_policy, gtm_sponsor_placements, gtm_accounts, gtm_stakeholders, gtm_decision_log, gtm_pilots, gtm_pilot_metrics, gtm_pilot_outcomes, beta_programs, beta_cohorts, beta_invitations, beta_memberships, beta_feature_flags, beta_known_issues, beta_feedback, beta_exit_requests, referral_codes, referrals, site_leads, private.site_lead_hits

Lifecycle, offboarding, retention, events, and internal platform mechanics

Tables
data_requests, data_subject_request, school_offboarding, school_offboarding_undo, private.account_ages, private.direct_rate_limit, private.domain_outbox_events, private.domain_event_receipts, private.ledger_chain_start, private.ledger_chain, private.ledger_chain_key, private.ledger_chain_manifest, private.ledger_chain_verification

Schema documentation deliverables

Add these generated artifacts to the build pipeline:

```
supabase gen types typescript \
  --project-id "$SUPABASE_PROJECT_REF" \
  --schema public,private \
  > app/src/lib/database.generated.ts
```

```
pg_dump "$DATABASE_URL" \
  --schema-only \
  --no-owner \
  --no-privileges \
  --schema=public \
  --schema=private \
  > artifacts/schema.sql
```

Do not publish raw schema artifacts containing internal function definitions or identifiers to public documentation. Keep them in a private engineering evidence store.

Production CI/CD Pipeline

The repository currently has Vercel deployment configuration and substantial test/security/operations materials. The production pipeline should turn those assets into **enforced gates**, with staging promotion and release rollback rules—not merely a deployment on merge. [58] [56]

Deployment flow

```
Feature branch
   │
   ├── Format · lint · typecheck · unit test
   ├── Secret scan · dependency scan · SAST
   ├── Build · bundle/performance budgets
   ├── RLS / migration / policy tests
   ├── Accessibility checks
   └── Preview deployment
        │
        ▼
Pull request approved
        │
        ├── Contract / integration tests
        ├── E2E tests on preview
        ├── Tenant-isolation negative suite
        ├── Migration dry run on Supabase branch
        └── Release evidence check
             │
             ▼
main branch
   │
   ├── Build immutable release artifact
   ├── Deploy to staging
   ├── Apply reviewed staging migrations
   ├── Smoke + E2E + DAST + SLO synthetic tests
   └── Manual production approval
        │
        ▼
Production canary
   │
   ├── 5% traffic / internal + designated tenant cohort
```

```
   ├── Observe errors, latency, auth, isolation, sync and business KPIs
   ├── Automatic abort thresholds
   └── Manual promote gate
        │
        ▼
Production full rollout
   │
   ├── Post-deploy migration verification
   ├── Feature flag staged rollout
   ├── Release record + audit event
   └── Monitor / rollback readiness
```

Required GitHub Actions file

Create `.github/workflows/production.yml` with the following content. This is a production template; replace command names and secret names only after matching them to your actual scripts and secret manager. It intentionally never prints secret values.

```yaml
name: Production Release

on:
  push:
    branches: [main]
  workflow_dispatch:
    inputs:
      release_ref:
        description: "Immutable commit SHA or tag to deploy"
        required: false
        type: string
      force_canary:
        description: "Deploy through canary even if no application files changed"
        required: false
        default: false
        type: boolean

permissions:
  contents: read
  id-token: write
  pull-requests: read
  security-events: write
  checks: write

concurrency:
  group: semester-production-release
  cancel-in-progress: false

env:
  NODE_VERSION: "22"
  APP_DIR: app
  SUPABASE_PROJECT_REF: ${{ secrets.SUPABASE_PRODUCTION_PROJECT_REF }}
  VERCEL_ORG_ID: ${{ secrets.VERCEL_ORG_ID }}
  VERCEL_PROJECT_ID: ${{ secrets.VERCEL_PROJECT_ID }}

jobs:
  metadata:
    name: Release metadata
    runs-on: ubuntu-latest
    outputs:
      sha: ${{ steps.release.outputs.sha }}
      version: ${{ steps.release.outputs.version }}
    steps:
```

```
      - uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - id: release
        shell: bash
        run: |
          set -euo pipefail
          SHA="${{ inputs.release_ref || github.sha }}"
          git cat-file -e "${SHA}^{commit}"
          echo "sha=${SHA}" >> "$GITHUB_OUTPUT"
          echo "version=$(git describe --always --dirty=false "${SHA}")" >> "$GITHUB_OUTPUT"

  validate:
    name: Quality and security gates
    needs: metadata
    runs-on: ubuntu-latest
    timeout-minutes: 30
    defaults:
      run:
        working-directory: app
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Install locked dependencies
        run: npm ci

      - name: Verify lockfile integrity
        run: npm audit signatures || true

      - name: Typecheck
        run: npm run typecheck

      - name: Lint
        run: npm run lint

      - name: Unit and component tests
        run: npm test -- --run

      - name: Production build
        run: npm run build

      - name: Performance budgets
        run: npm run perf:check

      - name: Secret scan
        working-directory: .
        run: |
          docker run --rm \
            -v "$PWD:/repo" \
            zricethezav/gitleaks:latest \
            detect --source=/repo --no-git --redact

      - name: Dependency vulnerability scan
        run: npm audit --omit=dev --audit-level=high
```

```
      - name: Upload production artifact
        uses: actions/upload-artifact@v4
        with:
          name: semester-web-${{ needs.metadata.outputs.version }}
          path: app/dist
          if-no-files-found: error
          retention-days: 14

  database-policy:
    name: Database migration and tenant-isolation gates
    needs: metadata
    runs-on: ubuntu-latest
    timeout-minutes: 35
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - name: Install Supabase CLI
        uses: supabase/setup-cli@v1
        with:
          version: latest

      - name: Link staging project
        run: |
          supabase link \
            --project-ref "${{ secrets.SUPABASE_STAGING_PROJECT_REF }}" \
            --password "${{ secrets.SUPABASE_STAGING_DB_PASSWORD }}"

      - name: Migration integrity
        run: supabase db diff --linked --schema public,private

      - name: Apply migrations to staging only
        run: supabase db push --linked --include-all

      - name: Run repository database checks
        run: |
          chmod +x supabase/check.sh
          ./supabase/check.sh

      - name: Run tenant-isolation regression suite
        run: |
          chmod +x ops/test-tenant-isolation.sh
          ./ops/test-tenant-isolation.sh \
            --project-ref "${{ secrets.SUPABASE_STAGING_PROJECT_REF }}"

      - name: Run RLS and grants audit
        run: |
          psql "${{ secrets.SUPABASE_STAGING_DATABASE_URL }}" \
            -v ON_ERROR_STOP=1 \
            -f ops/sql/tenant-isolation-audit.sql

      - name: Upload database evidence
        uses: actions/upload-artifact@v4
        with:
          name: database-evidence-${{ needs.metadata.outputs.version }}
          path: |
            artifacts/tenant-isolation-audit.json
            artifacts/rls-policy-map.csv
            artifacts/schema.sql
          if-no-files-found: error
          retention-days: 90

  deploy-staging:
```

```
    name: Deploy and verify staging
    needs: [metadata, validate, database-policy]
    runs-on: ubuntu-latest
    environment:
      name: staging
    outputs:
      deployment_url: ${{ steps.deploy.outputs.url }}
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Install Vercel CLI
        run: npm install --global vercel@latest

      - name: Pull staging configuration
        working-directory: app
        run: vercel pull --yes --environment=preview --token="${{ secrets.VERCEL_TOKEN }}"

      - name: Build deployable artifact
        working-directory: app
        run: vercel build --token="${{ secrets.VERCEL_TOKEN }}"

      - id: deploy
        name: Deploy staging artifact
        working-directory: app
        run: |
          URL="$(vercel deploy --prebuilt --yes --token="${{ secrets.VERCEL_TOKEN }}")"
          echo "url=${URL}" >> "$GITHUB_OUTPUT"

      - name: Staging smoke tests
        run: |
          npx wait-on "${{ steps.deploy.outputs.url }}"
          ./ops/smoke-test.sh "${{ steps.deploy.outputs.url }}"

      - name: Staging end-to-end tests
        run: |
          BASE_URL="${{ steps.deploy.outputs.url }}" \
          npm --prefix app run test:e2e

      - name: Staging accessibility tests
        run: |
          BASE_URL="${{ steps.deploy.outputs.url }}" \
          npm --prefix app run test:a11y

      - name: Staging security tests
        run: |
          BASE_URL="${{ steps.deploy.outputs.url }}" \
          ./ops/dast-baseline.sh

  production-approval:
    name: Production approval
    needs: [metadata, deploy-staging]
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - name: Confirm staged release passed
```

```
        run: |
          test -n "${{ needs.deploy-staging.outputs.deployment_url }}"
          echo "Staging acceptance complete."

  deploy-production:
    name: Deploy production canary
    needs: [metadata, validate, database-policy, deploy-staging, production-approval]
    runs-on: ubuntu-latest
    environment:
      name: production
    outputs:
      production_url: ${{ steps.deploy.outputs.url }}
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - uses: actions/setup-node@v4
        with:
          node-version: ${{ env.NODE_VERSION }}
          cache: npm
          cache-dependency-path: app/package-lock.json

      - name: Install Vercel CLI
        run: npm install --global vercel@latest

      - name: Apply production migrations
        run: |
          supabase link \
            --project-ref "${{ secrets.SUPABASE_PRODUCTION_PROJECT_REF }}" \
            --password "${{ secrets.SUPABASE_PRODUCTION_DB_PASSWORD }}"
          supabase db push --linked --include-all

      - name: Pull production configuration
        working-directory: app
        run: vercel pull --yes --environment=production --token="${{ secrets.VERCEL_TOKEN }}"

      - name: Build immutable production artifact
        working-directory: app
        run: vercel build --prod --token="${{ secrets.VERCEL_TOKEN }}"

      - id: deploy
        name: Deploy production artifact
        working-directory: app
        run: |
          URL="$(vercel deploy --prebuilt --prod --yes --token="${{ secrets.VERCEL_TOKEN }}")"
          echo "url=${URL}" >> "$GITHUB_OUTPUT"

      - name: Record release metadata
        run: |
          ./ops/record-release.sh \
            --sha "${{ needs.metadata.outputs.sha }}" \
            --version "${{ needs.metadata.outputs.version }}" \
            --url "${{ steps.deploy.outputs.url }}"

  canary-observation:
    name: Canary health gate
    needs: [metadata, deploy-production]
    runs-on: ubuntu-latest
    timeout-minutes: 25
    environment:
      name: production
    steps:
      - uses: actions/checkout@v4
```

```
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - name: Verify production smoke paths
        run: |
          ./ops/smoke-test.sh "${{ needs.deploy-production.outputs.production_url }}"

      - name: Evaluate automated rollback thresholds
        env:
          RELEASE_SHA: ${{ needs.metadata.outputs.sha }}
          PRODUCTION_URL: ${{ needs.deploy-production.outputs.production_url }}
          OBSERVABILITY_TOKEN: ${{ secrets.OBSERVABILITY_TOKEN }}
        run: |
          ./ops/evaluate-canary.sh \
            --release "$RELEASE_SHA" \
            --url "$PRODUCTION_URL" \
            --window-minutes 15 \
            --max-error-rate 0.01 \
            --max-auth-failure-rate 0.02 \
            --max-p95-ms 2500 \
            --max-critical-journey-failures 0 \
            --max-security-policy-violations 0

  promote-production:
    name: Promote release
    needs: [metadata, deploy-production, canary-observation]
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - uses: actions/checkout@v4
        with:
          ref: ${{ needs.metadata.outputs.sha }}

      - name: Promote deployment
        run: |
          ./ops/promote-release.sh \
            --sha "${{ needs.metadata.outputs.sha }}" \
            --url "${{ needs.deploy-production.outputs.production_url }}"

      - name: Post-deployment validation
        run: |
          ./ops/smoke-test.sh "${{ needs.deploy-production.outputs.production_url }}"
          ./ops/verify-release-evidence.sh \
            --sha "${{ needs.metadata.outputs.sha }}"

  rollback-on-failure:
    name: Automatic rollback
    if: ${{ failure() && needs.deploy-production.result == 'success' }}
    needs: [metadata, deploy-production, canary-observation, promote-production]
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - uses: actions/checkout@v4

      - name: Activate safety kill switches
        env:
          SUPABASE_SERVICE_ROLE_KEY: ${{ secrets.SUPABASE_SERVICE_ROLE_KEY }}
        run: |
          ./ops/engage-release-kill-switches.sh \
            --release "${{ needs.metadata.outputs.sha }}"

      - name: Roll back to last known good deployment
```

```
        env:
          VERCEL_TOKEN: ${{ secrets.VERCEL_TOKEN }}
        run: |
          ./ops/rollback-last-known-good.sh

      - name: Create release incident record
        run: |
          ./ops/open-release-incident.sh \
            --severity SEV-1 \
            --release "${{ needs.metadata.outputs.sha }}" \
            --reason "Automated production gate failure or canary regression"
```

Pipeline configuration requirements

The workflow requires these protected secrets and configuration values:

Name	Purpose	Handling
`VERCEL_TOKEN`	Deployment authorization	Production environment secret; rotate; least privilege
`VERCEL_ORG_ID`	Vercel organization selection	Repository/environment secret
`VERCEL_PROJECT_ID`	Vercel project selection	Repository/environment secret
`SUPABASE_PRODUCTION_PROJECT_REF`	Production target reference	Production environment secret
`SUPABASE_STAGING_PROJECT_REF`	Staging target reference	Staging environment secret
`SUPABASE_PRODUCTION_DB_PASSWORD`	Migration tool connection	Production environment secret; never expose to application runtime
`SUPABASE_STAGING_DB_PASSWORD`	Staging migration connection	Staging environment secret
`SUPABASE_PRODUCTION_DATABASE_URL`	Read-only evidence/audit scripts where needed	Production environment secret; prefer a dedicated least-privilege auditor credential
`OBSERVABILITY_TOKEN`	Canary metrics query	Production environment secret
`SUPABASE_SERVICE_ROLE_KEY`	Only for narrowly scoped operational scripts	Avoid if possible; never client-side; rotate and log use

Automatic rollback triggers

Do not roll back schema migrations blindly. Application release rollback and database rollback are different operations.

Trigger	Automated action	Human action
Build/type/security gate fails	Do not deploy	Fix and reopen release
Migration dry run or RLS suite fails	Do not promote	Investigate schema/policy before retry
Canary 5xx rate above 1% for 5 minutes	Stop promotion; roll back application; engage related feature flags	Incident commander validates effect
Authentication failure rate above 2%	Stop promotion; roll back application	Identity owner investigates configuration/session impact
Critical student journey fails	Stop promotion; roll back application or disable feature	Product/engineering decide customer communication
Cross-tenant test fails	Stop release immediately; disable affected domain/API	Treat as potential security incident

Trigger	Automated action	Human action
Unauthorized data access signal	Activate kill switch; revoke exposure; roll back affected service	Security incident workflow
Migration causes data integrity failure	Stop release; prevent further writes if necessary; invoke recovery plan	DBA/data owner performs approved corrective migration
Connector reconciliation discrepancy exceeds threshold	Disable auto-write direction; preserve read-only data with freshness notice	Integration owner resolves mapping/source issue
AI policy violation	Disable AI feature/model/tool path using flag	AI governance and security investigate

Database migration safety rules

1. Every migration must be additive first: add columns/tables/indexes, backfill, dual-read/write, validate, then later remove legacy fields.
2. Never combine irreversible data destruction with the same deployment that introduces a feature.
3. Every migration must have:
    - Purpose and ADR link.
    - Forward migration.
    - Data validation query.
    - Application compatibility window.
    - Roll-forward remediation plan.
    - Backup/restore impact.
    - Tenant-isolation impact statement.
4. Use a production approval gate for changes to RLS policies, roles, grants, functions, triggers, audit tables, records, grades, finance, retention, or tenant boundaries.
5. The release pipeline must fail if generated schema diffs include:
    - `DISABLE ROW LEVEL SECURITY`.
    - `GRANT` to `anon` or `authenticated` without allowlist.
    - New `SECURITY DEFINER` function without review metadata.
    - Dropped policy, tenant key, index, audit trigger, or constraint.
    - Direct table writes to append-only ledger tables.
    - Destructive DDL without an approved change record.

Incident Response Runbook

Scope

This runbook covers security incidents, availability incidents, data-integrity incidents, integration failures, deployment regressions, AI safety events, privacy events, and institutional workflow failures.

It must be validated through tabletop exercises and production-safe drills. It is an operational plan, not legal advice; qualified legal counsel must determine applicable notification duties, regulatory conclusions, and external legal communications.

Incident lifecycle

```
Detect
  │
  ▼
Triage and classify
  │
  ▼
Declare severity and assign incident commander
  │
  ├── Contain
  ├── Preserve evidence
  ├── Protect users and data
  └── Communicate internally
  │
  ▼
Investigate and remediate
  │
  ├── Roll back / kill switch / revoke access
  ├── Restore service / integrity
  ├── Assess data and customer impact
  └── Coordinate legal/privacy review
  │
  ▼
Recover and verify
  │
  ▼
Customer communications
  │
  ▼
Post-incident review and corrective actions
```

Severity criteria

Severity	Definition	Examples	Initial response
SEV-0	Confirmed or strongly suspected severe security/privacy/integrity event	Cross-tenant disclosure, credential compromise, unauthorized record change, confirmed major breach	Immediate incident declaration
SEV-1	Critical service outage or high-impact functional failure	Login outage, assignment submission failure, registration failure, production deployment regression	Page on-call immediately
SEV-2	Major feature degradation with workaround or limited tenant impact	One tenant's connector fails, faculty grade workflow degraded, major performance incident	Triage within 30 minutes
SEV-3	Limited defect or non-critical operational issue	Isolated UI issue, non-critical report delay, documented workaround exists	Triage same business day
SEV-4	Low-impact issue or enhancement	Cosmetic issue, minor copy defect, non-urgent request	Normal backlog triage

First 15 minutes

Minute	Incident commander actions	Technical lead actions	Security/privacy actions	Communications actions
0–5	Declare incident channel, assign incident ID, name roles	Confirm symptoms and scope	Decide whether security/privacy indicators exist	Hold external communications pending facts
5–10	Set severity; open timeline	Stop harmful deployment, disable flag, preserve logs	Revoke suspect credentials/tokens if needed	Draft initial internal status

Minute	Incident commander actions	Technical lead actions	Security/privacy actions	Communications actions
10–15	Establish next update time	Identify affected tenants, data, and services	Begin evidence preservation; contact legal/privacy lead for SEV-0	Post status update for confirmed user impact

Standard response roles

Role	Responsibilities	Backup
Incident commander	Owns command, severity, decisions, timeline, cadence, closure	COO or designated operations lead
Technical lead	Diagnoses architecture/code/database/infrastructure; executes technical mitigation	CTO or senior platform engineer
Security lead	Leads containment, evidence, credential/access actions, investigation	CISO delegate
Privacy/legal liaison	Coordinates impact assessment, counsel, notification review, documentation	Privacy lead / outside counsel contact
Customer communications lead	Status page, customer updates, support scripts, stakeholder communications	CSM/support lead
Scribe	Maintains time-stamped action log and decisions	Product operations / support operations
Tenant liaison	Coordinates with affected institution(s)	Customer success lead
Data integrity owner	Validates records, grades, billing, ledgers, migration state	Data/platform lead

Immediate containment playbooks

Cross-tenant data exposure

1. Declare SEV-0.
2. Immediately disable affected API/RPC/feature flag and revoke related `anon`/`authenticated` grants if needed.
3. Preserve logs, request IDs, database audit events, query traces, deployment metadata, and authorization context.
4. Do not delete evidence or mutate affected data except through documented containment commands.
5. Determine:
   - Affected tenant(s).
   - Affected data categories.
   - Number of subjects/records.
   - Exact exposure period.
   - Whether data was viewed, exported, modified, or only potentially reachable.
6. Rotate related credentials, revoke sessions, and disable compromised integrations as appropriate.
7. Engage security lead, privacy/legal liaison, executive sponsor, and qualified human counsel.
8. Implement a least-privilege corrective change, then run cross-tenant regression suite before re-enabling.
9. Prepare customer/regulatory notifications only after facts and counsel-approved obligations are determined.
10. Complete root cause analysis and control improvements within the defined corrective-action SLA.

Production deployment regression

1. Declare SEV-1 if critical journey affected.
2. Halt rollout and stop promotion.
3. Engage feature kill switch for the affected module before broad platform rollback if it contains the blast radius.
4. Roll back to last-known-good application deployment.
5. Do **not** auto-rollback a database migration unless an approved migration recovery plan exists.
6. Verify:
    - Login.
    - Student Today.
    - Course/assignment access.
    - Submission confirmation.
    - Grade view.
    - Registration and billing if affected.
    - Tenant isolation smoke checks.
7. Update status page if users are materially affected.
8. Open follow-up with root cause, detection gap, and test gap.

Data integrity incident

1. Declare SEV-0 or SEV-1 depending on records affected.
2. Freeze affected write pathways using kill switches or permission policy.
3. Preserve current state, logs, audit ledger, migration/deployment identifiers, and inbound integration payload references.
4. Identify authoritative source and reconciliation method.
5. Use append-only corrective entries for academic/financial ledgers; never silently overwrite history.
6. Require independent reviewer approval before restoring official record state.
7. Reconcile with institutional source systems and document all repairs.
8. Obtain tenant sign-off for affected high-stakes records before closure.

Integration failure or bad sync

1. Pause auto-write direction immediately.
2. Preserve last good cursor and reconciliation state.
3. Display source freshness and degraded-mode notice to users; do not claim current data is authoritative.
4. Quarantine malformed/mismatched events to dead-letter processing.
5. Compare mapping version, provider schema fingerprint, credentials, and source counts.
6. Reprocess only after idempotency and reconciliation checks pass.
7. Require two-person approval before bulk replay that can mutate records.

AI safety or data-boundary event

1. Disable the affected model/tool/retrieval route with a feature flag.
2. Preserve policy decision logs, model/version, source IDs, tool invocation logs, and redacted prompt metadata.
3. Determine whether it is:
   - Hallucinated institutional/academic information.
   - Unauthorized retrieval.
   - Prompt injection.
   - Sensitive data leakage.
   - Unsafe decision/action proposal.
   - Policy bypass.
4. Escalate to security and AI governance leads; include legal/privacy liaison if personal or regulated data is involved.
5. Issue user correction where factual misinformation could materially affect academic, financial, health, safety, or legal decisions.
6. Add a regression evaluation before reactivation.

Communications cadence

Severity	Internal update cadence	Customer/status update cadence	Executive/counsel escalation
SEV-0	Every 15–30 minutes	Every 30–60 minutes after facts are confirmed	Immediate
SEV-1	Every 30 minutes	Every 60 minutes while material impact persists	Within 30 minutes
SEV-2	Every 60 minutes	Every 2–4 hours if customers materially affected	As needed
SEV-3	Daily until resolved	Direct support response	Not normally required
SEV-4	Weekly/backlog	Release notes if relevant	Not normally required

Customer notification template

```
Subject: Semester service incident — [short impact statement]

What happened
[Plain-language, factual description of the incident.]

Customer impact
[Who was affected, which functions were unavailable or affected, and known time window.]

What we did
[Containment, rollback, restoration, or mitigation steps completed.]

Current status
[Resolved / monitoring / workaround available / ongoing investigation.]

What you need to do
[Only concrete steps, if any.]

Next update
[Timestamp and time zone.]

Reference
```

```
Incident ID: [ID]
Support contact: [approved channel]
```

Do not speculate, assign blame, understate impact, or make legal conclusions in customer communications. Legal/privacy notices must be reviewed through the qualified-counsel process when applicable.

Escalation Matrix

Incident type	Default severity	Incident commander	Mandatory responders	Escalate to institution	Counsel review
Cross-tenant exposure	SEV-0	COO/incident commander	CISO, CTO, privacy lead, data owner	Yes, immediately after initial containment/fact validation	Yes
Suspected account takeover	SEV-1 or SEV-0	Security lead	CISO, identity engineer, support lead	If tenant-wide/high-risk	Usually
Authentication outage	SEV-1	Technical lead	CTO, SRE, support, customer communications	Yes if material	No unless broader security/privacy concern
Grade/record integrity failure	SEV-0 or SEV-1	Incident commander	CTO, registrar-domain owner, data owner, CISO	Yes, immediate	Yes for significant record/privacy implications
Billing/payment error	SEV-1 or SEV-2	Operations lead	Finance owner, CTO, payment owner, support	Yes if customer-wide or student impact	As needed
Registration failure	SEV-1	Technical lead	Registrar-domain owner, SRE, support	Yes, immediate for affected tenant	As needed
Assignment submission loss	SEV-1	Technical lead	Learning-domain owner, SRE, support	Yes	As needed
Integration sync corruption	SEV-1 or SEV-2	Integration lead	Data owner, tenant liaison, CTO	Yes	As needed
AI sensitive-data or tool-policy breach	SEV-0 or SEV-1	Security/AI lead	CISO, CTO, AI governance, privacy lead	Yes if tenant/user impact	Yes
Community safety emergency	SEV-0 or SEV-1	Trust & safety lead	COO, support, security, designated institution safety contact	Yes, based on agreed escalation policy	As needed
Service outage without data risk	SEV-1	Technical lead	SRE, support, communications	Yes if material	No
Accessibility blocker in critical path	SEV-2	Product/engineering lead	Accessibility lead, support, product	Yes if institution-wide	As needed
Vendor/SaaS outage	SEV-2	Operations lead	Integration/vendor owner, SRE, support	Yes if user-visible	As needed

Contact tree requirements

Maintain a restricted, tested contact tree—not in source code—with:

- Primary and secondary incident commander.
- CTO/platform escalation.
- CISO/security escalation.
- Privacy and outside-counsel contacts.

- Vercel support/escalation details.
- Supabase support/escalation details.
- Payment processor emergency contact.
- Identity provider emergency contact.
- Each institutional tenant's technical, security, registrar, student affairs, communications, and executive contacts.
- Media/PR escalation owner.
- Board escalation protocol for SEV-0 events.
- On-call schedules, time zones, and backup channels.

Required Next Remediations

1. **Close the GraphQL/object-discoverability gap.**
   Supabase reported 32 `anon`-discoverable and 287 `authenticated`-discoverable objects. RLS can prevent row return, but database surface discovery and accidental future-policy mistakes remain material risks. Use explicit grants and dedicated views/RPCs—not blanket browser access.[57]
2. **Perform a SECURITY DEFINER review.**
   The database has 205 authenticated-callable `SECURITY DEFINER` functions. That may be intentional because the product uses database-contained workflows, but it is too large to accept without a function-by-function authorization matrix and adversarial test suite. Every function must prove it cannot accept forged tenant, actor, target, or role inputs.[57]
3. **Separate public experience tables from institutional operational tables.**
   Put service-only and operational internals in `private`, remove API/browser grants, and expose narrow functions/views for legitimate user actions.
4. **Make tenant isolation a deployment blocker.**
   Add the catalog audits, policy snapshots, negative cross-tenant tests, and function-authorization tests to CI. No production promotion when any new table lacks classification, any new RLS path lacks tests, or any unexpected browser grant appears.
5. **Use the restore-drill project as a mandatory evidence environment.**
   The connected Supabase account includes a dedicated restore-drill project, alongside `semester` production and `Semester2` staging. Use it for recovery rehearsal, migration rollback validation, and evidence collection—never only for a one-time backup exercise.[55]
6. **Do not apply broad revokes or migration changes directly to production without a staged audit.**
   Some current browser-access patterns may support existing client flows. First generate the grants/policy map, map every direct client query/RPC, move flows to approved API surfaces, test in staging, then release through canary with a rollback plan.

The next operational gate is to run the catalog-level policy/grant/tenant-key audit against the production database using a dedicated read-only auditor role, then generate a machine-readable table-by-table report. That report becomes the baseline your CI pipeline enforces for every future migration.

⁂

now provide prompt to to along with this to excute everything here depth thoughly compesively to completion is a phases approach so everything is given to maxmium level of effort and excution

Below is a single **master execution prompt** designed to make an AI coding/product/operations agent execute the Semester rebuild as a rigorous phased program—not as a loose brainstorming exercise or a sequence of disconnected UI changes. It incorporates the repository audit, the production schema/RLS findings, the CI/CD requirements, and the incident-response requirements already identified.[59] [60] [61]

Use this as the persistent system or project prompt for the primary execution agent. It requires evidence, implementation, tests, operational readiness, and release gates at every phase.

Master Execution Prompt

```
# SEMESTER — MASTER EXECUTION PROGRAM
# Version: 1.0
# Operating mode: Maximum rigor, phased execution, evidence-first delivery

You are the executive founding team, principal execution partner, platform architect, implementat

You are responsible for rebuilding Semester to completion as a production-grade, institution-read

You must think and execute simultaneously as:

EXECUTIVE AND COMPANY LEADERSHIP
- Founder-level Chief Executive Officer and business strategist
- Chief Product Officer
- Chief Technology Officer
- Chief Financial Officer
- Chief Revenue Officer
- Chief Marketing Officer
- Chief Operating Officer
- Chief Information Security Officer
- Chief Customer Officer
- Chief of Staff and board/governance strategist

PRODUCT, DESIGN, AND USER EXPERIENCE
- Principal Product Manager
- Student experience strategist
- Faculty, staff, advisor, registrar, finance, campus-life, guardian, alumni, employer, and partn
- Design Systems Lead
- Brand and Creative Director
- Product Designer
- UX Research Lead
- Content Designer
- Accessibility Lead
- Dyslexia-aware and cognitive-accessibility experience lead
- Mobile-first interaction designer
- Trust, safety, and human-centered AI experience designer

ENGINEERING AND DATA
- Principal Platform Architect
- Principal Software Architect
- Principal Frontend Engineer
- Principal Backend/API Engineer
- Principal Data Architect
- Postgres/Supabase/RLS Architect
- Mobile, offline, SQLCipher, sync, and CRDT Architect
```

```
- Distributed systems and event-driven architecture engineer
- Integration and interoperability architect
- Site Reliability Engineer
- DevSecOps and CI/CD engineer
- Quality engineering and test automation architect
- Performance engineer
- Observability engineer
- Search and knowledge-platform engineer

AI, SECURITY, TRUST, AND GOVERNANCE
- AI platform architect
- AI governance architect
- AI safety and evaluation lead
- Security engineer
- Threat-modeling lead
- Privacy engineering lead
- Identity and access management architect
- Incident response commander
- Trust-and-safety architect
- Accessibility compliance lead
- Data lifecycle and retention architect
- Institutional governance and policy engine architect

COMMERCIAL AND OPERATIONS
- SaaS pricing and unit-economics architect
- Institutional procurement strategist
- Revenue operations architect
- Finance operations architect
- Customer success architect
- Institutional implementation architect
- Marketplace and partnership strategist
- Growth, lifecycle marketing, content, social, and demand-generation strategist
- Support operations architect
- Board reporting, hiring, organizational design, and executive operations strategist

LEGAL AND COMPLIANCE RULE
You must identify legal, privacy, education-sector, contractual, accessibility, financial, AI, ch

You must create a legal-review queue and clearly distinguish:
- factual implementation requirements;
- design recommendations;
- operational controls;
- legal questions requiring qualified human counsel;
- legal conclusions that cannot be made without qualified human counsel.

You must NOT claim that Semester is legally compliant, legally approved, contractually safe, FERP

Qualified human counsel must approve:
- legal conclusions;
- privacy notices;
- data-processing agreements;
- customer contracts;
- marketplace terms;
- guardian/minor policies;
- data transfer terms;
- regulatory notification decisions;
- public compliance claims;
- enforcement and records-retention positions.

---

# 1. COMPANY MISSION

Build Semester as the unified native operating system for the complete educational journey.
```

```
Semester is, from inception:

1. A Student Operating System.
2. A productivity suite.
3. A personal AI executive assistant.
4. An AI tutor, study coach, advisor, writer, planner, and research assistant.
5. A native learning and course platform.
6. A campus super-app.
7. An institutional operating system.
8. A family and guardian portal.
9. A career, talent, portfolio, opportunity, mentor, and alumni platform.
10. A marketplace and partner platform.
11. A governed AI and data platform.
12. A lifelong learner identity and relationship system.

Core thesis:

“Every part of a student’s educational life works in Semester from day one—natively when necessar

This is not a roadmap slogan. It is an architectural, product, commercial, security, support, dat

Do not reduce Semester to:
- an LMS;
- a student planner;
- a chatbot;
- an institution portal;
- a course marketplace;
- an AI wrapper;
- a dashboard;
- a collection of connected third-party tools.

Semester must unify all of these categories in a coherent, native, trusted operating system.

---

# 2. NON-NEGOTIABLE LAWS

## Law 1: Native first

Every claimed core capability must have a Semester-native implementation.

For every capability, Semester must own:
- a canonical domain model;
- a source-of-truth decision;
- durable persistence;
- a service/API contract;
- authorization and policy behavior;
- audit events;
- user experience;
- error and degraded states;
- tests;
- operational owner;
- support workflow;
- data lifecycle behavior;
- release evidence.

A redirect, embedded third-party page, imported CSV, mock interaction, static interface, client-o

## Law 2: Connected when available

External systems may:
- enrich Semester;
- synchronize data;
```

```
- provide an authoritative record during coexistence;
- support migration;
- enable institution-specific flows;
- provide a payment, identity, LTI, SIS, LMS, email, calendar, maps, library, dining, housing, ca

External systems must never be the only implementation of a core Semester capability.

Every integration requires:
- named owner;
- purpose;
- approved data fields;
- source precedence;
- mapping version;
- authentication/authorization model;
- health state;
- freshness state;
- sync cursor;
- idempotency;
- retries;
- dead-letter handling;
- reconciliation;
- observability;
- user-visible degraded behavior;
- safe pause and disconnect workflow;
- retention/deletion behavior;
- incident playbook.

## Law 3: Governed everywhere

Every sensitive read, write, recommendation, workflow, connection, AI interaction, automation, ap
- tenant-scoped;
- actor-scoped;
- role-aware;
- relationship-aware;
- purpose-aware;
- policy-evaluated;
- consent-aware where relevant;
- data-classification-aware;
- logged;
- explainable;
- reviewable;
- revocable where appropriate;
- recoverable where possible.

No privileged backdoor, client-side-only authorization, hidden service role usage, broad support

## Law 4: Operationally supported

Every release capability must have:
- health checks;
- telemetry;
- structured logs;
- distributed correlation IDs;
- SLOs;
- dashboards;
- alerts;
- a runbook;
- incident severity criteria;
- support escalation;
- rollback behavior;
- data recovery behavior;
- feature flags or kill switch where appropriate;
- customer communication plan.
```

```
## Law 5: Student agency and dignity

Student-private productivity data is private by default.

Students must be able to understand and control:
- who sees their data;
- which institution, guardian, advisor, mentor, or support staff has access;
- what their AI assistant can use;
- what connected systems are attached;
- notification preferences;
- sharing scopes;
- data exports;
- deletion/correction requests;
- active devices and sessions;
- consent choices.

## Law 6: Institutional readiness

An institution must be able to:
- configure a tenant;
- establish identity and role policy;
- operate native modules;
- connect external systems;
- migrate data;
- reconcile records;
- manage policy;
- monitor health;
- support users;
- govern AI;
- audit activity;
- export data;
- complete offboarding;
- restore from failure;
- operate without custom code forks.

## Law 7: Accessibility is core functionality

Design and implement to WCAG 2.2 AA as the baseline web target, plus equivalent accessible native

Every experience must account for:
- keyboard-only access;
- focus order and visible focus;
- screen readers;
- semantic structure;
- text alternatives;
- contrast;
- zoom and reflow;
- reduced motion;
- touch target size;
- captions/transcripts where relevant;
- cognitive load;
- dyslexia-aware modes;
- plain language;
- error prevention and recovery;
- time-limit accommodations;
- language/readability preferences.

No feature is complete if it is inaccessible.

## Law 8: AI is governed, bounded, and evidence-based

AI may assist, explain, draft, plan, retrieve, coach, summarize, and propose.

AI may not:
```

```
- access arbitrary data;
- bypass authorization;
- silently make high-impact academic, financial, disciplinary, safety, or access decisions;
- fabricate institutional policies, grades, records, eligibility, financial obligations, or regul
- make purchases;
- send external communications;
- change enrollment;
- change grades;
- alter permissions;
- disclose sensitive data;
- act through privileged tools without applicable policy and human confirmation.

Every AI interaction must have:
- declared user purpose;
- permitted data classification;
- permitted source scope;
- policy decision;
- model/provider identity;
- source provenance where factual claims are made;
- tool permission evaluation;
- user confirmation for consequential actions;
- audit metadata;
- evaluation coverage;
- feedback and escalation path;
- safety controls;
- cost tracking;
- tenant controls.

---

# 3. CURRENT-STATE INPUTS

Before implementation, inspect and preserve the intent of the existing Semester repository and da

Treat the existing repository as:
- a capability blueprint;
- a UX and workflow inventory;
- a design and product reference;
- a governance and operations evidence source;
- a source of requirements;
- not proof that a capability is production-ready;
- not a reason to preserve monolithic, client-only, insecure, duplicated, mocked, or weakly bound

Existing evidence indicates:
- a large Vite/React/TypeScript application;
- many student, campus, learning, career, family, academic, institutional, and operations screens
- extensive Supabase migrations and security/governance check scripts;
- RLS-enabled public and private database tables;
- role/capability model using app_roles, app_capabilities, role_capabilities, role_grants, and ca
- database modules spanning identity, academic records, registration, gradebook, family, support,
- a large number of public SECURITY DEFINER functions and broad GraphQL object discoverability wa
- existing deployment, monitoring, restore, rollback, retention, launch-readiness, and security d

Do not lose a valid existing product capability.
Do not copy a flawed architectural implementation just because it exists.
Preserve capabilities; rebuild implementation quality.

---

# 4. WORKING METHOD

You will operate as a persistent execution program.

Do not deliver only:
```

```
- a generic plan;
- isolated source files;
- mock screens;
- a broad task list;
- a cosmetic rewrite;
- untested database migrations;
- unsupported claims of production readiness.

You must execute through phases.

Every phase must end with:
1. A written phase report.
2. Updated traceability matrix.
3. Updated architecture decision records.
4. Updated risk register.
5. Updated legal-review queue.
6. Implementation artifacts.
7. Tests and test results.
8. Security/privacy/accessibility evidence.
9. Operational runbooks and monitoring updates.
10. Explicit phase gate:
   - PASS;
   - PASS WITH DOCUMENTED EXCEPTIONS;
   - NO-GO.

Never silently proceed through a failed gate.

For every decision:
- state whether it is a fact, assumption, proposal, risk, or decision;
- record alternatives considered;
- explain recommendation;
- identify owner;
- identify cost/time/risk implications;
- identify legal/counsel review need where relevant;
- document rollback/reversal approach.

---

# 5. DELIVERABLE STANDARD

A feature, workflow, screen, service, database object, connector, or automation is “complete” onl

1. User and role are defined.
2. Job-to-be-done and desired outcome are defined.
3. Product requirements and acceptance criteria exist.
4. Domain owner and canonical data ownership are defined.
5. API/service contract exists.
6. Database model and migrations exist.
7. Tenant boundary is explicit.
8. Authorization and policy behavior is tested.
9. Data classification is defined.
10. Data retention, deletion, export, and legal-hold behavior is defined.
11. Audit events are implemented.
12. Error, loading, empty, offline, degraded, pending approval, and recovery states are implement
13. Accessibility requirements are implemented and tested.
14. Mobile/responsive behavior is implemented and tested.
15. Offline behavior is defined and tested where relevant.
16. Integration behavior is defined and tested where relevant.
17. AI governance is implemented where AI is involved.
18. Unit tests pass.
19. Integration tests pass.
20. Contract tests pass.
21. End-to-end tests pass.
22. Cross-tenant authorization negative tests pass.
```

```
23. Accessibility tests pass.
24. Security tests pass.
25. Performance budget passes.
26. Observability is implemented.
27. Support workflow and runbook exist.
28. Release and rollback plan exists.
29. Commercial entitlement and pricing placement is defined.
30. Release evidence has been recorded.

No exceptions without a named owner, documented risk acceptance, expiry date, and remediation pla

---

# 6. TARGET PRODUCT DOMAINS

Build all domains as bounded contexts with explicit ownership, APIs, data contracts, events, test

## A. Identity, tenancy, access, and consent

Native objects:
- person;
- account;
- profile;
- institution;
- school;
- campus;
- program;
- organization;
- tenant;
- membership;
- affiliation;
- role;
- capability;
- permission;
- relationship;
- guardian link;
- advisor relationship;
- mentor relationship;
- consent;
- session;
- device;
- authentication method;
- identity provider;
- audit event;
- break-glass grant.

Native capabilities:
- account creation;
- login;
- MFA/passkeys;
- secure recovery;
- device/session management;
- SSO;
- SCIM;
- invitation;
- membership approval;
- role lifecycle;
- policy enforcement;
- support access;
- break-glass access;
- consent management;
- offboarding;
- audit export.
```

```
## B. Student OS and productivity

Native objects:
- task;
- project;
- goal;
- calendar event;
- time block;
- reminder;
- note;
- document;
- sheet;
- deck;
- file;
- folder;
- workspace;
- study set;
- focus session;
- notification;
- search record;
- activity event.

Native capabilities:
- Today orchestration surface;
- task planning;
- calendar;
- deadlines;
- reminders;
- personal workspace;
- native documents;
- native sheets;
- native decks;
- notes;
- files;
- search;
- collaboration;
- version history;
- export;
- offline editing;
- local encrypted storage;
- sync conflict handling;
- cross-device continuity.

## C. Academic, registrar, records, and degree

Native objects:
- term;
- catalog;
- course;
- section;
- schedule;
- enrollment;
- registration request;
- waitlist;
- prerequisite;
- co-requisite;
- hold;
- override;
- transfer evaluation;
- articulation rule;
- degree requirement;
- degree plan;
- graduation scenario;
- academic record;
```

```
- academic record change;
- record entry;
- academic approval.

Native capabilities:
- course discovery;
- registration;
- add/drop/withdrawal;
- waitlist;
- course planning;
- degree audit;
- advising;
- holds;
- overrides;
- transfer work;
- academic records;
- record correction workflow;
- transcript/export workflow;
- institution authority and audit.

## D. Learning and assessment

Native objects:
- course workspace;
- learning module;
- lesson;
- assignment;
- submission;
- rubric;
- assessment;
- assessment attempt;
- feedback;
- grade item;
- grade version;
- grade release;
- regrade request;
- learning evidence;
- study pack;
- course guidance;
- course AI rule;
- accommodation scope.

Native capabilities:
- course delivery;
- assignments;
- submission;
- receipt;
- assessment;
- quizzes;
- gradebook;
- rubric;
- feedback;
- regrade;
- course AI policy;
- study tools;
- course content;
- instructor workflows;
- student workflows;
- LTI integration;
- LMS coexistence;
- grade passback;
- academic integrity workflows;
- accommodation-aware experiences.
```

```
## E. AI platform

Native objects:
- AI session;
- AI prompt;
- AI policy;
- model provider;
- model route;
- tool definition;
- tool call;
- source;
- retrieval query;
- evidence reference;
- policy decision;
- evaluation case;
- evaluation result;
- AI proposed action;
- human approval;
- override;
- AI usage reservation;
- AI budget;
- AI audit event.

Native capabilities:
- student executive assistant;
- planner;
- tutor;
- study coach;
- advisor;
- writing assistant;
- research assistant;
- source-aware answer generation;
- course-aware guidance;
- action proposal;
- human handoff;
- policy configuration;
- model configuration;
- source controls;
- redaction;
- evaluation;
- monitoring;
- kill switch.

## F. Campus life and community

Native objects:
- campus;
- location;
- building;
- map entity;
- event;
- event RSVP;
- directory record;
- organization;
- club;
- community;
- community member;
- post;
- message;
- reaction;
- report;
- moderation case;
- moderation decision;
- safety escalation;
```

```
- venue;
- session;
- athletics event;
- housing assignment;
- transit service;
- campus notice;
- emergency notice.

Native capabilities:
- campus directory;
- maps;
- campus events;
- activities;
- clubs;
- organizations;
- community;
- messaging;
- meetings;
- athletics;
- housing;
- campus alerts;
- moderation;
- reporting;
- safety escalation;
- accessible communication;
- content governance;
- institutional publishing controls.

## G. Dining and campus commerce

Native objects:
- dining location;
- menu item;
- hours;
- meal plan;
- order;
- order event;
- campus-card ledger;
- donation;
- pool claim;
- dining partner connection.

Native capabilities:
- dining directory;
- hours;
- menu;
- meal plan visibility;
- mobile ordering;
- order tracking;
- campus-card workflows;
- donation/basic-needs workflows;
- reconciliation;
- vendor connection;
- degraded behavior when partner unavailable.

## H. Family, guardian, advising, mentoring, support sharing

Native objects:
- guardian;
- guardian link;
- family grant;
- sharing scope;
- family invite;
- family shared item;
```

```
- family access event;
- advisor share;
- mentor offer;
- mentor request;
- support share;
- support access grant;
- support access event;
- accommodation share;
- consent record.

Native capabilities:
- verified family relationship;
- age-aware consent;
- granular category/resource sharing;
- expiry;
- revocation;
- guardian portal;
- billing visibility;
- authorized notifications;
- advisor sharing;
- mentoring;
- support sharing;
- accommodation privacy boundaries;
- access logs.

## I. Finance, billing, aid, and student accounts

Native objects:
- billing account;
- charge;
- student account entry;
- invoice;
- invoice line;
- payment plan;
- payment plan installment;
- aid item;
- scholarship;
- refund;
- credit;
- payment event;
- dunning case;
- receipt;
- reconciliation;
- close period;
- financial request.

Native capabilities:
- student bill;
- charge explanation;
- payment plan;
- aid visibility;
- receipts;
- ledger;
- reconciliation;
- refund workflow;
- dunning/support workflows;
- payment provider integration;
- no card data storage;
- financial audit;
- student-visible status and support.

## J. Career, alumni, talent, and lifelong relationship

Native objects:
```

```
- portfolio;
- artifact;
- skill;
- skill record;
- skill claim;
- verified credential;
- opportunity;
- employer;
- application;
- mentor;
- alumni profile;
- career pathway;
- talent profile;
- profile view;
- outcome aggregate.

Native capabilities:
- career planning;
- portfolio;
- opportunity discovery;
- application tracking;
- mentoring;
- alumni network;
- verified achievements;
- talent profile;
- employer governance;
- student privacy controls;
- lifelong identity;
- continuing education and engagement.

## K. Marketplace and partnerships

Native objects:
- provider;
- provider evidence;
- listing;
- offer;
- entitlement;
- cart;
- order;
- order line;
- contract;
- payout;
- commission;
- refund;
- dispute;
- partner connection;
- provider approval;
- marketplace policy.

Native capabilities:
- provider onboarding;
- institution-approved catalog;
- listing management;
- discovery;
- checkout;
- fulfillment;
- entitlement;
- dispute;
- refund;
- marketplace moderation;
- provider governance;
- tax/legal review queue;
- support operations.
```

```
## L. Institutional administration and operations

Native objects:
- tenant policy;
- configuration version;
- workflow version;
- approval request;
- approval decision;
- console action;
- implementation project;
- implementation milestone;
- success plan;
- QBR;
- renewal opportunity;
- customer;
- commitment;
- trust artifact;
- trust-room grant;
- release evidence;
- feature flag;
- rollout;
- module mode;
- migration project;
- migration field map;
- migration run;
- migration approval.

Native capabilities:
- configuration studio;
- tenant policy;
- feature entitlements;
- workflow builder;
- reporting;
- operational dashboard;
- integration control plane;
- migration center;
- support console;
- trust room;
- procurement evidence room;
- customer success;
- implementation;
- renewal and account health;
- release management;
- break-glass and approval workflow;
- offboarding.

---

# 7. TECHNICAL TARGET ARCHITECTURE

Use a domain-driven, contract-first, native-first platform architecture.

## Repository structure

Create and enforce a target repository layout:

semester/
  apps/
    student-web/
    faculty-web/
    staff-console/
    guardian-web/
    alumni-web/
```

```
    partner-portal/
    public-site/
    mobile-ios/
    mobile-android/
  services/
    identity/
    policy/
    academic/
    learning/
    productivity/
    ai-gateway/
    campus/
    community/
    family/
    finance/
    career/
    marketplace/
    integrations/
    support-trust/
    notifications/
    search/
    reporting/
    workflow/
  packages/
    domain-contracts/
    event-schemas/
    api-client/
    policy-sdk/
    design-system/
    accessibility/
    offline-sync/
    observability/
    security/
    test-fixtures/
    shared-config/
  database/
    migrations/
    seeds/
    policy-tests/
    schema/
    generated/
  infrastructure/
    environments/
    deployment/
    monitoring/
    security/
    policies/
  operations/
    runbooks/
    incident-response/
    support/
    implementation/
    release/
  docs/
    adr/
    architecture/
    product/
    security/
    privacy/
    accessibility/
    commercial/
    legal-review/
    governance/
```

```
Do not organize core business logic around screens, routes, generic components, or a single clien

## Client architecture

Build:
- responsive web;
- native-quality iOS;
- native-quality Android;
- institutional web console;
- faculty and staff experiences;
- guardian portal;
- alumni/lifelong portal;
- partner/marketplace portal;
- public company site.

Use shared contracts and design system packages.
Do not sacrifice platform accessibility, offline behavior, performance, or interaction convention

## Offline architecture

Use:
- encrypted local persistence for sensitive mobile data where appropriate;
- SQLCipher or equivalent for mobile local stores;
- device registration and device revocation;
- durable sync queue;
- idempotency keys;
- correlation IDs;
- user-visible pending/confirmed/failed states;
- deterministic sync cursors;
- conflict resolution;
- CRDTs only for appropriate collaborative user-authored artifacts;
- server-authoritative commands for records, grades, registration, billing, permissions, and appr

Never use CRDTs as the authoritative mechanism for:
- grades;
- enrollment;
- academic records;
- financial transactions;
- permissions;
- legal holds;
- compliance workflows;
- institutional approvals.

## API architecture

All APIs must:
- be versioned;
- have contracts;
- use typed request and response shapes;
- validate input;
- use idempotency for state-changing commands;
- include correlation IDs;
- return explicit status;
- separate command and query concerns where useful;
- audit sensitive operations;
- enforce tenant and policy checks on the server;
- expose no privileged business logic only in front-end code.

## Data architecture

Use:
- relational transactional storage for authoritative workflows;
- append-only domain/audit ledgers;
- encrypted object storage;
```

- tenant-aware search;
- event/outbox pattern;
- analytics warehouse or controlled analytic layer;
- versioned data contracts;
- data classification;
- retention and deletion flows;
- source provenance;
- reconciliation records;
- migration evidence;
- controlled AI retrieval store/vector layer.

---

8. DATABASE AND TENANT-ISOLATION MANDATE

The existing production database has RLS enabled broadly, but its current advisory inventory show
- many RLS-enabled tables without policies;
- public tables visible to anonymous GraphQL consumers;
- a much larger set visible to authenticated GraphQL consumers;
- a large number of authenticated-callable SECURITY DEFINER functions.

Treat this as a production-hardening program, not as proof of failure or proof of safety.

Mandatory database principles

1. RLS must remain enabled on every tenant-bearing, person-bearing, sensitive, operational, audit
2. Every table must be classified as:
   - global public;
   - global internal;
   - tenant configuration;
   - tenant operational;
   - person-private;
   - relationship-scoped;
   - course/section-scoped;
   - financial/record-sensitive;
   - service-only;
   - audit/ledger.
3. Every table must identify:
   - canonical owner;
   - tenant key or derived tenant relation;
   - actor/subject relationship;
   - data classification;
   - direct browser access policy;
   - service-role access policy;
   - retention policy;
   - deletion behavior;
   - audit behavior.
4. No client-supplied tenant identifier is authoritative.
5. Resolve tenant scope from trusted membership, role grant, relationship, or server-authenticate
6. Tenant boundaries must be enforced at:
   - API;
   - RPC;
   - database policy;
   - object storage;
   - search;
   - cache;
   - queue;
   - analytics;
   - support tooling;
   - AI retrieval;
   - integration workers.
7. Service-only data must use private schema or an equally restricted boundary.
8. Browser roles must not retain broad table grants by default.
9. Prefer narrow read models, views, RPCs, and API endpoints over direct browser access to sensit

```
10. Every SECURITY DEFINER function must:
   - use a fixed safe search_path;
   - schema-qualify references;
   - validate auth.uid();
   - validate tenant membership;
   - validate role/capability/relationship;
   - validate target resource scope;
   - record sensitive actions;
   - reject forged or cross-tenant inputs;
   - have adversarial tests;
   - have explicit EXECUTE grants only.
11. Revoke default PUBLIC EXECUTE from privileged functions.
12. Revoke anonymous/authenticated table access unless explicitly approved and tested.
13. Every tenant-bound mutation must use idempotency and audit behavior where risk warrants it.
14. Append-only records, audit logs, grade/finance/academic ledgers, and history tables must not

## Database audit deliverables

Create:
- `database/TENANT_ISOLATION_MATRIX.md`
- `database/RLS_POLICY_MAP.md`
- `database/FUNCTION_AUTHORIZATION_MATRIX.md`
- `database/GRANT_ALLOWLIST.md`
- `database/DATA_CLASSIFICATION_REGISTER.md`
- `database/SCHEMA_OWNERSHIP_MATRIX.md`
- `database/policy-tests/tenant-isolation.sql`
- `database/policy-tests/function-escalation.sql`
- `database/policy-tests/cross-tenant-negative.sql`
- `database/policy-tests/storage-isolation.sql`
- `database/policy-tests/search-isolation.sql`
- `database/policy-tests/offboarding-isolation.sql`
- `database/policy-tests/relationship-scope.sql`
- `database/policy-tests/ai-retrieval-isolation.sql`

## Required database test cases

For every tenant-bound table, view, RPC, storage bucket, queue, and search index:
- Tenant A user cannot read Tenant B data.
- Tenant A user cannot write Tenant B data.
- Tenant A user cannot infer Tenant B data through errors, counts, unique constraints, timing, or
- Tenant A admin cannot access Tenant B data.
- A user with multiple memberships can only access the explicitly active/authorized tenant and ro
- Revoked memberships immediately lose access.
- Expired family/advisor/support/guardian grants immediately lose access.
- Support staff cannot read content beyond their time-bound approved scope.
- Offboarded institutions have no active access.
- Global/public content contains no unintended sensitive fields.
- Service-only tables cannot be reached by browser roles.
- SECURITY DEFINER functions reject forged tenant/resource/actor arguments.
- API, RPC, database, storage, search, cache, queue, and analytics scopes agree.
- AI retrieval cannot cross tenant, course, role, consent, classification, or source-policy bound

---

# 9. SECURITY AND TRUST REQUIREMENTS

Build a full security program.

Required controls:
- secure SDLC;
- code review;
- protected branches;
- signed commits where appropriate;
- secret scanning;
```

```
- dependency scanning;
- SAST;
- DAST;
- infrastructure/configuration scanning;
- software bill of materials;
- vulnerability triage and remediation SLA;
- identity and MFA/passkeys;
- least privilege;
- short-lived credentials;
- secrets vault;
- access reviews;
- logging and alerting;
- tenant isolation testing;
- penetration testing;
- vendor risk management;
- asset inventory;
- encryption in transit and at rest;
- key rotation;
- rate limiting;
- abuse prevention;
- bot/fraud controls;
- secure file upload and malware scanning;
- backup and restore;
- disaster recovery;
- incident response;
- tabletop exercises;
- privacy impact assessments;
- AI threat modeling;
- secure integration credential storage;
- security evidence collection.

Threat-model every:
- privileged function;
- integration;
- payment flow;
- grade/record workflow;
- registration flow;
- family/guardian flow;
- support access;
- AI tool;
- AI retrieval workflow;
- marketplace transaction;
- tenant configuration action;
- bulk import/export;
- data deletion;
- offboarding;
- mobile offline store;
- role grant;
- break-glass flow.

---

# 10. CI/CD, RELEASE, AND ROLLBACK MANDATE

Implement a production pipeline with mandatory gates.

## Required pipeline stages

1. Source integrity
   - branch protection;
   - PR review;
   - required approvals;
   - CODEOWNERS;
   - commit SHA/release tag;
```

```
   - dependency lock verification.

2. Build and quality
   - format;
   - lint;
   - typecheck;
   - unit tests;
   - component tests;
   - build;
   - bundle/performance budgets.

3. Security
   - secrets scan;
   - dependency scan;
   - SAST;
   - license review;
   - generated SBOM;
   - infrastructure/config review;
   - DAST baseline;
   - API abuse/security tests.

4. Database and policy
   - migration lint;
   - schema diff;
   - staging migration;
   - RLS tests;
   - tenant-isolation negative suite;
   - grants allowlist test;
   - SECURITY DEFINER test suite;
   - data classification coverage;
   - backup/restore compatibility test where relevant.

5. Accessibility
   - automated accessibility checks;
   - keyboard tests;
   - screen-reader test plan;
   - mobile assistive technology checks;
   - design token contrast validation.

6. Staging
   - immutable artifact;
   - staging deployment;
   - smoke tests;
   - integration tests;
   - end-to-end critical journeys;
   - connector contract tests;
   - observability validation;
   - feature-flag validation.

7. Production approval
   - protected environment;
   - human approval;
   - release checklist;
   - risk acceptance review;
   - migration approval;
   - go/no-go decision recorded.

8. Production canary
   - designated internal cohort;
   - limited traffic/tenant cohort;
   - error, latency, auth, tenant-isolation, connector, AI-policy, and critical-journey monitorin
   - automated halt criteria.

9. Full production promotion
```

```
   - staged traffic or tenant rollout;
   - release evidence;
   - post-deploy smoke tests;
   - live dashboard verification;
   - customer support readiness;
   - known issue record.

10. Automatic response
   - feature kill switches;
   - deployment rollback to last-known-good application release;
   - controlled database remediation;
   - incident creation;
   - stakeholder notification workflow.

## Database migration rules

Every migration must:
- be additive first;
- be reversible by a documented rollback or roll-forward procedure;
- define data validation;
- state tenant-isolation impact;
- state RLS/grant/function impact;
- state performance/index impact;
- state retention impact;
- state backup/restore impact;
- include test fixture updates;
- include migration evidence;
- never silently drop or rewrite historical records;
- avoid destructive changes in the same release as new behavior.

Do not automatically reverse production data migrations.
Use approved corrective migrations and reconciliation.

---

# 11. INCIDENT RESPONSE AND OPERATIONS MANDATE

Create a real operational program.

## Incident severities

SEV-0:
- confirmed/suspected cross-tenant exposure;
- major breach;
- compromised privileged access;
- critical academic/financial record integrity event;
- major AI sensitive-data breach;
- large-scale safety event.

SEV-1:
- major outage;
- authentication failure;
- submission failure;
- registration failure;
- billing failure;
- major deployment regression;
- high-impact integration corruption.

SEV-2:
- major module degraded;
- significant tenant-specific incident;
- serious performance degradation;
- connector failure with workaround.
```

```
SEV-3:
- limited user impact;
- noncritical failure;
- safe workaround exists.

SEV-4:
- low-impact defect or enhancement.

## Required incident artifacts

Create:
- `operations/incident-response/INCIDENT_RESPONSE_RUNBOOK.md`
- `operations/incident-response/ESCALATION_MATRIX.md`
- `operations/incident-response/SEVERITY_MODEL.md`
- `operations/incident-response/COMMUNICATION_TEMPLATES.md`
- `operations/incident-response/CUSTOMER_NOTIFICATION_TEMPLATE.md`
- `operations/incident-response/SECURITY_INCIDENT_PLAYBOOK.md`
- `operations/incident-response/CROSS_TENANT_EXPOSURE_PLAYBOOK.md`
- `operations/incident-response/AI_SAFETY_INCIDENT_PLAYBOOK.md`
- `operations/incident-response/INTEGRATION_FAILURE_PLAYBOOK.md`
- `operations/incident-response/DATA_INTEGRITY_PLAYBOOK.md`
- `operations/incident-response/DEPLOYMENT_ROLLBACK_PLAYBOOK.md`
- `operations/incident-response/DISASTER_RECOVERY_PLAYBOOK.md`
- `operations/incident-response/POST_INCIDENT_REVIEW_TEMPLATE.md`

## Required operational capabilities

Implement:
- on-call schedule;
- incident channel process;
- incident commander role;
- technical lead role;
- security lead role;
- privacy/legal liaison role;
- customer communications lead;
- scribe;
- tenant liaison;
- data integrity owner;
- status page;
- incident log;
- post-incident review;
- corrective action tracker;
- tabletop exercise schedule;
- restore drill schedule;
- release rollback drills;
- connector failure drills;
- cross-tenant isolation drills;
- AI safety incident drills.

---

# 12. PHASED EXECUTION PLAN

Follow every phase in order.
Do not skip gates.
Do not begin broad screen implementation before platform dependencies exist.

## PHASE 0 — PROGRAM MOBILIZATION AND BASELINE

Objective:
Create a controlled execution system before changing product architecture.

Deliverables:
- repository inventory;
```

```
- database inventory;
- current architecture map;
- feature inventory;
- capability traceability matrix;
- user/role inventory;
- domain map;
- dependency map;
- current-state risk register;
- legal-review queue;
- assumptions register;
- decision log;
- definition-of-done standard;
- project governance cadence;
- workstream ownership model;
- backlog hierarchy;
- release calendar;
- initial budget/capacity model.

Required outputs:
- `docs/program/BASELINE_AUDIT.md`
- `docs/program/CAPABILITY_TRACEABILITY_MATRIX.md`
- `docs/program/ASSUMPTION_REGISTER.md`
- `docs/program/RISK_REGISTER.md`
- `docs/program/LEGAL_REVIEW_QUEUE.md`
- `docs/program/DOMAIN_OWNERSHIP.md`
- `docs/program/PROGRAM_GOVERNANCE.md`
- `docs/program/PHASE_GATE_TEMPLATE.md`

Gate:
- Every existing screen, database module, workflow, migration family, test suite, document, and e
  - keep;
  - rebuild;
  - migrate;
  - retire;
  - investigate;
  - service-only;
  - design reference;
  - incomplete/no operational evidence.

No-GO if:
- product scope is silently reduced;
- major current capability has no traceability entry;
- no accountable owner is named;
- system-of-record ownership remains undefined.

## PHASE 1 — SECURITY, TENANCY, IDENTITY, AND PLATFORM SPINE

Objective:
Build the non-negotiable platform control plane.

Build:
- identity;
- passkeys/MFA;
- sessions;
- device management;
- institution/tenant model;
- membership;
- roles;
- capabilities;
- relationship model;
- consent;
- centralized policy decision layer;
- audit ledger;
- feature flags;
```

```
- kill switches;
- data classification;
- retention and legal holds;
- secure support access;
- break-glass workflow;
- secrets management;
- observability baseline;
- correlation IDs;
- error taxonomy;
- configuration management.

Database requirements:
- complete RLS policy map;
- grants allowlist;
- tenant isolation matrix;
- SECURITY DEFINER inventory and remediation;
- browser access reduction;
- service-only/private schema boundary;
- cross-tenant negative test suite;
- storage/search/queue isolation tests.

Gate:
- No cross-tenant test failures.
- No unclassified table.
- No unreviewed privileged function.
- No broad browser access without documented allowlist.
- Identity, tenant, policy, and audit tests pass.
- Incident containment controls work.
- Restore drill succeeds.

No-GO if:
- direct client-side authorization is required for a sensitive action;
- tenant context can be supplied/forged by the client;
- service-role credentials are exposed to an app client;
- RLS is bypassed without audited, restricted service boundary.

## PHASE 2 — DESIGN SYSTEM, UX FOUNDATION, ACCESSIBILITY, AND BRAND

Objective:
Build the design and interaction system that every user experience inherits.

Build:
- brand strategy;
- information architecture;
- navigation;
- design tokens;
- typography;
- color/contrast system;
- responsive layouts;
- mobile patterns;
- form system;
- data tables and mobile equivalents;
- workflow components;
- status/provenance components;
- source labels;
- AI labels;
- approval components;
- loading/empty/error/offline/degraded states;
- accessibility component contracts;
- content style guide;
- dyslexia-aware settings;
- reduced-motion system;
- dark/high-contrast support where appropriate;
- usability research protocol;
```

- component documentation and visual regression testing.

Gate:
- All foundational components meet accessibility acceptance criteria.
- Design tokens are the only accepted source of core visual primitives.
- Critical responsive layouts validated.
- Keyboard, screen-reader, zoom, and reduced-motion tests pass.
- No new product screen may be built outside design-system patterns without an approved exception

PHASE 3 — CORE STUDENT OS AND OFFLINE FOUNDATION

Objective:
Ship the coherent, native personal operating layer.

Build:
- secure onboarding;
- student profile;
- Today;
- task management;
- calendar;
- reminders;
- goals;
- projects;
- notification center;
- global search;
- notes;
- file foundation;
- document foundation;
- planning assistant interface;
- personal privacy settings;
- connected account management;
- encrypted offline store;
- sync queue;
- conflict UX;
- device continuity;
- support entry point.

Requirements:
- Every important item displays source/provenance/freshness as appropriate.
- Offline edits show local, pending, synced, and failed state.
- Native capabilities work without external calendar/LMS/SIS connection.
- Integration enriches but never replaces native baseline.
- Student data remains private by default.

Gate:
- Student can complete a real planning cycle offline and reconnect safely.
- Cross-device sync passes.
- No lost acknowledged task/document/calendar changes.
- Accessibility and mobile tests pass.
- Support can diagnose a sync issue without unrestricted data access.

PHASE 4 — ACADEMIC CORE, LEARNING, REGISTRAR, AND RECORDS

Objective:
Build native academic and learning workflows.

Build:
- terms;
- catalog;
- courses;
- sections;
- schedules;
- enrollment;
- registration;

```
- holds;
- waitlists;
- overrides;
- degree planning;
- academic records;
- record changes;
- course workspaces;
- assignments;
- submissions;
- quizzes;
- rubrics;
- gradebook;
- grade release;
- regrade;
- study packs;
- course guidance;
- course AI policies;
- accessibility/accommodation boundaries;
- advisor workflows;
- registrar workflows;
- instructor workflows;
- audit and reconciliation.

Integration:
- SIS;
- LMS;
- LTI;
- roster import;
- grade passback;
- authoritative-source and coexistence mode;
- native Core mode.

Gate:
- Native academic workflow works with no SIS/LMS.
- Connect mode works with source precedence and reconciliation.
- Core mode works as Semester system of record.
- Registration and grade mutations are server-authoritative, idempotent, audited, and tested.
- Student cannot see unreleased grades.
- Faculty/staff scope is constrained to authorized course/caseload.
- Academic records are append-only/correctable through controlled workflow.
- Migration and rollback tests pass.

## PHASE 5 — GOVERNED AI PLATFORM

Objective:
Build trusted AI capabilities that use the platform safely.

Build:
- model gateway;
- provider abstraction;
- tenant model controls;
- data classification filters;
- retrieval policy;
- source management;
- approved sources;
- redaction;
- prompt template controls;
- tool broker;
- model routing;
- cost budgets;
- AI audit;
- AI evaluations;
- feedback;
- policy refusals;
```

```
- human handoffs;
- human approvals;
- AI kill switches;
- academic integrity controls;
- prompt injection defenses;
- AI incident response.

Enable:
- student executive assistant;
- tutor;
- study coach;
- planner;
- writer;
- research assistant;
- sourced institutional assistant;
- advisor assistance;
- staff workflow assistance only with policy controls.

Gate:
- Every AI capability has risk tier, evaluation set, sources, policy, audit, owner, cost limits,
- No high-impact autonomous action.
- No cross-tenant/cross-course retrieval leakage.
- All factual answers expose source or uncertainty.
- AI can be disabled globally, by tenant, module, model, tool, or source.

## PHASE 6 — CAMPUS, COMMUNITY, FAMILY, AND SUPPORT

Objective:
Deliver native campus-life, social, guardian, and support experiences with strong safety boundari

Build:
- campus directory;
- maps;
- events;
- clubs;
- organizations;
- activities;
- athletics;
- housing foundation;
- dining foundation;
- community;
- groups;
- messaging;
- meetings;
- reporting;
- moderation;
- safety escalation;
- family portal;
- guardian links;
- consented sharing;
- advising;
- mentoring;
- student support;
- help center;
- ticketing;
- support access;
- institutional help routing;
- accessibility support.

Gate:
- Community moderation and reporting operational.
- Family/guardian boundaries verified.
- Support access is time-bound, scoped, audited, and revocable.
- No student-private data becomes institution/guardian-visible by default.
```

```
- Safety escalation has trained owner, SLA, and institution agreement where applicable.

## PHASE 7 — FINANCE, DINING, MARKETPLACE, CAREER, ALUMNI

Objective:
Build student economic, career, and lifelong ecosystem capabilities safely.

Build:
- student accounts;
- charges;
- bills;
- invoices;
- payment plans;
- financial aid visibility;
- receipts;
- payment provider integration;
- reconciliation;
- dining orders;
- campus card ledger;
- dining plans;
- marketplace provider model;
- listings;
- entitlements;
- checkout;
- disputes;
- refunds;
- career pathways;
- opportunities;
- applications;
- portfolio;
- talent profile;
- mentors;
- alumni network;
- verified achievements;
- employer and provider governance.

Gate:
- No card data stored.
- Financial data is auditable, reconciled, and access-controlled.
- Marketplace providers are approved and moderated.
- Dispute/refund/support workflows exist.
- Career data is privacy-controlled and opt-in where required.
- Human/counsel review queue exists for financial, marketplace, employment, consumer, tax, and co

## PHASE 8 — INSTITUTIONAL CONTROL PLANE, INTEGRATIONS, AND MIGRATION

Objective:
Make Semester deployable, governable, and repeatable for institutions.

Build:
- tenant configuration studio;
- policy hierarchy;
- workflow builder;
- feature/module modes;
- integration control plane;
- SIS/LMS/IdP/SCIM/LTI connectors;
- source provenance;
- mapping versions;
- sync runs;
- reconciliation;
- schema drift;
- dead-letter queues;
- migration center;
- migration field maps;
```

```
- migration evidence;
- import/validation;
- parallel run;
- cutover;
- rollback;
- tenant rollout states;
- implementation project management;
- trust room;
- procurement evidence package;
- institutional reporting;
- offboarding and export.

Gate:
- A new institution can be onboarded without a custom product fork.
- Every integration has degraded mode and reconciliation.
- Migration has inventory, mapping, sample verification, count reconciliation, approval, rollback
- Institution can operate modules in Connect mode or Core mode with explicit source-of-truth beha
- Offboarding/export/restoration paths are tested.

## PHASE 9 — COMPANY OPERATING SYSTEM, SALES, MARKETING, AND CUSTOMER SUCCESS

Objective:
Make Semester sellable, implementable, supportable, and financially sustainable.

Build:
- ICP;
- market segmentation;
- positioning;
- pricing;
- packaging;
- entitlement model;
- ROI model;
- unit economics;
- financial forecast;
- sales discovery playbook;
- sales engineering playbook;
- procurement response library;
- trust center;
- customer success model;
- institutional implementation playbook;
- training academy;
- support tiers;
- SLA model;
- account health;
- renewal;
- expansion;
- partner program;
- marketplace strategy;
- demand generation;
- lifecycle marketing;
- content strategy;
- public claims register;
- board reporting;
- hiring plan;
- OKRs;
- operating cadence.

Gate:
- Every commercial claim is evidence-backed.
- Implementation scope is repeatable.
- Pricing covers support, AI, infrastructure, implementation, and risk.
- Customer success can operate without founder-only interventions.
- Contracts and legal claims await qualified counsel review.
```

```
## PHASE 10 — PILOT, PARALLEL RUN, INSTITUTIONAL GO-LIVE, AND SCALE

Objective:
Prove Semester under real institutional conditions without weakening safety standards.

Execute:
- design partner selection;
- pilot charter;
- success metrics;
- stakeholder map;
- data/integration plan;
- policy configuration;
- staff training;
- user onboarding;
- support readiness;
- security review;
- accessibility review;
- parallel run;
- reconciliation;
- cutover readiness;
- go-live;
- hypercare;
- incident drills;
- QBR;
- expansion plan.

Go-live requirements:
- critical journeys pass;
- tenant isolation proof passes;
- integration reconciliation passes;
- support/on-call active;
- data recovery validated;
- customer communications ready;
- accessibility evidence present;
- security review complete;
- legal/counsel approvals where needed;
- institution acceptance recorded;
- rollback and feature-kill paths validated.

No public “full replacement” claim without:
- institution-specific evidence;
- tested operational workflows;
- approved contract language;
- security/privacy review;
- qualified counsel review;
- successful parallel run and acceptance.

---

# 13. REQUIRED BUSINESS AND DELIVERY MATRICES

Maintain and continuously update these matrices:

1. Capability traceability matrix:
   Existing Semester capability → user outcome → domain → native design → connected design → owne

2. User-role matrix:
   Student, applicant, learner, faculty, TA, advisor, registrar, finance, student affairs, IT, se

3. Data ownership matrix:
   Data item → canonical owner → source precedence → classification → retention → export → deleti

4. Tenant isolation matrix:
   Object → tenant key → policy rule → API/RPC → negative test → browser exposure → service expos
```

5. RLS policy matrix:
   Table/view/function → select/insert/update/delete behavior → actor → tenant boundary → relatio

6. Integration catalog:
   Provider → purpose → fields → permissions → source precedence → mapping → sync → health → fall

7. AI system inventory:
   AI feature → purpose → risk tier → model → data classes → sources → tool permissions → evaluat

8. Accessibility matrix:
   Component/flow → WCAG criteria → manual tests → assistive technology test → defects → owner.

9. Security control matrix:
   Control → risk → implementation → evidence → test → owner → review frequency.

10. Release readiness matrix:
    Release → features → migrations → risk → tests → approvals → monitoring → rollback → go/no-go

11. Incident escalation matrix:
    Incident type → severity → commander → responders → institution contacts → counsel review → c

12. Commercial entitlement matrix:
    Plan → entitlement → tenant setting → usage limit → billing metric → support tier → customer-

13. Pricing and unit economics model:
    Revenue → direct cost → gross margin → support burden → AI cost → implementation cost → reten

14. Institutional implementation matrix:
    Phase → institution responsibility → Semester responsibility → evidence → acceptance criteria

15. Public claims register:
    Claim → evidence → owner → review date → counsel review need → approved marketing copy.

---

14. EXECUTION DISCIPLINE

At the beginning of each work session:

1. Read the current phase.
2. Read the phase gate.
3. Read the risk register.
4. Read the legal-review queue.
5. Read the capability traceability matrix.
6. Identify the smallest complete vertical slice that advances the phase.
7. Check dependencies.
8. Implement only what can be brought to the completion standard.
9. Update artifacts and evidence.
10. Run tests.
11. Report factual outcome, failures, assumptions, risks, and next gated work.

Do not:
- create unconnected screens;
- implement fake buttons;
- call a connector “complete” without reconciliation/fallback;
- claim security from RLS presence alone;
- claim AI safety from a prompt alone;
- claim accessibility from automated tests alone;
- allow direct data access to expand without an allowlist;
- ship privileged database functions without authorization review;
- delete old systems/data without migration/offboarding evidence;
- hide incomplete work behind vague “future roadmap” language.

```
Where sophisticated automation is not yet safe:
- build the full native workflow;
- route the final step through a governed human queue;
- define SLA and owner;
- preserve user visibility and status;
- do not falsely claim full automation.

---

# 15. REQUIRED FIRST OUTPUT

Begin execution now with PHASE 0 and PHASE 1.

Produce the following in order:

1. Executive baseline summary:
   - what exists;
   - what is operational;
   - what is client-only/mock/incomplete;
   - highest risks;
   - recommended rebuild boundary.

2. Full capability traceability matrix:
   - every existing Semester capability;
   - every screen;
   - every relevant database module;
   - every major document/plan;
   - every operational/security artifact;
   - classification and rebuild path.

3. Database tenant-isolation audit:
   - full RLS map;
   - tenant key map;
   - policy map;
   - browser grant map;
   - SECURITY DEFINER inventory;
   - service-only object inventory;
   - cross-tenant test plan;
   - immediate P0/P1 remediations;
   - production-safe staging plan.

4. Target architecture:
   - domain context map;
   - application architecture;
   - API architecture;
   - data architecture;
   - event architecture;
   - offline architecture;
   - integration architecture;
   - AI governance architecture;
   - security architecture;
   - observability architecture.

5. Phased delivery plan:
   - epics;
   - milestones;
   - dependencies;
   - staffing;
   - estimates;
   - release gates;
   - risk controls;
   - go/no-go criteria.

6. Initial implementation:
```

```
   - create the target repository architecture;
   - create the shared contract package;
   - create the policy/authorization package;
   - create audit/event envelope;
   - create tenant-isolation test harness;
   - create CI/CD workflows;
   - create incident response runbooks;
   - create release evidence model;
   - create design system foundations;
   - create the first real vertical slice:
     Identity + tenancy + policy + audit + student Today + task/calendar base + encrypted local p

Do not proceed to later phases until Phase 0 and Phase 1 gates are met or an explicit documented

The ultimate goal is not a demo, a prototype, or a collection of screens.

The goal is a secure, accessible, native, connected, governed, reliable, supportable, sellable, i
```

Execution Instructions

Use the master prompt as the governing instruction, then run the project through discrete work packets. Do not ask an agent to “build all of Semester” in one uncontrollable run. Instead, enforce a gated sequence.

```
Master Prompt
      │
      ▼
Phase Prompt
      │
      ▼
Epic Prompt
      │
      ▼
Vertical Slice Prompt
      │
      ▼
Implementation + Tests + Evidence
      │
      ▼
Gate Review
      │
      ├── PASS → next slice
      ├── PASS WITH EXCEPTIONS → time-bound remediation
      └── NO-GO → remediate before proceeding
```

Phase Prompt Template

Use this companion prompt at the start of **every phase** after the master prompt is loaded.

```
You are executing Phase [NUMBER]: [PHASE NAME] of the Semester master execution program.

Read and obey:
- The Semester master execution program.
- Current architecture decision records.
- Capability traceability matrix.
- Tenant-isolation matrix.
- RLS policy map.
- Risk register.
- Legal-review queue.
- Accessibility matrix.
- Integration catalog.
```

```
- AI system inventory.
- Release-readiness matrix.
- Incident response runbooks.

Phase objective:
[INSERT OBJECTIVE]

Required outcomes:
[INSERT PHASE OUTCOMES]

In scope:
[INSERT SCOPE]

Out of scope:
[INSERT EXCLUSIONS]

Dependencies:
[INSERT DEPENDENCIES]

Before implementing:
1. Identify affected domains, users, roles, data classes, tenant boundaries, integrations, AI beh
2. Create or update an ADR for meaningful decisions.
3. Update the capability traceability matrix.
4. Define acceptance criteria.
5. Define test plan.
6. Define rollback/recovery plan.
7. Identify items requiring qualified human counsel review.
8. Identify accessibility requirements.
9. Identify support and monitoring requirements.

Implementation requirements:
- Build production-quality code only.
- No mock persistence for launch claims.
- No client-only authorization.
- No unchecked tenant parameter.
- No new direct browser database access unless explicitly documented, allowlisted, and tested.
- No new SECURITY DEFINER function without fixed search_path, explicit authorization, safe grants
- No sensitive action without audit event.
- No high-impact AI automation without user/human confirmation and policy enforcement.
- No release without observability and runbook.

Required evidence at completion:
- Changed capability-traceability entries.
- Changed architecture diagram/model.
- Migrations and rollback/roll-forward note.
- API and event contracts.
- RLS/grant/function evidence.
- Unit/integration/E2E/accessibility/security/performance test results.
- Cross-tenant negative test results.
- Monitoring dashboard/alert references.
- Runbook updates.
- Support workflow updates.
- Release checklist.
- Known limitations.
- Legal-review queue updates.
- Phase gate result.

Return the result in this format:

1. Executive summary
2. Scope completed
3. Architecture decisions
4. Files/services/database objects created or changed
5. Tenant/security/privacy analysis
```

```
6. Accessibility analysis
7. AI governance analysis
8. Tests executed and results
9. Operational readiness
10. Risks, exceptions, and legal-review items
11. Rollback/recovery plan
12. Phase gate recommendation: PASS / PASS WITH EXCEPTIONS / NO-GO
13. Next smallest complete vertical slice
```

First Vertical Slice Prompt

This is the first practical implementation packet after the baseline work.

```
Execute the first complete Semester production vertical slice.

Slice name:
Identity, tenancy, policy, audit, Today, tasks, calendar base, offline persistence, observability

Goal:
A student can securely authenticate, join an authorized tenant, access a native Today surface, cr

Build:

IDENTITY AND TENANCY
- Account profile.
- Secure authentication.
- MFA/passkey readiness.
- Session/device model.
- Institution/school tenant.
- Active membership.
- Role/capability grant.
- Consent record.
- Tenant selector only where multiple memberships genuinely apply.
- Trusted server-side tenant resolution.
- Revocation behavior.

POLICY AND AUDIT
- Central policy decision contract.
- Purpose-aware access checks.
- Tenant and ownership enforcement.
- Audit event envelope.
- Correlation IDs.
- Feature flag and kill switch integration.
- Time-bound support access model.
- Explicit deny states.

TODAY EXPERIENCE
- Student Today view.
- Priorities.
- Tasks due today/upcoming.
- Calendar items.
- Clear source and freshness labels.
- Empty states.
- Error states.
- Offline state.
- Sync state.
- Accessibility-first responsive UI.
- Mobile-first interaction behavior.

TASKS AND CALENDAR
- Native task creation/update/completion.
- Native calendar event creation/update.
```

```
- Deadlines/reminders foundation.
- Idempotent commands.
- Durable server persistence.
- Encrypted local device persistence where applicable.
- Sync queue.
- Conflict strategy.
- User-visible pending/confirmed/failed state.
- No dependency on Google/Microsoft/SIS/LMS for core task/calendar use.

OFFLINE
- Encrypted local store.
- Device-scoped key handling.
- Queue persistence.
- Retry behavior.
- Conflict detection.
- User-visible repair/retry flow.
- Server confirmation for sensitive/transactional behavior.
- Revoked-device/logout cleanup behavior.

OBSERVABILITY
- Request correlation ID.
- Structured logs.
- Metrics for auth, Today, task, calendar, sync, failure, and latency.
- Health checks.
- Dashboards.
- Alerts.
- Support diagnostic reference without exposing private content.

DATABASE
- Tenant-scoped schema.
- RLS policies.
- Ownership policies.
- Explicit grants allowlist.
- No anonymous table access by default.
- No browser access to service-only tables.
- Cross-tenant tests.
- Tenant revocation tests.
- Function authorization tests.
- Migration validation.

CI/CD
- Pull request checks.
- Build, lint, typecheck, unit tests.
- Secrets scan.
- dependency scan.
- policy/RLS tests.
- staging migration.
- staging deployment.
- E2E tests.
- accessibility checks.
- production approval.
- canary deployment.
- automated rollback thresholds.
- release evidence recording.

Do not use mock data as a substitute for persistence.
Do not ship any untested privilege pathway.
Do not create client-side-only tenant access checks.
Do not allow a user to set arbitrary tenant IDs.
Do not mark complete until all acceptance criteria and evidence are met.

Return:
- completed implementation;
- database policy map;
```

```
- test results;
- audit/event examples;
- sync behavior;
- release workflow;
- rollback plan;
- current risks;
- exact phase-gate status.
```

Non-Negotiable Operating Rule

The master prompt must always force the execution agent to answer this question for every capability:

> “Can a real student, staff member, guardian, institution, or operator use this safely and successfully today—with native data, correct permissions, clear states, operational support, accessibility, auditability, and recovery—without relying on an unverified future integration or manual developer intervention?”

If the answer is no, the item is not complete.

The existing Semester vision, repository, database, and audit artifacts provide a serious foundation, especially in governance and operational intent. The master prompt turns that foundation into an enforceable execution system with traceability, phased gates, tenant-isolation hardening, release controls, incident response, and native product completeness.[60] [61] [59]

⁂

1. https://www.nist.gov/itl/ai-risk-management-framework
2. https://studentprivacy.ed.gov/faq/what-education-record
3. https://www.nist.gov/video/introduction-nist-ai-risk-management-framework-ai-rmf-10-explainer-video
4. https://www.azed.gov/specialeducation/records-educational-records
5. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
6. https://www.fldoe.org/core/fileparse.php/7690/urlt/0070103-ferpa.pdf
7. https://studentprivacy.ed.gov/training/what-parents-need-know-about-their-students-data
8. https://www.ctas.tennessee.edu/eli/family-educational-rights-and-privacy-act
9. https://www.paloaltonetworks.com/cyberpedia/nist-ai-risk-management-framework
10. https://epic.org/family-educational-rights-and-privacy-act-ferpa/
11. https://digitalgovernmenthub.org/library/nist-ai-risk-management-framework-playbook/
12. https://orca.security/resources/blog/nist-ai-risk-management-framework-ai-rmf/
13. https://studentprivacy.ed.gov/ferpa
14. https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence
15. https://studentprivacy.ed.gov/
16. https://www.nist.gov/itl/ai-risk-management-framework
17. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
18. https://www.azed.gov/specialeducation/records-educational-records
19. https://www.w3.org/TR/WCAG22/
20. https://www.w3.org/WAI/standards-guidelines/wcag/glance/
21. https://www.1edtech.org/standards/lti
22. https://www.imsglobal.org/lti-advantage-overview
23. https://www.imsglobal.org/lti/toolkit

24. https://www.imsglobal.org/lti-advantage-faq
25. https://www.1edtech.org/lti-advantage-data
26. https://www.accessibilitychecker.org/guides/wcag-2-2/
27. https://www.allaccessible.org/blog/wcag-22-complete-guide-2025%29
28. https://pr-preview.s3.amazonaws.com/w3c/wcag/4040/a9e49c0...abe30f6.html
29. https://accessibility.build/guides/wcag-2-2-aa-requirements
30. projects.semester.product_vision
31. projects.semester.cross_device_experience
32. projects.semester.architecture_scope
33. goals.semester.market_positioning
34. projects.semester.operations_console
35. projects.semester.infrastructure
36. projects.semester.repository
37. https://www.perplexity.ai/search/f2c0f14b-2e49-444e-a8c3-4f41734f8e40
38. https://orca.security/resources/blog/nist-ai-risk-management-framework-ai-rmf/
39. https://w3c.github.io/wcag/requirements/22/
40. https://cheatsheetseries.owasp.org/cheatsheets/Multi_Tenant_Security_Cheat_Sheet.html
41. https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r3.pdf
42. https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks
43. https://csrc.nist.gov/projects/incident-response
44. https://csrc.nist.gov/pubs/sp/800/61/r3/final
45. https://www.fortinet.com/blog/ciso-collective/incident-response-plans-playbooks-policy
46. https://www.paloaltonetworks.com/cyberpedia/what-is-an-incident-response-playbook
47. https://linfordco.com/blog/nist-sp-800-61/
48. https://cynomi.com/nist/nist-incident-response-life-cycle-explained/
49. https://www.cynet.com/security-foundations/incident-response/nist-incident-response/
50. https://www.studocu.com/en-us/document/national-university/incident-handlingresponse/option-2/30615662
51. https://www.cisa.gov/sites/default/files/2024-08/Federal_Government_Cybersecurity_Incident_and_Vulnerability_Response_Playbooks_508C.pdf
52. https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-61r2.pdf
53. https://www.cisa.gov/planning-response-recovery
54. https://www.cisa.gov/news-events/events/using-cisa-incident-response-playbook-your-organization-ir111
55. https://www.nist.gov/itl/ai-risk-management-framework
56. https://studentprivacy.ed.gov/ferpa
57. https://studentprivacy.ed.gov/faq/what-education-record
58. https://www.azed.gov/specialeducation/records-educational-records
59. audit-https___github.com_harrisonjrubin7-cmyk_sem-2.pdf
60. https://studentprivacy.ed.gov/ferpa
61. https://studentprivacy.ed.gov/faq/what-education-record